Kesh — Spécifications détaillées

Logiciel de comptabilité suisse pour indépendants et PME

Guy Corbaz

Avril 2026

Table des matières

1 Partie 1 — Product Requirements Document (PRD) 8

2 Product Requirements Document - Kesh 9
2.1 Résumé Exécutif . . . 9
2.1.1 Ce qui rend Kesh spécial . . . 9
2.2 Classification du projet . . . 10
2.3 Critères de succès . . . 10
2.3.1 Succès utilisateur . . . 10
2.3.2 Succès business . . . 10
2.3.3 Succès technique . . . 11
2.3.4 Résultats mesurables . . . 11
2.4 Parcours utilisateurs . . . 11
2.4.1 Parcours 1 — Marc, graphiste indépendant à Lausanne (utilisateur principal) . . . 11
2.4.2 Parcours 2 — Sophie, trésorière d'une association sportive à Berne (non-comptable, association) . . . 12
2.4.3 Parcours 3 — Thomas, administrateur système (admin/ops) . . . 13
2.4.4 Parcours 4 — Lisa, comptable dans une fiduciaire à Zurich (utilisateur avancé) . . . 14
2.4.5 Synthèse des capacités révélées par les parcours . . . 14
2.5 Exigences spécifiques au domaine . . . 15
2.5.1 Conformité & réglementaire . . . 15
2.5.2 Contraintes techniques . . . 15
2.5.3 Intégrations . . . 16
2.6 Innovation & Patterns Novateurs . . . 16
2.6.1 Axes d'innovation identifiés . . . 16
2.6.2 Approche de validation . . . 17
2.6.3 Atténuation des risques . . . 17
2.7 Exigences spécifiques Web App . . . 17
2.7.1 Vue d'ensemble . . . 17
2.7.2 Matrice des navigateurs . . . 17
2.7.3 Architecture frontend . . . 17
2.7.4 Design . . . 18
2.7.5 Considérations d'implémentation . . . 18
2.8 Scoping & Développement Phasé . . . 18
2.8.1 Stratégie MVP & Philosophie . . . 18

2.8.2 v0.1 — MVP (cœur comptable + facturation) . . . 18
2.8.3 v0.2 — MVP (complétude comptable) . . . 19
2.8.4 Phase 3 — Post-MVP . . . 19
2.8.5 Phase 4 — Vision future . . . 19
2.8.6 Stratégie d'atténuation des risques . . . 19
2.9 Exigences Fonctionnelles . . . 20
2.9.1 Configuration & Onboarding . . . 20
2.9.2 Gestion des Utilisateurs & Sécurité . . . 20
2.9.3 Plan Comptable & Écritures . . . 21
2.9.4 Carnet d'Adresses & Contacts . . . 21
2.9.5 Catalogue Produits/Services . . . 21
2.9.6 Facturation . . . 21
2.9.7 Paiements (v0.2) . . . 22
2.9.8 Import Bancaire & Réconciliation . . . 22
2.9.9 TVA (v0.2) . . . 22
2.9.10 Budgets (v0.2) . . . 23
2.9.11 Clôture & Exercice (v0.2) . . . 23
2.9.12 Pièces Justificatives (v0.2) . . . 23
2.9.13 Rapports & Exports . . . 23
2.9.14 Interface & Aide . . . 23
2.9.15 Internationalisation . . . 23
2.9.16 Déploiement & Maintenance . . . 24
2.9.17 Modèles de Documents (v0.2) . . . 24
2.9.18 Page d'Accueil . . . 24
2.9.19 Comptes Bancaires . . . 24
2.9.20 Lettrage (v0.2) . . . 24
2.9.21 Versioning des parseurs (v0.2) . . . 24
2.9.22 Traçabilité & Audit . . . 25
2.9.23 Résilience Frontend . . . 25
2.10 Exigences Non-Fonctionnelles . . . 25
2.10.1 Performance . . . 25
2.10.2 Sécurité . . . 25
2.10.3 Fiabilité & Intégrité . . . 25
2.10.4 Accessibilité . . . 26
2.10.5 Internationalisation . . . 26
2.10.6 Maintenabilité . . . 26
2.10.7 Déploiement . . . 26

3 Partie 2 — Architecture technique 27

4 Architecture Decision Document 28
4.1 Analyse du Contexte Projet . . . 28
4.1.1 Vue d'ensemble des Exigences . . . 28
4.1.2 Contraintes & Dépendances Techniques . . . 29
4.1.3 Préoccupations Transversales Identifiées . . . 29
4.1.4 Décisions Architecturales Préliminaires (issues de la revue collaborative) . . . 29
4.1.5 Structure Workspace Cargo . . . 31

4.1.6 Stratégie de Test . . . 31
4.2 Évaluation Starter Template . . . 31
4.2.1 Domaine Technologique Principal . . . 31
4.2.2 Versions Vérifiées (avril 2026) . . . 31
4.2.3 Options Considérées . . . 32
4.2.4 Sélection Retenue . . . 32
4.2.5 Commandes d'Initialisation . . . 32
4.2.6 Architecture de Serving . . . 32
4.2.7 Outils Recommandés . . . 33
4.2.8 Décisions Architecturales (étape 3) . . . 33
4.3 Décisions Architecturales Principales . . . 33
4.3.1 Analyse des Priorités . . . 33
4.3.2 Data Architecture . . . 34
4.3.3 Authentification & Sécurité . . . 34
4.3.4 API & Communication . . . 34
4.3.5 Frontend Architecture . . . 35
4.3.6 Infrastructure & Déploiement . . . 35
4.3.7 Analyse d'Impact des Décisions . . . 36
4.4 Patterns d'Implémentation & Règles de Cohérence . . . 36
4.4.1 Points de Conflit Identifiés . . . 36
4.4.2 Naming Patterns . . . 36
4.4.3 Structure Patterns . . . 37
4.4.4 Format Patterns . . . 38
4.4.5 Communication & Process Patterns . . . 38
4.4.6 Règles Obligatoires pour Tous les Agents AI . . . 39
4.5 Structure du Projet & Frontières . . . 39
4.5.1 Structure Complète du Répertoire . . . 39
4.5.2 Mapping Exigences → Structure . . . 43
4.5.3 Frontières Architecturales . . . 44
4.5.4 Flux de Données . . . 45
4.6 Résultats de Validation de l'Architecture . . . 45
4.6.1 Validation de Cohérence [OK] . . . 45
4.6.2 Validation de Couverture des Exigences [OK] . . . 46
4.6.3 Validation de Préparation à l'Implémentation [OK] . . . 46
4.6.4 Checklist de Complétude . . . 46
4.6.5 Évaluation de Préparation . . . 46
4.6.6 Consignes pour les Agents AI . . . 47
4.6.7 Modifications PRD à Reporter . . . 47

5 Partie 3 — Spécification UX 48

6 UX Design Specification Kesh 49
6.1 Executive Summary . . . 49
6.1.1 Vision Projet . . . 49
6.1.2 Utilisateurs Cibles . . . 49
6.1.3 Principes UX Fondateurs . . . 50
6.1.4 Défis UX Clés . . . 50
6.1.5 Opportunités UX . . . 50

6.1.6 Expérience d'Installation . . . 51
6.1.7 Flux d'Onboarding . . . 51
6.1.8 Correspondance Onboarding / Administration . . . 52
6.1.9 Bannières Contextuelles . . . 52
6.1.10 États Vides . . . 52
6.1.11 Navigation . . . 52
6.1.12 Raccourcis Clavier (v0.1) . . . 52
6.1.13 Impacts PRD Identifiés . . . 52
6.2 Core User Experience . . . 53
6.2.1 Expérience Définissante . . . 53
6.2.2 Stratégie Plateforme . . . 53
6.2.3 Interactions Sans Effort . . . 53
6.2.4 Moments Critiques de Succès . . . 54
6.2.5 Principes d'Expérience . . . 54
6.3 Desired Emotional Response . . . 54
6.3.1 Objectifs Émotionnels Primaires . . . 54
6.3.2 Cartographie du Parcours Émotionnel . . . 55
6.3.3 Micro-Émotions Critiques . . . 55
6.3.4 Implications Design . . . 56
6.3.5 Principes de Design Émotionnel . . . 56
6.4 UX Pattern Analysis & Inspiration . . . 57
6.4.1 Analyse des Produits Inspirants . . . 57
6.4.2 Patterns UX Transférables . . . 57
6.4.3 Anti-Patterns à Éviter . . . 58
6.4.4 Stratégie d'Inspiration Design . . . 59
6.4.5 Architecture Intelligence Locale . . . 59
6.5 Design System Foundation . . . 59
6.5.1 Choix du Design System . . . 59
6.5.2 Justification du Choix . . . 59
6.5.3 Approche d'Implémentation . . . 60
6.5.4 Stratégie de Personnalisation . . . 60
6.5.5 Stratégie de Test du Design System . . . 61
6.6 Expérience Utilisateur Définissante . . . 62
6.6.1 L'Interaction Définissante . . . 62
6.6.2 Modèle Mental de l'Utilisateur . . . 62
6.6.3 Critères de Succès de l'Expérience Core . . . 62
6.6.4 Patterns UX : Établis vs Novateurs . . . 63
6.6.5 Mécanique de l'Expérience Définissante . . . 63
6.7 Visual Design Foundation . . . 64
6.7.1 Système de Couleurs . . . 64
6.7.2 Système Typographique . . . 64
6.7.3 Espacements & Layout . . . 65
6.7.4 Considérations d'Accessibilité . . . 65

7 Partie 4 — Epics & Stories 67

8 Kesh - Epic Breakdown 68
8.1 Vue d'ensemble . . . 68

8.2 Inventaire des Exigences . . . 68
8.2.1 Exigences Fonctionnelles . . . 68
8.2.1.1 Configuration & Onboarding . . . 68
8.2.1.2 Gestion des Utilisateurs & Sécurité . . . 68
8.2.1.3 Plan Comptable & Écritures . . . 69
8.2.1.4 Carnet d'Adresses & Contacts . . . 69
8.2.1.5 Catalogue Produits/Services . . . 69
8.2.1.6 Facturation . . . 69
8.2.1.7 Paiements (v0.2) . . . 70
8.2.1.8 Import Bancaire & Réconciliation . . . 70
8.2.1.9 TVA (v0.2) . . . 70
8.2.1.10 Budgets (v0.2) . . . 71
8.2.1.11 Clôture & Exercice (v0.2) . . . 71
8.2.1.12 Pièces Justificatives (v0.2) . . . 71
8.2.1.13 Rapports & Exports . . . 71
8.2.1.14 Interface & Aide . . . 71
8.2.1.15 Internationalisation . . . 72
8.2.1.16 Déploiement & Maintenance . . . 72
8.2.1.17 Modèles de Documents (v0.2) . . . 72
8.2.1.18 Page d'Accueil . . . 72
8.2.1.19 Comptes Bancaires . . . 72
8.2.1.20 Lettrage (v0.2) . . . 72
8.2.1.21 Versioning des parseurs (v0.2) . . . 72
8.2.1.22 Traçabilité & Audit . . . 73
8.2.1.23 Résilience Frontend . . . 73
8.2.2 Exigences Non-Fonctionnelles . . . 73
8.2.2.1 Performance . . . 73
8.2.2.2 Sécurité . . . 73
8.2.2.3 Fiabilité & Intégrité . . . 73
8.2.2.4 Accessibilité . . . 74
8.2.2.5 Internationalisation . . . 74
8.2.2.6 Maintenabilité . . . 74
8.2.2.7 Déploiement . . . 74
8.2.3 Exigences Additionnelles (Architecture) . . . 74
8.2.3.1 Starter Template & Infrastructure . . . 74
8.2.3.2 Séparation des Responsabilités . . . 75
8.2.3.3 Authentification & API . . . 75
8.2.3.4 Frontend Architecture . . . 75
8.2.3.5 Patterns d'Implémentation Obligatoires . . . 76
8.2.3.6 Conventions de Nommage . . . 76
8.2.3.7 Stratégie de Test . . . 76
8.2.3.8 Infrastructure CI/CD . . . 76
8.2.3.9 Séquence d'Implémentation Recommandée . . . 77
8.2.4 Exigences UX Design . . . 77
8.2.4.1 Design Tokens & Fondation Visuelle . . . 77
8.2.4.2 Composants Sur Mesure à Créer . . . 77
8.2.4.3 Accessibilité . . . 78
8.2.4.4 Interaction Mode Guidé / Expert . . . 78

8.2.4.5 Flux d'Onboarding . . . 78
8.2.4.6 Navigation & Structure . . . 79
8.2.4.7 Patterns d'Interaction . . . 79
8.2.4.8 Messages & Feedback . . . 79
8.2.4.9 Expérience d'Installation . . . 80
8.2.4.10 Stratégie de Test UX . . . 80
8.2.5 Carte de Couverture des Exigences . . . 80
8.3 Liste des Epics . . . 82
8.3.1 v0.1 — Cœur comptable + facturation . . . 82
8.3.2 Epic 1 : Fondations & Authentification . . . 82
8.3.3 Epic 2 : Onboarding & Configuration . . . 82
8.3.4 Epic 3 : Plan Comptable & Écritures . . . 82
8.3.5 Epic 4 : Carnet d'Adresses & Catalogue . . . 83
8.3.6 Epic 5 : Facturation QR Bill . . . 83
8.3.7 Epic 6 : Import Bancaire & Réconciliation . . . 83
8.3.8 Epic 7 : Rapports & Exports . . . 83
8.3.9 Epic 8 : Déploiement & Opérations . . . 83
8.3.10 v0.2 — Complétude comptable . . . 83
8.3.11 Epic 9 : TVA Suisse . . . 83
8.3.12 Epic 10 : Avoirs & Paiements . . . 83
8.3.13 Epic 11 : Budgets . . . 84
8.3.14 Epic 12 : Clôture d'Exercice . . . 84
8.3.15 Epic 13 : Justificatifs, Lettrage & Compléments v0.2 . . . 84
8.4 Epic 1 : Fondations & Authentification . . . 84
8.4.1 Story 1.1 : Scaffold Cargo workspace & SvelteKit . . . 84
8.4.2 Story 1.2 : Docker & configuration . . . 84
8.4.3 Story 1.3 : Types core (newtypes & validation) . . . 85
8.4.4 Story 1.4 : Schéma de base & repository pattern . . . 85
8.4.5 Story 1.5 : Authentification (login/logout/JWT) . . . 86
8.4.6 Story 1.6 : Refresh token & gestion de session . . . 86
8.4.7 Story 1.7 : Gestion des utilisateurs (CRUD) . . . 86
8.4.8 Story 1.8 : RBAC & verrouillage optimiste . . . 87
8.4.9 Story 1.9 : Design system & tokens . . . 87
8.4.10 Story 1.10 : Layout & page de login . . . 88
8.4.11 Story 1.11 : Wrapper fetch & accessibilité . . . 88
8.5 Epic 2 : Onboarding & Configuration . . . 88
8.5.1 Story 2.1 : Internationalisation (i18n) backend . . . 88
8.5.2 Story 2.2 : Flux d'onboarding — Chemin A (Exploration) . . . 89
8.5.3 Story 2.3 : Flux d'onboarding — Chemin B (Production) . . . 89
8.5.4 Story 2.4 : Page d'accueil & navigation . . . 90
8.5.5 Story 2.5 : Mode Guidé / Expert . . . 90
8.6 Epic 3 : Plan Comptable & Écritures . . . 91
8.6.1 Story 3.1 : Plan comptable (chargement & gestion) . . . 91
8.6.2 Story 3.2 : Saisie d'écritures en partie double . . . 91
8.6.3 Story 3.3 : Modification & suppression d'écritures . . . 92
8.6.4 Story 3.4 : Recherche, pagination & tri . . . 92
8.6.5 Story 3.5 : Notifications, aide contextuelle & audit . . . 93
8.7 Epic 4 : Carnet d'Adresses & Catalogue . . . 93

8.7.1 Story 4.1 : Carnet d'adresses (CRUD contacts) 93
8.7.2 Story 4.2 : Conditions de paiement & catalogue produits/services . 94
8.8 Epic 5 : Facturation QR Bill 94
8.8.1 Story 5.1 : Création de factures (brouillon) 94
8.8.2 Story 5.2 : Validation & numérotation des factures 95
8.8.3 Story 5.3 : Génération PDF QR Bill 95
8.9 Epic 6 : Import Bancaire & Réconciliation 95
8.9.1 Story 6.1 : Import CAMT.053 96
8.9.2 Story 6.2 : Import CSV (multi-encodage & profils banque) 96
8.9.3 Story 6.3 : Détection de doublons & rejet partiel 96
8.9.4 Story 6.4 : Réconciliation & matching automatique 97
8.9.5 Story 6.5 : Réconciliation manuelle & règles d'affectation 97
8.10 Epic 7 : Rapports & Exports 98
8.10.1 Story 7.1 : Rapports comptables (bilan, résultat, balance, journaux) 98
8.10.2 Story 7.2 : Export PDF & CSV 98
8.11 Epic 8 : Déploiement & Opérations 99
8.11.1 Story 8.1 : Docker-compose & Dockerfile production 99
8.11.2 Story 8.2 : Migrations automatiques & détection de version 99
8.11.3 Story 8.3 : Résilience frontend (DB inaccessible) 99
8.11.4 Story 8.4 : CI/CD GitHub Actions 100
8.11.5 Story 8.5 : Guide d'installation PDF 100
8.12 Epic 9 : TVA Suisse 101
8.12.1 Story 9.1 : Configuration des taux TVA 101
8.12.2 Story 9.2 : Calcul TVA & rapport par période 101
8.13 Epic 10 : Avoirs & Paiements 101
8.13.1 Story 10.1 : Avoirs (notes de crédit) 102
8.13.2 Story 10.2 : Génération de fichiers de paiement pain.001 102
8.13.3 Story 10.3 : Flux de paiement en deux temps 102
8.14 Epic 11 : Budgets 103
8.14.1 Story 11.1 : Budgets annuels & suivi 103
8.15 Epic 12 : Clôture d'Exercice 103
8.15.1 Story 12.1 : Clôture & report des soldes 103
8.16 Epic 13 : Justificatifs, Lettrage & Compléments v0.2 104
8.16.1 Story 13.1 : Pièces justificatives 104
8.16.2 Story 13.2 : Lettrage 104
8.16.3 Story 13.3 : Versioning parseurs, modèles & manuels 105

Chapitre 1

Partie 1 — Product Requirements Document (PRD)

stepsCompleted : - step-01-init - step-02-discovery - step-02b-vision - step-02c-executive-summary - step-03-success - step-04-journeys - step-05-domain - step-06-innovation - step-07-project-type - step-08-scoping - step-09-functional - step-10-nonfunctional - step-11-polish - step-12-complete inputDocuments : - docs/kesh-prd-v0.2.md - docs/change_request.md documentCounts : briefs : 0 research : 0 brainstorming : 0 projectDocs : 2 workflowType : ‘prd’ classification : projectType : Web App (SPA Svelte + API Axum) domain : Comptabilité & gestion PME suisse (mini-ERP) complexity : high projectContext : brownfield —

Chapitre 2

Product Requirements Document - Kesh

Author : Guy **Date :** 2026-04-01

2.1 Résumé Exécutif

Kesh est un logiciel de comptabilité et de gestion destiné aux indépendants, TPE et petites associations en Suisse. Gratuit et open source (EUPL 1.2), il s'auto-héberge sur l'infrastructure de l'utilisateur — NAS, VPS ou serveur local — via Docker. Les données comptables restent intégralement sous le contrôle de l'utilisateur, sans transmission à un tiers.

L'application web implémente nativement les standards suisses : facturation QR Bill 2.2, paiements pain.001.001.09.ch.03, import de relevés CAMT.053.001.04, plan comptable helvétique et TVA suisse à taux configurables avec historique. L'interface est disponible en français, allemand, italien et anglais.

Face aux solutions cloud par abonnement et aux logiciels locaux vieillissants, Kesh offre une alternative moderne, gratuite et souveraine. Les standards bancaires suisses ISO 20022 sont aujourd'hui matures et Docker démocratise l'auto-hébergement : le moment est opportun pour une solution open source qui n'existait pas jusqu'ici. L'installation via docker-compose et un onboarding assisté permettent d'être opérationnel en moins de 15 minutes.

2.1.1 Ce qui rend Kesh spécial

- **Gratuit et open source** — Aucun abonnement. Code source ouvert sous licence EUPL 1.2. Seul le support est payant.
- **Auto-hébergé** — Les données comptables restent sur l'infrastructure de l'utilisateur. Aucune donnée transmise à un tiers.
- **Standards suisses natifs** — QR Bill, pain.001, CAMT.053, plan comptable PME/association, TVA configurable. Conçu pour la Suisse dès le premier jour.

— **Multilingue natif** — FR, DE, IT, EN. L'interface et les documents sont dans la langue de l'instance.
— **Pensé pour des utilisateurs sans formation comptable spécialisée** — Onboarding assisté, plan comptable pré-configuré, aide contextuelle intégrée.

2.2 Classification du projet

Critère	Valeur
Type	Web App (SPA Svelte + API REST Axum)
Domaine	Comptabilité & gestion PME suisse
Complexité	Haute — standards SIX/ISO 20022, TVA suisse, comptabilité en partie double
Contexte	Brownfield — PRD v0.2 existant, 3 change requests, 50+ décisions structurantes
Licence	EUPL 1.2

2.3 Critères de succès

2.3.1 Succès utilisateur

— Préparer et émettre une facture QR Bill (lignes libres ou produits/services, validation, génération PDF) en moins de 5 minutes
— Importer un relevé bancaire (CAMT.053/CSV) et réconcilier les transactions connues en quelques clics
— Clôturer un exercice et produire un bilan/compte de résultat sans assistance externe
— Être opérationnel (onboarding + première saisie) en moins de 15 minutes depuis le premier lancement
— Établir un budget annuel et suivre les écarts avec le réalisé tout au long de l'exercice
— Utiliser le logiciel sans formation comptable spécialisée grâce à l'aide contextuelle

2.3.2 Succès business

— L'auteur gère sa propre comptabilité effective dans Kesh pendant au moins un exercice complet (12 mois)
— Le logiciel fonctionne de manière fiable pour une utilisation quotidienne en conditions réelles
— Des utilisateurs adoptent Kesh pour leur comptabilité effective (pas uniquement en test)
— Le projet open source attire des retours constructifs de la communauté

2.3.3 Succès technique

- **Intégrité comptable** — La balance débit/crédit est toujours juste, aucun écart possible
- **Zéro perte de données** — Écritures, factures et justificatifs toujours intègres
- **Intégrité des imports** — Aucune transaction en doublon suite à un import multiple ou un chevauchement de relevés
- **Standards conformes** — Les fichiers pain.001 passent la validation du schéma XSD officiel SIX. Les QR Bill sont conformes aux spécifications SIX.
- **Performance** — Import d'un relevé mensuel (200 transactions) en < 2s, pages < 300ms
- **Disponibilité** — L'application redémarre proprement après un crash ou une coupure
- **Sécurité** — Aucune donnée accessible sans authentification valide. Mots de passe hashés (argon2/bcrypt), jamais stockés en clair.

2.3.4 Résultats mesurables

- Un indépendant peut gérer sa comptabilité annuelle complète dans Kesh : saisie, facturation, paiements, import bancaire, TVA, clôture
- Les PDF QR Bill générés sont conformes aux spécifications SIX et scannables
- Les fichiers pain.001 passent la validation XSD officielle SIX
- Le décompte TVA généré correspond aux montants vérifiés manuellement
- Le bilan et le compte de résultat générés correspondent aux résultats vérifiés manuellement sur un jeu de données de référence

2.4 Parcours utilisateurs

2.4.1 Parcours 1 — Marc, graphiste indépendant à Lausanne (utilisateur principal)

Scène d'ouverture : Marc est graphiste indépendant depuis 3 ans. Chaque mois, il paie un abonnement pour un logiciel comptable cloud qu'il utilise à 10% de ses capacités. Il envoie 5-10 factures par mois, importe son relevé PostFinance, et une fois par an il exporte ses chiffres pour son fiduciaire. Il aimerait quelque chose de simple, gratuit, et hébergé sur son NAS Synology.

Action montante : Marc découvre Kesh. Il copie trois lignes dans son docker-compose, lance l'application. L'assistant d'onboarding lui demande : « Vous êtes indépendant ? ». Il saisit ses coordonnées, son QR-IBAN, et Kesh installe le plan comptable adapté. En 10 minutes, il est prêt.

Il crée son premier client dans le carnet d'adresses, prépare une facture avec deux lignes depuis son catalogue de prestations (« Création logo — 1'500 CHF », « Retouches photo — 500 CHF »), et valide. Le PDF QR Bill s'ouvre dans un nouvel onglet. Le QR Code est conforme. Il télécharge et envoie par email.

En début d'année, Marc saisit son budget prévisionnel : revenus attendus par mois, charges fixes (loyer atelier, assurances), charges variables (logiciels, matériel). Chaque

trimestre, il consulte le rapport budget vs réalisé et ajuste ses dépenses.

En fin de mois, il importe son relevé PostFinance (CAMT.053). Les paiements de ses factures sont automatiquement proposés pour réconciliation. Les frais bancaires et un paiement carte arrivent dans la file — il crée les contreparties en deux clics. Kesh lui propose de créer des règles pour la prochaine fois.

Scénario d'avoir : Un client de Marc conteste une facture validée de 1'500 CHF — le travail livré ne correspondait pas à la demande. Marc ne peut pas supprimer la facture validée. Il crée un avoir depuis la séquence dédiée, lié à la facture d'origine. L'avoir génère automatiquement l'écriture de contre-passation. Le solde du client revient à zéro.

Scénario d'erreur : Marc prépare un lot de paiements fournisseurs et génère le fichier pain.001. Mais il a saisi un IBAN invalide pour un fournisseur. Kesh valide le format IBAN avant la génération et signale l'erreur : « IBAN invalide pour le fournisseur Müller — paiement exclu du lot ». Marc corrige l'IBAN dans le carnet d'adresses et régénère le fichier. Les autres paiements du lot ne sont pas impactés.

Climax : En fin de trimestre, Marc génère son rapport TVA. Les montants correspondent. Il remplit le formulaire sur le portail de l'AFC en 5 minutes. En fin d'année, il clôture l'exercice. Le bilan et le compte de résultat sont générés en PDF. Le rapport budget vs réalisé annuel montre qu'il a dépassé son budget matériel de 15% — il en tient compte pour le budget de l'année suivante. Son fiduciaire valide les comptes sans remarque.

Résolution : Marc ne paie plus d'abonnement. Ses données sont sur son NAS. Sa comptabilité lui prend 30 minutes par mois au lieu de 2 heures. Il recommande Kesh à ses collègues indépendants.

2.4.2 Parcours 2 — Sophie, trésorière d'une association sportive à Berne (non-comptable, association)

Scène d'ouverture : Sophie est bénévole comme trésorière du club de volleyball de son quartier. Elle n'a aucune formation comptable. Jusqu'ici, elle gérait les comptes sur un tableur Excel — erreurs fréquentes, pas de bilan propre pour l'AG.

Action montante : Le président de l'association installe Kesh sur le serveur du club. Il crée deux comptes : un admin pour lui, un comptable pour Sophie. Sophie se connecte et découvre un plan comptable « association » pré-configuré. Les tooltips l'aident à comprendre les termes comptables.

Elle commence sa première saisie d'écriture. « Débit ? Crédit ? » — elle hésite. Le tooltip lui explique : « Débit = l'argent entre dans ce compte, Crédit = l'argent sort de ce compte ». Elle saisit les cotisations des membres, les factures du gymnase, les recettes de la buvette. Pour le gymnase, elle crée le fournisseur dans le carnet d'adresses et saisit la facture reçue avec le justificatif PDF scanné.

En début d'année, Sophie saisit le budget voté par l'AG : cotisations attendues, subventions, dépenses prévues (location gymnase, équipement, tournois). Elle crée une version « Budget AG » qui servira de référence.

Scénario d'écriture déséquilibrée : Sophie saisit une écriture de 500 CHF au débit

mais oublie la contrepartie crédit. Kesh refuse la validation : « Écriture déséquilibrée — le total des débits (500.00) ne correspond pas au total des crédits (0.00) ». Sophie comprend et ajoute la ligne crédit manquante.

Scénario de doublons : Sophie importe le relevé bancaire de décembre, mais elle avait déjà importé les derniers jours de décembre avec le relevé de novembre. Kesh détecte les doublons et les signale — aucune transaction en double. Elle souffle de soulagement.

Scénario de concurrence : Le trésorier adjoint de l'association se connecte en même temps que Sophie. Il modifie le libellé d'une écriture que Sophie est en train de corriger. Sophie sauvegarde — Kesh affiche une modal : « Cette écriture a été modifiée par un autre utilisateur. Voulez-vous recharger ? ». Sophie recharge et voit la modification du trésorier adjoint. Elle applique sa correction par-dessus. Pas de perte de données.

Scénario de session expirée : Sophie part en pause café. 20 minutes plus tard, elle clique sur « Sauvegarder » — une modal apparaît : « Session expirée — veuillez vous reconnecter ». Elle se reconnecte. Le formulaire qu'elle remplissait est perdu — elle devra le ressaisir. Une leçon apprise : sauvegarder régulièrement.

Climax : À l'AG, Sophie présente le bilan, le compte de résultat ET le comparatif budget vs réalisé générés par Kesh. Les membres voient immédiatement que les dépenses d'équipement ont dépassé le budget de 800 CHF, compensées par des recettes de buvette meilleures que prévu. C'est la première fois que les comptes sont clairs et professionnels. Le budget de l'année suivante est voté sur la base du réalisé.

Résolution : Sophie gère désormais les comptes de l'association avec confiance. Le président, en mode consultation, peut vérifier les soldes à tout moment sans risque de modifier quoi que ce soit.

2.4.3 Parcours 3 — Thomas, administrateur système (admin/ops)

Scène d'ouverture : Thomas est informaticien. Un ami indépendant et une association locale lui demandent d'installer Kesh. Il gère deux instances sur son VPS.

Action montante : Thomas crée deux docker-compose distincts, chacun avec sa propre base MariaDB. Il configure les variables d'environnement (`KESH_ADMIN_PASSWORD`, `KESH_LANG`), le reverse proxy nginx avec TLS, et lance les containers. L'endpoint `/health` lui confirme que tout est opérationnel.

Il crée les comptes utilisateurs pour chaque instance — un admin et un comptable pour l'ami indépendant, un admin, un comptable et un compte consultation pour l'association. Il configure la politique de mot de passe (12 caractères minimum).

Scénario de changement de mot de passe : Le comptable de l'ami indépendant souhaite changer son mot de passe initial. Depuis son profil, il saisit l'ancien mot de passe, puis le nouveau. Kesh vérifie la politique de mot de passe (12 caractères minimum) et confirme le changement par une banner de succès.

Scénario de mise à jour : Thomas télécharge la nouvelle image Docker. Au redémarrage, Kesh détecte la nouvelle version et affiche « Backup recommandé avant de continuer ». Thomas fait un dump MariaDB, confirme, et les migrations s'appliquent.

Mais l'instance de l'association ne démarre pas — le healthcheck `/health` retourne 503. Les logs Docker montrent une erreur de connexion MariaDB : le container de base de données n'a pas fini de redémarrer. Thomas attend 10 secondes, le healthcheck repasse au vert. Fausse alerte, mais les logs étaient clairs.

Un jour, le comptable de l'association oublie son mot de passe. Thomas le réinitialise depuis l'interface admin. Le trésorier quitte l'association — Thomas désactive son compte. L'historique de ses actions reste intact.

Résolution : Thomas maintient deux instances Kesh avec un effort minimal. Les backups MariaDB sont automatisés via son NAS. Les mises à jour se font en 5 minutes.

2.4.4 Parcours 4 — Lisa, comptable dans une fiduciaire à Zurich (utilisateur avancé)

Scène d'ouverture : Lisa travaille dans une petite fiduciaire à Zurich. Elle gère la comptabilité de 8 clients indépendants et TPE. Chacun a sa propre instance Kesh installée par leur informaticien.

Action montante : Lisa se connecte à l'instance de chaque client avec son compte « Comptable ». Elle connaît bien Kesh — les raccourcis, les règles d'affectation. Pour chaque client, elle importe le relevé bancaire mensuel. Les règles d'affectation qu'elle a créées au fil des mois font que 80% des transactions sont pré-affectées. Elle valide en lot, traite les exceptions manuellement.

Elle prépare les paiements fournisseurs pour un client : sélectionne les factures ouvertes, génère le fichier pain.001. Le client l'upload dans son e-banking.

Scénario d'import partiel : Lisa importe un fichier CSV d'une banque en ligne pour un nouveau client. Le format est différent de ce qu'elle connaît — les colonnes ne correspondent pas au profil standard. Kesh rejette partiellement l'import : « 12 transactions importées, 3 rejetées — format de date non reconnu en lignes 45, 67, 89 ». Lisa consulte les lignes en erreur, crée un profil banque personnalisé pour ce format, et réimporte les 3 lignes manquantes.

Climax : Fin d'année. Lisa clôture les exercices de ses 8 clients. Pour chacun : rapport TVA du dernier trimestre, écriture de clôture, report des soldes, génération du bilan et du compte de résultat. Les PDF sont conformes et professionnels. Ses clients sont satisfaits.

Résolution : Lisa recommande Kesh à ses clients parce que c'est gratuit pour eux et qu'elle travaille efficacement. La fiduciaire propose le support Kesh comme service payant.

2.4.5 Synthèse des capacités révélées par les parcours

Parcours	Capacités clés	Scénarios d'erreur couverts
Marc (indépendant)	Onboarding, catalogue, facturation QR Bill, import CAMT, réconciliation, règles d'affectation, TVA, budget, clôture, rapports	Validation IBAN pain.001, création d'avoir sur facture contestée
Sophie (association)	Multi-utilisateurs/rôles, plan comptable association, justificatifs, aide contextuelle, budget AG, mode consultation	Écriture déséquilibrée, doublons import, verrouillage optimiste, session expirée
Thomas (admin)	Docker-compose, multi-instances, healthcheck, gestion utilisateurs, politique MdP, migration/mise à jour, désactivation compte	Healthcheck 503, changement de mot de passe
Lisa (fiduciaire)	Efficacité experte, règles d'affectation, batch paiements, profils banque, clôture multi-clients	Import CSV partiel, profil banque personnalisé

Correspondance tests E2E Playwright : Chaque parcours et scénario d'erreur constitue directement un scénario de test de bout en bout.

2.5 Exigences spécifiques au domaine

2.5.1 Conformité & réglementaire

- **Code des obligations suisse (CO art. 957-964)** — Intégrité des écritures, conservation obligatoire 10 ans. Jamais de suppression automatique de données comptables, même archivées.
- **Standards SIX** — QR Bill 2.2 (conformité stricte : dimensions 46x46mm, position A4, police, croix suisse), pain.001.001.09.ch.03 (validation XSD), CAMT.053.001.04
- **TVA suisse (AFC)** — Taux configurables avec historique et dates de validité, méthode effective/forfaitaire, arrondis commercial au centime
- **Numéro IDE (CHE)** — Validation format et checksum sur le carnet d'adresses (CHE-123.456.789)
- **nLPD** — Post-MVP (export/anonymisation des données personnelles)

2.5.2 Contraintes techniques

- **Arithmétique exacte** — `rust_decimal` pour tous les montants, jamais de flottants (f64)

- **Intégrité comptable** — Balance débit/crédit toujours équilibrée, validation à chaque saisie
- **Immutabilité post-clôture** — Aucune modification des écritures après clôture d'exercice, uniquement contre-passation
- **Formats suisses** — Apostrophe comme séparateur de milliers (`1'234.56`) dans tous les PDF et rapports, dates en `dd.mm.yyyy`
- **Encoding** — Support UTF-8 et ISO-8859-1 pour l'import CSV (détection automatique)
- **Sécurité** — Hash des mots de passe (argon2/bcrypt), JWT avec refresh silencieux
- **Migrations rétrocompatibles** — Les données des exercices passés doivent rester lisibles après mise à jour du schéma
- **Justificatifs sur filesystem** — Les pièces justificatives sont stockées dans un volume Docker dédié (pas en BDD). La stratégie de backup doit couvrir le dump MariaDB et le volume de fichiers. Sur NAS, les deux sont inclus dans l'outil de backup natif. Sur VPS, un script de backup documenté couvre les deux.

2.5.3 Intégrations

- Formats bancaires suisses via fichiers (CAMT.053, pain.001) — pas d'API directe avec les banques
- QR-IBAN pour la facturation QR Bill
- Plans comptables suisses standards (PME, association, indépendant)
- Fichiers de test officiels SIX pour la validation en CI

2.6 Innovation & Patterns Novateurs

2.6.1 Axes d'innovation identifiés

Innovation de positionnement : Kesh est la première solution open source, gratuite et auto-hébergée pour la comptabilité suisse. Cette combinaison n'existe pas sur le marché actuel. L'innovation n'est pas technologique mais dans le modèle de distribution et le positionnement.

Innovation de distribution : Docker-compose rend l'auto-hébergement accessible à des non-développeurs. Un NAS Synology ou un VPS simple suffisent. Le « time to value » de 15 minutes est un différenciateur fort.

Innovation IA (post-MVP) : Intégration optionnelle d'intelligence artificielle pour assister la gestion comptable : - Classification automatique des écritures (ex. : facture assurance → compte assurances) - Catégorisation intelligente des transactions importées - Détection d'anomalies comptables - Aide à la saisie par suggestion contextuelle

Principes IA : - Fonctionnalité strictement optionnelle — Kesh fonctionne parfaitement sans IA - IA auto-hébergée (modèle local) : aucune restriction, données restent locales - IA externe (Claude, OpenAI, etc.) : avertissement explicite que des données comptables seront transmises à un tiers, consentement utilisateur requis

2.6.2 Approche de validation

- L'innovation de positionnement se valide par l'adoption : des utilisateurs choisissent Kesh plutôt qu'une solution payante
- L'innovation IA se validera en post-MVP par la précision de la classification automatique comparée aux règles d'affectation manuelles

2.6.3 Atténuation des risques

Risque	Mitigation
L'auto-hébergement freine l'adoption	Docker-compose simplifie au maximum, documentation claire
L'IA externe compromet la confidentialité	Avertissement explicite, consentement requis, IA locale privilégiée
L'IA locale manque de performance	Fonctionnalité optionnelle, les règles d'affectation manuelles restent disponibles

2.7 Exigences spécifiques Web App

2.7.1 Vue d'ensemble

Kesh est une Single Page Application (SPA) Svelte communiquant avec une API REST Axum. L'application est conçue pour un usage desktop/laptop exclusivement — pas de responsive mobile.

2.7.2 Matrice des navigateurs

Navigateur	Support
Chrome	2 dernières versions
Firefox	2 dernières versions
Safari	2 dernières versions
Edge	2 dernières versions
Mobile	Non supporté

2.7.3 Architecture frontend

- **SPA Svelte** — navigation côté client, pas de rendu serveur
- **API REST** — communication JSON avec le backend Axum, préfixe `/api/v1/`
- **Pas de temps réel** — requêtes HTTP classiques, verrouillage optimiste pour la concurrence
- **Pas de SEO** — application derrière authentification, pas d'indexation nécessaire. Site vitrine via GitHub Pages si besoin.

2.7.4 Design

- **Desktop uniquement** — largeur minimale 1280px
- **Accessibilité** — inspiré WCAG AA (contraste, navigation clavier, labels) sans contrainte stricte
- **Notifications** — banner (succès/warning/erreur), modal (erreur bloquante)
- **Aide contextuelle** — tooltips sur les termes comptables et champs techniques

2.7.5 Considérations d'implémentation

- Frontend servi en fichiers statiques par nginx (production) ou Axum directement (développement)
- Authentification JWT avec refresh silencieux, expiration 15 min d'inactivité
- Internationalisation côté backend (API expose les traductions), langue configurée au niveau de l'instance
- Formats suisses : dates `dd.mm.yyyy`, montants avec apostrophe `1'234.56`

2.8 Scoping & Développement Phasé

2.8.1 Stratégie MVP & Philosophie

Approche MVP : Résolution de problème — le minimum pour qu'un indépendant ou une association puisse gérer sa comptabilité suisse de bout en bout.

Ressources : Développeur solo avec Claude Code. Le phasage en v0.1/v0.2 permet d'avoir un produit utilisable rapidement pour maintenir la motivation.

2.8.2 v0.1 — MVP (cœur comptable + facturation)

Parcours utilisateurs supportés : Marc (indépendant) et Sophie (association) — parcours de base sans TVA ni budgets.

Fonctionnalités : - Configuration système et onboarding assisté (type d'orga, coordonnées, comptes bancaires) - Compte admin initial (username `admin`, mot de passe via variable d'env) - Authentification multi-utilisateurs avec RBAC (Admin / Comptable / Consultation) - Politique de mot de passe configurable, désactivation d'utilisateurs - Verrouillage optimiste pour la concurrence - Plan comptable suisse (PME, association, indépendant) chargeable et personnalisable - Saisie comptable en partie double, journaux (Achats, Ventes, Banque, Caisse, OD) - Carnet d'adresses unifié (personnes/entreprises, flag client/fournisseur) - Catalogue de produits/services (nom, description, prix unitaire, TVA) - Facturation QR Bill 2.2 avec lignes libres ou depuis catalogue, numérotation configurable - Conditions de paiement configurables - Multi-comptes bancaires - Import de relevés bancaires CAMT.053, CSV (parseur tolérant + profils banque) - Détection de doublons à l'import (hash fichier + vérification par transaction) - Réconciliation bancaire : flux unique, matching automatique, création manuelle de contrepartie - Rapports : bilan, compte de résultat, balance, journaux (PDF, CSV) - Recherche, pagination, tri et filtres - Notifications UX : banner (succès/warning/erreur) + modal (erreur

bloquante) - Aide contextuelle (tooltips) - Disclaimer légal « ne remplace pas un fiduciaire » - Healthcheck endpoint `/health` - Rate limiting sur le login - Script de seed rechargeable (démo/test) - Déploiement docker-compose (Axum + MariaDB, 2 containers, reverse proxy TLS optionnel) - Langues : FR, DE, IT, EN

2.8.3 v0.2 — MVP (complétude comptable)

Parcours utilisateurs complétés : Tous les parcours (Marc, Sophie, Thomas, Lisa) avec tous les scénarios.

Fonctionnalités : - TVA suisse à taux configurables avec historique, arrondis commercial au centime par ligne, rapport par période - Budgets annuels par compte individuel avec montants mensuels, versions multiples, rapport comparatif budget vs réalisé - Avoirs (notes de crédit) avec séquence de numérotation séparée - Clôture d'exercice avec report des soldes + bilan d'ouverture - Génération de fichiers de paiement pain.001.001.09.ch.03 - Pièces justificatives attachées aux écritures (filesystem, volume Docker dédié) - Lettrage - Export CSV global par table - Versioning des parseurs/générateurs SIX - Détection de nouvelle version au démarrage avec avertissement backup avant migration - 3 manuels PDF embarqués (démarrage, utilisateur, administrateur)

2.8.4 Phase 3 — Post-MVP

- Dashboard configurable
- Gestion de stocks simplifiée
- Import MT940 (SWIFT legacy)
- Export XLSX
- Réconciliation automatique des paiements batch agrégés
- Envoi de factures par email
- Romanche (RM)
- Reset mot de passe par email
- Export structuré pour migration Kesh-à-Kesh
- Conformité nLPD (export/anonymisation des données personnelles)
- Sous-comptes clients et fournisseurs (grand livre auxiliaire par tiers)
- IA optionnelle : classification automatique des écritures, catégorisation des transactions, détection d'anomalies

2.8.5 Phase 4 — Vision future

- Évolution vers un mini-ERP pour PME suisses
- Communauté open source active avec contributeurs
- Support payant comme modèle économique
- Intégrations tierces (Open Banking, API partenaires)

2.8.6 Stratégie d'atténuation des risques

Type de risque	Risque	Mitigation
Technique	QR Bill pixel-perfect (specs SIX strictes)	Tests unitaires + validation contre specs SIX dès le début
Technique	Parseur CAMT.053 (XML complexe)	Utiliser les fichiers de test officiels SIX
Technique	Intégrité comptable (calculs débit/crédit)	Tests unitaires exhaustifs sur le moteur comptable (`rust_decimal`)
Marché	Faible adoption	Kesh répond d'abord au besoin propre du développeur — adoption externe en bonus
Ressources	Motivation solo sur la durée	Phasage v0.1/v0.2 pour livrer un produit utilisable rapidement

2.9 Exigences Fonctionnelles

2.9.1 Configuration & Onboarding

- FR1 : L'administrateur peut installer Kesh via docker-compose en moins de 15 minutes
- FR2 : L'administrateur peut configurer l'application via des variables d'environnement
- FR3 : L'administrateur peut configurer le nom d'utilisateur et le mot de passe du compte admin initial via des variables d'environnement
- FR4 : L'assistant d'onboarding guide l'utilisateur à travers le type d'organisation, les coordonnées, et la configuration des comptes bancaires
- FR5 : Le système installe automatiquement le plan comptable et les journaux adaptés au type d'organisation choisi
- FR6 : L'administrateur peut configurer la politique de mot de passe
- FR7 : Le système affiche un disclaimer légal « ne remplace pas un fiduciaire »
- FR8 : Le système expose un endpoint de healthcheck (`/health`)

2.9.2 Gestion des Utilisateurs & Sécurité

- FR9 : L'administrateur peut créer, désactiver et gérer les comptes utilisateurs
- FR10 : L'administrateur peut attribuer un ou plusieurs rôles à un utilisateur (Admin, Comptable, Consultation)
- FR11 : Le système contrôle l'accès aux fonctionnalités en fonction des rôles (RBAC)
- FR12 : L'utilisateur peut s'authentifier avec un identifiant et un mot de passe
- FR13 : Le système renouvelle silencieusement la session et la termine après 15 minutes d'inactivité (configurable)
- FR14 : L'utilisateur peut changer son propre mot de passe

— FR15 : L'administrateur peut réinitialiser le mot de passe d'un utilisateur
— FR16 : Le système bloque les tentatives de connexion après 5 échecs en 15 minutes, avec un délai de déblocage de 30 minutes
— FR17 : Le système utilise le verrouillage optimiste pour gérer la concurrence entre utilisateurs

2.9.3 Plan Comptable & Écritures

— FR18 : L'utilisateur peut charger un plan comptable suisse standard (PME, association, indépendant)
— FR19 : L'utilisateur peut ajouter, modifier et archiver des comptes dans le plan comptable
— FR20 : L'utilisateur peut saisir des écritures en partie double (débit/crédit)
— FR21 : Le système refuse toute écriture déséquilibrée (débit ≠ crédit)
— FR22 : L'utilisateur peut saisir des écritures dans différents journaux (Achats, Ventes, Banque, Caisse, OD)
— FR23 : L'utilisateur peut supprimer des écritures tant que l'exercice est ouvert
— FR24 : Le système interdit la modification et la suppression des écritures d'un exercice clôturé

2.9.4 Carnet d'Adresses & Contacts

— FR25 : L'utilisateur peut gérer un carnet d'adresses unifié (personnes et entreprises)
— FR26 : L'utilisateur peut marquer un contact comme client, fournisseur, ou les deux
— FR27 : Le système valide le format et le checksum du numéro IDE (CHE) lorsqu'il est saisi (champ optionnel)
— FR28 : L'utilisateur peut associer des conditions de paiement par défaut à un contact

2.9.5 Catalogue Produits/Services

— FR29 : L'utilisateur peut gérer un catalogue de produits/services (nom, description, prix unitaire, taux TVA)
— FR30 : L'utilisateur peut sélectionner des articles du catalogue lors de la création d'une facture

2.9.6 Facturation

— FR31 : L'utilisateur peut créer une facture avec des lignes libres ou depuis le catalogue (quantité, prix, TVA)
— FR32 : L'utilisateur peut supprimer une facture en brouillon
— FR33 : L'utilisateur peut valider une facture, ce qui lui attribue un numéro séquentiel définitif
— FR34 : Le système génère un PDF QR Bill conforme aux spécifications SIX 2.2 pour chaque facture validée

— FR35 : L'utilisateur peut configurer le format de numérotation des factures
— FR36 : L'utilisateur peut annuler une facture validée uniquement par la création d'un avoir (v0.2)
— FR37 : Le système gère une séquence de numérotation séparée pour les avoirs (v0.2)
— FR38 : Le PDF s'ouvre dans un nouvel onglet du navigateur

2.9.7 Paiements (v0.2)

— FR39 : L'utilisateur peut sélectionner des factures fournisseurs ouvertes pour générer un lot de paiements
— FR40 : Le système génère un fichier pain.001.001.09.ch.03 conforme au schéma XSD SIX
— FR41 : Le système valide le format IBAN avant la génération du fichier de paiement

2.9.8 Import Bancaire & Réconciliation

— FR42 : L'utilisateur peut importer des relevés bancaires aux formats CAMT.053 et CSV
— FR43 : Le système détecte les fichiers déjà importés et les transactions en doublon
— FR44 : Le système propose automatiquement des contreparties pour les transactions connues
— FR45 : L'utilisateur peut créer manuellement une contrepartie pour les transactions inconnues et réconcilier
— FR46 : Le système propose la création d'une règle d'affectation après une affectation manuelle
— FR47 : L'utilisateur peut gérer les règles d'affectation automatique
— FR48 : L'utilisateur peut éclater manuellement une transaction agrégée en sous-lignes
— FR49 : Le système extrait les sous-transactions (`TxDtls`) des fichiers CAMT quand elles sont présentes
— FR50 : L'utilisateur peut importer un relevé lié à un compte bancaire spécifique (multi-comptes)
— FR51 : Le système gère le rejet partiel d'un import avec un listing détaillé des erreurs
— FR52 : Le parseur CSV supporte UTF-8 et ISO-8859-1 avec détection automatique
— FR53 : L'utilisateur peut configurer des profils de format par banque pour l'import CSV

2.9.9 TVA (v0.2)

— FR54 : L'administrateur peut configurer les taux de TVA avec des dates de validité
— FR55 : Le système applique les arrondis TVA au centime par ligne (arrondi commercial)
— FR56 : L'utilisateur peut générer un rapport TVA par période (trimestriel/semestriel)

2.9.10 Budgets (v0.2)

- FR57 : L'utilisateur peut créer un budget annuel par compte individuel avec des montants mensuels
- FR58 : L'utilisateur peut gérer plusieurs versions d'un budget par exercice (initial, révisé)
- FR59 : L'utilisateur peut générer un rapport comparatif budget vs réalisé avec écarts

2.9.11 Clôture & Exercice (v0.2)

- FR60 : L'utilisateur peut clôturer un exercice comptable
- FR61 : Le système reporte automatiquement les soldes vers le nouvel exercice
- FR62 : L'utilisateur peut saisir un bilan d'ouverture (soldes de départ)

2.9.12 Pièces Justificatives (v0.2)

- FR63 : L'utilisateur peut attacher des fichiers (PDF, images) à une écriture
- FR64 : Le système stocke les justificatifs dans un volume de stockage dédié, séparé de la base de données

2.9.13 Rapports & Exports

- FR65 : L'utilisateur peut générer un bilan, un compte de résultat, une balance des comptes et des journaux
- FR66 : L'utilisateur peut exporter les rapports en PDF et CSV
- FR67 : Les PDF générés respectent les formats suisses (apostrophe séparateur de milliers, dates dd.mm.yyyy)
- FR68 : L'utilisateur peut exporter l'ensemble des données par table en CSV
- FR69 : L'utilisateur peut rechercher des écritures par montant, libellé, numéro de facture ou date
- FR70 : L'utilisateur peut trier, filtrer et paginer toutes les listes

2.9.14 Interface & Aide

- FR71 : Le système affiche des notifications banner pour les succès, avertissements et erreurs non-bloquantes
- FR72 : Le système affiche des modals pour les erreurs bloquantes (session expirée, conflit de version)
- FR73 : Le système fournit des tooltips contextuels sur les termes comptables et champs techniques
- FR74 : L'utilisateur peut accéder à 3 manuels PDF embarqués (démarrage, utilisateur, administrateur) (v0.2)

2.9.15 Internationalisation

- FR75 : Le système est disponible en français, allemand, italien et anglais

— FR76 : La langue de l'interface est configurée au niveau de l'instance. Elle est découplée de la langue des données comptables (libellés, noms de comptes). L'utilisateur peut changer la langue de l'interface sans impacter les données saisies.

2.9.16 Déploiement & Maintenance

— FR77 : Le système se déploie via docker-compose avec 2 containers (application + base de données). Un reverse proxy TLS est optionnel et géré par l'infrastructure de l'utilisateur.
— FR78 : Le système détecte une nouvelle version au démarrage et avertit de faire un backup avant migration
— FR79 : Le système applique les migrations de schéma automatiquement
— FR80 : Le système fournit un script de seed rechargeable pour la démo et les tests

2.9.17 Modèles de Documents (v0.2)

— FR81 : L'utilisateur peut personnaliser les modèles de documents générés (factures, rapports) : logo, coordonnées, textes de pied de page

2.9.18 Page d'Accueil

— FR82 : Le système affiche une page d'accueil après connexion avec accès rapide aux fonctions principales (dernières écritures, factures ouvertes, soldes des comptes bancaires)

2.9.19 Comptes Bancaires

— FR83 : L'utilisateur peut configurer et gérer ses comptes bancaires (nom, IBAN, banque)
— FR84 : L'assistant d'onboarding propose la configuration des comptes bancaires avec validation QR-IBAN/IBAN

2.9.20 Lettrage (v0.2)

— FR85 : L'utilisateur peut lettrer des écritures entre elles pour marquer les correspondances (facture ↔ paiement)
— FR86 : L'utilisateur peut délettrer des écritures précédemment lettrées tant que l'exercice est ouvert

2.9.21 Versioning des parseurs (v0.2)

— FR87 : Le système identifie la version du format utilisé lors de l'import (CAMT.053, pain.001) et sélectionne le parseur/générateur correspondant

2.9.22 Traçabilité & Audit

— FR88 : Le système enregistre un journal d'audit des actions utilisateurs sur les données comptables (création, modification, suppression d'écritures, clôture d'exercice) avec l'identifiant utilisateur et l'horodatage

2.9.23 Résilience Frontend

— FR89 : Le frontend s'affiche même si la base de données est inaccessible, avec un message d'erreur explicite invitant à vérifier l'état du serveur

2.10 Exigences Non-Fonctionnelles

2.10.1 Performance

— Les pages se chargent en moins de 300ms
— L'import d'un relevé mensuel (200 transactions) s'exécute en moins de 2s
— La génération d'un PDF (facture QR Bill, rapport) s'exécute en moins de 3s
— Le système supporte 2-5 utilisateurs simultanés par instance sans dégradation

2.10.2 Sécurité

— Tous les mots de passe sont hashés avec argon2 ou bcrypt, jamais stockés en clair
— Aucune donnée n'est accessible sans authentification JWT valide
— La communication est chiffrée via TLS en production (nginx reverse proxy)
— Le rate limiting protège l'endpoint de connexion contre le brute-force
— Les données comptables ne sont jamais transmises à un tiers (sauf IA externe avec consentement explicite, post-MVP)
— Les credentials de base de données ne sont jamais exposés dans les logs ou l'API

2.10.3 Fiabilité & Intégrité

— La balance débit/crédit est garantie correcte à tout moment — aucun écart possible
— Aucune donnée comptable n'est perdue en cas de crash ou de redémarrage
— Les fichiers pain.001 générés passent la validation XSD officielle SIX
— Les QR Bill générés sont conformes aux spécifications SIX 2.2 (dimensions, position, contenu)
— Les migrations de schéma préservent l'intégrité des données des exercices passés
— L'arithmétique monétaire utilise exclusivement des types décimaux exacts (`rust_decimal`), jamais de flottants

2.10.4 Accessibilité

— L'interface s'inspire de WCAG AA (contraste suffisant, navigation clavier, labels sur les formulaires) sans contrainte stricte de conformité
— Le zoom navigateur reste fonctionnel à 200%

2.10.5 Internationalisation

— Toutes les chaînes de l'interface sont externalisées (format Fluent `.ftl`)
— Les formats régionaux suisses sont respectés : dates `dd.mm.yyyy`, montants avec apostrophe `1'234.56`
— Les PDF générés respectent les mêmes formats régionaux

2.10.6 Maintenabilité

— Le code source est documenté selon les meilleures pratiques (doc comments Rust, JSDoc Svelte)
— Aucun code dupliqué (principe DRY)
— Tests unitaires sur toute la logique métier (moteur comptable, TVA, calculs financiers)
— Tests d'intégration sur les parseurs (CAMT.053, QR Bill, pain.001) avec les fichiers de test officiels SIX
— Tests E2E avec Playwright couvrant chaque parcours utilisateur
— L'API est versionnée (`/api/v1/`) dès le premier jour

2.10.7 Déploiement

— L'application se déploie via une seule commande `docker-compose up`
— L'image Docker résultante pèse moins de 100 Mo
— Les logs applicatifs sont émis sur stdout/stderr (standard Docker)
— Le healthcheck `/health` vérifie la connexion à la base de données

Chapitre 3

Partie 2 — Architecture technique

stepsCompleted : [1, 2, 3, 4, 5, 6, 7, 8] lastStep : 8 status : ‘complete’ completedAt : ‘2026-04-02’ inputDocuments : - _bmad-output/planning-artifacts/prd.md - docs/kesh-prd-v0.2.md - docs/change_request.md workflowType : ‘architecture’ project_name : ‘kesh’ user_name : ‘Guy’ date : ‘2026-04-02’ —

Chapitre 4

Architecture Decision Document

This document builds collaboratively through step-by-step discovery. Sections are appended as we work through each architectural decision together.

4.1 Analyse du Contexte Projet

4.1.1 Vue d'ensemble des Exigences

Exigences fonctionnelles : 89 exigences fonctionnelles (FR1-FR89) réparties en 22 domaines. Le cœur architectural se concentre sur : - Moteur comptable en partie double avec intégrité absolue (FR20-FR24, FR60-FR62) - Import/parsing de standards bancaires suisses ISO 20022 (FR42-FR53) - Génération de documents conformes SIX : QR Bill 2.2, pain.001.001.09.ch.03 (FR31-FR41) - RBAC multi-utilisateurs avec verrouillage optimiste (FR9-FR17) - Réconciliation bancaire semi-automatique avec règles évolutives (FR44-FR49) - Internationalisation 4 langues avec formats suisses natifs (FR75-FR76)

Phasage : v0.1 couvre le cœur comptable + facturation QR Bill + import CAMT + réconciliation. v0.2 ajoute TVA, budgets, avoirs, pain.001, clôture, justificatifs.

Exigences non-fonctionnelles : - Performance : pages < 300ms, import 200 transactions < 2s, PDF < 3s - Sécurité : argon2/bcrypt, JWT + refresh silencieux, rate limiting, TLS (nginx) - Intégrité : balance toujours juste, zéro perte de données, arithmétique décimale exacte - Conformité : CO suisse (art. 957-964), standards SIX, TVA AFC - Accessibilité : inspiré WCAG AA sans contrainte stricte - Déploiement : docker-compose, image < 100 Mo, logs stdout/stderr

Échelle & Complexité : - Domaine principal : Web app full-stack (SPA Svelte + API REST Axum) - Niveau de complexité : Haute — standards bancaires ISO 20022, conformité comptable suisse, TVA multi-taux avec historique - Composants architecturaux estimés : 10 crates backend + frontend SPA - Charge : 2-5 utilisateurs simultanés par instance (auto-hébergé)

4.1.2 Contraintes & Dépendances Techniques

- **Stack imposée** : Rust (Axum), Svelte, MariaDB 10.6+, Docker
- **Arithmétique** : `rust_decimal` exclusivement, jamais de f64 pour les montants
- **Standards SIX** : QR Bill 2.2 (dimensions 46×46mm, position A4, police, croix suisse), pain.001.001.09.ch.03 (validation XSD, SPS 2026), CAMT.053.001.04
- **Encodage** : UTF-8 + ISO-8859-1 (import CSV, détection automatique)
- **Configuration** : variables d'environnement
- **Licence** : EUPL 1.2
- **Web app uniquement** : pas de Tauri (décision PRD final)

4.1.3 Préoccupations Transversales Identifiées

1. **Intégrité comptable** — Validation débit=crédit sur chaque écriture, immutabilité post-clôture, `rust_decimal` partout. Traverse : saisie, import, réconciliation, clôture, rapports.
2. **Internationalisation** — 4 langues (FR/DE/IT/EN), Fluent `.ftl`, backend expose les traductions. Traverse : toute l'UI, les PDF générés, les messages d'erreur, l'aide contextuelle.
3. **Formats suisses** — Apostrophe séparateur de milliers (`1'234.56`), dates `dd.mm.yyyy`. Traverse : affichage UI, PDF, exports CSV.
4. **Authentification & Autorisation** — JWT + refresh, RBAC 3 rôles, rate limiting. Traverse : toutes les routes API, toutes les vues frontend.
5. **Validation métier** — IBAN/QR-IBAN, IDE CHE (checksum), XSD pain.001, conformité QR Bill SIX. Traverse : carnet d'adresses, facturation, paiements.
6. **Audit & Conservation** — CO suisse exige 10 ans de conservation. Écritures verrouillées post-clôture, pas de suppression définitive. Traverse : toute modification de données comptables.
7. **Gestion d'erreurs structurée** — Import partiel avec listing détaillé, doublons, conflit de version (optimistic locking), session expirée. Traverse : import, saisie concurrente, UI.

4.1.4 Décisions Architecturales Préliminaires (issues de la revue collaborative)

#	Décision	Choix	Justification
1	Séparation core/db	Stricte — `kesh-core` sans I/O	Testabilité, évolutivité, logique métier pure
2	Versioning parseurs SIX	Multi-version pour CAMT.053/pain.001, dernière version seulement pour QR Bill	Les banques ne migrent pas toutes en même temps ; SIX impose une date de bascule pour QR Bill

#	Décision	Choix	Justification
3	Concurrence imports bancaires	Blocage (mutex par compte bancaire)	Simplicité, pas de réconciliation de conflits pour 2-5 utilisateurs
4	Phasage architectural	Architecture complète dès v0.1	Éviter les refactors lourds, interfaces v0.2 définies sans implémentation
5	Extension stocks (CR-003)	Points d'extension prévus dès v0.1	Trait/interface pour entités gérées, sans code mort
6	Crates autonomes	Structure indépendante (zéro dépendance), publication remise à plus tard	Bonne architecture sans contraintes de publication prématurées
7	Types des crates publiables	Types autonomes par crate, conversion via From/Into côté Kesh	Indépendance totale entre crates
8	Documentation	Complète dès le jour 1 — `///` Rust, `//!` modules, JSDoc Svelte	Hygiène, maintenabilité, attractivité open source
9	Module de réconciliation	`kesh-reconciliation` dédié — matching, règles d'affectation, mutex	Un module par domaine métier, éviter les modules gigantesques
10	Glossaire comptable	PDF statique avec how-to (v0.2) + tooltips courts (clés Fluent) dans l'UI	Ne pas surcharger l'interface, tout en aidant les non-comptables
11	Seed rechargeable	`kesh-seed` crate séparée, passe par `kesh-core` → `kesh-db`, contourne `kesh-api`	Validations métier préservées, pas besoin de serveur HTTP
12	Génération PDF	Chaque crate gère son propre PDF — `kesh-qrbill` (pixel-perfect SIX) et `kesh-report` (tabulaire) séparés	Besoins trop différents pour une couche partagée

#	Décision	Choix	Justification
13	`kesh-i18n`	Crate séparée — chargement Fluent + formatage suisse (montants, dates)	Dépendance transversale utilisée par report, api, qrbill
14	Stratégie de test	3 niveaux : unitaires (core seul), intégration (seed+db), E2E Playwright (api complet seedé)	Périmètres clairs, zéro ambiguïté

4.1.5 Structure Workspace Cargo

```
Crates publiables (indépendants)       Crates Kesh (internes)

┌──────────────────┐                   ┌──────────────────────┐
│ kesh-import      │──────────────┐    │ kesh-core            │
│ kesh-payment     │──────────┐   │    │ kesh-db              │
│ kesh-qrbill      │────────┐ │   │    │ kesh-api             │
└──────────────────┘        │ │   │    │ kesh-reconciliation  │
                            ▼ ▼   ▼    │ kesh-i18n            │
                      Conversions via   │ kesh-report          │
                      From/Into dans    │ kesh-seed            │
                      kesh-core/api     └──────────────────────┘
```

4.1.6 Stratégie de Test

```
Tests unitaires        : kesh-core seul (logique pure, pas d'I/O)
Tests d'intégration    : kesh-seed + kesh-db (données réalistes en base)
Tests E2E (Playwright) : kesh-api complet avec base seedée
```

4.2 Évaluation Starter Template

4.2.1 Domaine Technologique Principal

Full-stack : Rust (Axum) backend + Svelte SPA frontend, déployé via Docker.

4.2.2 Versions Vérifiées (avril 2026)

Technologie	Version	Usage
Axum	0.8.x	Framework HTTP Tokio
SQLx	0.8.6	Driver MySQL/MariaDB async
rust_decimal	1.39.0	Arithmétique financière exacte
fluent-bundle	0.16.x	Internationalisation Fluent

Technologie	Version	Usage
tower-http	0.5.x	Middleware CORS, compression, tracing, ServeDir
Svelte	5.55.x	Framework réactif
SvelteKit	2.55.x	Framework applicatif (mode SPA)
Playwright	1.57.x	Tests E2E

4.2.3 Options Considérées

Frontend : - **Option A — SvelteKit SPA (`adapter-static`)** [OK] : routing intégré, layouts, error pages, build Vite, Playwright intégrable. Sortie = fichiers statiques. - **Option B — Svelte pur + Vite** [NON] : trop minimal, il faudrait réinventer routing et layouts.

Backend : - **Cargo workspace manuel** [OK] : pas de starter template pertinent pour un workspace multi-crates custom. Scaffold manuel = norme Rust.

4.2.4 Sélection Retenue

Composant	Choix	Initialisation
Frontend	SvelteKit SPA (`adapter-static`, `ssr=false`)	`npx sv create frontend`
Backend	Cargo workspace manuel (10 crates)	`Cargo.toml` workspace + `cargo init` par crate

4.2.5 Commandes d'Initialisation

```
# Frontend — SvelteKit SPA
npx sv create frontend
# → Svelte 5, TypeScript, Playwright, adapter-static
# → ssr = false dans root layout

# Backend — Cargo workspace
mkdir -p crates/{kesh-core,kesh-db,kesh-api,kesh-reconciliation,kesh-i18n,kesh-repo
# → Cargo.toml racine avec [workspace] members
# → cargo init pour chaque crate
```

4.2.6 Architecture de Serving

```
Prod :  Navigateur → (nginx TLS optionnel) → Axum :3000 (SPA + API /api/v1/*)
Dev :   Navigateur → Vite :5173 (hot reload) → proxy /api/* → Axum :3000
```

- **Axum sert tout** : fichiers statiques SPA via `tower-http::ServeDir` + API REST
- **TLS non géré par Kesh** : HTTP pur, TLS = infrastructure (nginx/Traefik/Caddy)
- **DB inaccessible** : le frontend est servi quand même, message d'erreur côté client

— **Docker-compose** : 2 containers (kesh + mariadb), nginx optionnel

4.2.7 Outils Recommandés

— `cargo-nextest` — exécution de tests plus rapide
— `cargo-deny` — audit licences et sécurité
— `cargo-udeps` — détection de dépendances inutilisées

4.2.8 Décisions Architecturales (étape 3)

#	Décision	Choix	Justification
15	TLS	Pas géré par Kesh — HTTP pur	TLS = problème d'infrastructure, pas d'application
16	Serving frontend	Axum sert tout via `tower-http::ServeDir`	Source unique, pas de nginx requis
17	DB inaccessible	Frontend servi, message d'erreur client	L'utilisateur voit toujours quelque chose
18	Docker-compose	2 containers : kesh + mariadb	Simplifié vs 3 containers avec nginx
19	Dev workflow	Vite dev server + proxy vers Axum	Hot reload frontend sans rebuild

Note : L'initialisation du projet (workspace Cargo + SvelteKit) sera la première story d'implémentation.

Note PRD : Les points suivants devront être mis à jour dans le PRD : 1. Axum sert le frontend en production (plus nginx par défaut) 2. nginx optionnel (TLS/reverse proxy uniquement) 3. docker-compose simplifié à 2 containers 4. Le frontend s'affiche même si la DB est inaccessible

4.3 Décisions Architecturales Principales

4.3.1 Analyse des Priorités

Décisions critiques (bloquent l'implémentation) : - Data architecture, authentification, structure API, serving frontend

Décisions importantes (façonnent l'architecture) : - Composants UI, state management, logging, CI/CD

Décisions différées (post-MVP) : - Documentation API formelle (OpenAPI), permissions granulaires, scaling horizontal

4.3.2 Data Architecture

Décision	Choix	Justification
Accès base de données	SQLx direct + repository pattern dans `kesh-db`	Contrôle total sur le SQL, requêtes comptables complexes. Migration vers PostgreSQL possible en réécrivant les repositories sans toucher core/api
Migrations	`sqlx migrate`, fichiers versionnés dans `crates/kesh-db/migrations/`	Zéro perte de données en cas de migration
Cache	Aucun	Simplicité. 2-5 utilisateurs, réévaluation si problème de performance
Validation des données	Types forts dans `kesh-core` (newtypes pour IBAN, CHE, montants) + validation API	Intégrité garantie à deux niveaux : types métier + validation HTTP

4.3.3 Authentification & Sécurité

Décision	Choix	Justification
JWT	`jsonwebtoken` crate	Mature, stable
Hashing mots de passe	Argon2id	Standard actuel, résistant GPU
Token structure	Access token JWT (~15 min) + Refresh token UUID opaque en base	Révocation immédiate possible (désactivation, changement MdP)
RBAC	Hiérarchique : Consultation < Comptable < Admin	Simple, chaque rôle hérite du précédent. Point d'extension prévu pour permissions granulaires post-MVP
Rate limiting	`/api/v1/auth/login` uniquement, compteur en mémoire par IP, middleware tower	Le plus simple possible

4.3.4 API & Communication

Décision	Choix	Justification
Design API	REST, préfixe `/api/v1/`, routes kebab-case	Convention standard
Pagination	offset/limit	Simple, suffisant pour le volume
Format d'erreur	Structuré avec code métier + message + details	Le frontend peut afficher des messages traduits
Documentation API	Pas de documentation formelle MVP, le code fait foi	Réévaluation post-MVP
Sérialisation	`serde` / `serde_json`, champs JSON en camelCase	`#[serde(rename_all = "camelCase")]` — convention frontend JS

4.3.5 Frontend Architecture

Décision	Choix	Justification
State management	Svelte stores natifs (writable, derived)	Simple, intégré, suffisant pour cette taille
Composants UI	shadcn-svelte (Svelte 5, Tailwind CSS v4)	Code dans le projet, modifiable, accessible, pas de lock-in
Communication API	`fetch` natif + wrapper léger (JWT, refresh, erreurs)	Zéro dépendance externe
Formatage montants/dates	Frontend via `Intl.NumberFormat` / `Intl.DateTimeFormat` (locale `de-CH`)	API navigateur native, zéro dépendance, apostrophe suisse native

4.3.6 Infrastructure & Déploiement

Décision	Choix	Justification
CI/CD	GitHub Actions	Standard open source, repo sur GitHub
Logging	`tracing` crate (écosystème Tokio/Axum), stdout/stderr	Logs structurés, conforme Docker
Configuration	Variables d'environnement via `dotenvy` + fichier `.env` (dev et prod)	Mécanisme unique partout
Scaling	Pas de scaling horizontal. Instances autonomes (1 instance = 1 base)	2-5 utilisateurs max par instance, même en PME

4.3.7 Analyse d'Impact des Décisions

Séquence d'implémentation : 1. Scaffold workspace Cargo + SvelteKit SPA 2. `kesh-core` — types forts, newtypes, validation métier 3. `kesh-db` — repository pattern, migrations SQLx, schéma initial 4. `kesh-api` — serveur Axum, auth JWT, RBAC, ServeDir 5. Frontend — SvelteKit, shadcn-svelte, wrapper fetch, stores 6. Crates métier — import, qrbill, reconciliation, etc.

Dépendances croisées : - `kesh-core` n'a aucune dépendance sur les autres crates internes - `kesh-db` dépend de `kesh-core` (types métier) - `kesh-api` dépend de `kesh-core`, `kesh-db`, `kesh-i18n`, `kesh-reconciliation` - `kesh-reconciliation` dépend de `kesh-core`, `kesh-db` - `kesh-report` dépend de `kesh-core`, `kesh-db`, `kesh-i18n` - `kesh-seed` dépend de `kesh-core`, `kesh-db` - Crates publiables (`kesh-import`, `kesh-payment`, `kesh-qrbill`) : zéro dépendance interne

4.4 Patterns d'Implémentation & Règles de Cohérence

4.4.1 Points de Conflit Identifiés

12 catégories où des agents AI pourraient diverger, toutes traitées ci-dessous.

4.4.2 Naming Patterns

Base de données :

Élément	Convention	Exemple
Tables	snake_case, pluriel	`journal_entries`, `import_rules`
Colonnes	snake_case	`company_id`, `fiscal_year_start`
Clés étrangères	`{table_singulier}_id`	`account_id`, `entry_id`
Index	`idx_{table}_{colonnes}`	`idx_accounts_company_id`
Contraintes unique	`uq_{table}_{colonnes}`	`uq_users_email`

API REST :

Élément	Convention	Exemple
Routes	kebab-case, pluriel	`/api/v1/journal-entries`
Paramètres route	`:id`	`/api/v1/accounts/:id`
Query params	camelCase	`?companyId=1&pageSize=50`

Code Rust :

Élément	Convention	Exemple
Structs/Enums	PascalCase	`JournalEntry`, `AccountType`

Élément	Convention	Exemple
Fonctions/méthodes	snake_case	`create_entry`, `validate_iban`
Modules	snake_case	`journal_entries`, `bank_import`
Fichiers	snake_case	`journal_entries.rs`
Newtypes	PascalCase	`Iban(String)`, `CheNumber(String)`, `Money(Decimal)`

Code Svelte/TypeScript :

Élément	Convention	Exemple
Composants	PascalCase	`JournalEntryForm.svelte`
Fichiers routes SvelteKit	kebab-case	`routes/journal-entries/+page.svelte`
Fonctions/variables	camelCase	`fetchEntries()`, `accountList`
Types/interfaces	PascalCase	`JournalEntry`, `ApiError`
Stores	camelCase	`currentCompany`, `journalEntries`

4.4.3 Structure Patterns

Organisation des tests : - Rust : tests unitaires co-localisés (`#[cfg(test)] mod tests`), tests d'intégration dans `crates/{crate}/tests/` - Svelte : tests co-localisés (`Component.test.ts` à côté de `Component.svelte`) - E2E Playwright : `frontend/tests/e2e/`

Organisation frontend : par feature

```
frontend/src/lib/
├── features/
│   ├── journal-entries/    # composants, stores, types, API
│   ├── invoicing/
│   ├── bank-import/
│   ├── reconciliation/
│   ├── accounts/
│   ├── contacts/
│   └── reports/
├── shared/                 # composants partagés, utils, types communs
└── app/                    # layout, auth, navigation, config
```

4.4.4 Format Patterns

Réponses API : - Succès lecture : donnée directe (pas de wrapper) - Succès liste : `{ "items": [...], "total": 42, "offset": 0, "limit": 50 }` - Succès création : 201 + ressource créée - Succès suppression : 204 (pas de body) - Erreur : `{ "error": { "code": "ENTRY_UNBALANCED", "message": "...", "details": {...} } }`

Conventions de données JSON : - Champs : camelCase (`companyId`, `fiscalYearStart`) - Dates : ISO 8601 (`"2026-04-02"`, `"2026-04-02T14:30:00Z"`) - Montants : string décimal (`"1234.56"`) — jamais de float - Champs vides : omis (pas de `"field": null`), sauf si la distinction vide/absent est sémantique

Codes HTTP :

Situation	Code
Succès lecture	200
Création réussie	201
Suppression réussie	204
Validation échouée	400
Non authentifié	401
Non autorisé (RBAC)	403
Ressource non trouvée	404
Conflit (optimistic lock)	409
Rate limited	429
Erreur serveur	500

4.4.5 Communication & Process Patterns

Gestion des erreurs Rust : - `AppError` centralisé dans `kesh-api` → `IntoResponse` Axum - Types d'erreur par crate (`CoreError`, `DbError`) → conversion via `From<T>` vers `AppError` - Logging `tracing` côté serveur, message traduit côté client, jamais de stack trace exposée

Gestion des erreurs Frontend : - Wrapper `fetch` intercepte toutes les erreurs - 401 → refresh automatique, si échec → redirection login - 409 → modal conflit de version - 400 → détails de validation sous les champs - 500 → banner d'erreur générique

Loading states Frontend : - Variable `loading` booléenne par requête dans le composant - Skeleton/spinner local (pas de spinner global) - Boutons désactivés pendant la soumission

Logging Rust :

Niveau	Usage
`error!`	Erreur inattendue, bug, panique rattrapée
`warn!`	Situation anormale mais gérée (import partiel, doublon)
`info!`	Événements métier (login, clôture, import terminé)
`debug!`	Détails techniques (requêtes SQL, durées)
`trace!`	Très verbeux, dev uniquement

Verrouillage optimiste : - Champ `version` (integer) sur chaque entité modifiable - Frontend envoie `version` dans PUT/PATCH - `version` en base ≠ reçue → 409 Conflict - Pattern uniforme : écritures, comptes, contacts, factures

4.4.6 Règles Obligatoires pour Tous les Agents AI

1. **Jamais de f64 pour les montants** — `rust_decimal::Decimal` exclusivement
2. **Toute écriture comptable doit être équilibrée** — validation dans `kesh-core` avant persistance
3. **Tout code public documenté** — `///` Rust, JSDoc Svelte
4. **Tests unitaires pour toute logique métier** — pas de code métier sans test
5. **Nommage conforme aux conventions ci-dessus** — vérifiable par revue de code
6. **Erreurs structurées avec code métier** — jamais de string d'erreur en dur côté frontend
7. **Verrouillage optimiste sur toute entité modifiable** — champ `version` systématique

4.5 Structure du Projet & Frontières

4.5.1 Structure Complète du Répertoire

```
kesh/
├── .github/
│   └── workflows/
│       ├── ci.yml                      # Build, tests, clippy, fmt
│       └── release.yml                 # Build Docker image
├── .env.example                     # Template variables d'environnement
├── .gitignore
├── Cargo.toml                          # Workspace root
├── Cargo.lock
├── docker-compose.yml                  # kesh + mariadb
├──  Dockerfile                                                  #  Multi-
stage: build Rust + build Svelte → image finale
├── README.md
│
├── crates/
│   ├── kesh-core/                      # Logique métier pure, zéro I/O
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│  │     ├── types/             # Newtypes: Money, Iban, CheNumber, etc.
│  │     ├── accounting/        # Partie double, validation balance, journaux
│  │      ├── validation/           # Validations métier (IBAN, CHE, QR-
IBAN)
│  │     ├── chart_of_accounts/    # Plans comptables, classes, types de comptes
│   │       └── errors.rs               # CoreError
```

```
│   │
│   ├── kesh-db/                 # Persistance MariaDB, repository pattern
│   │   ├── Cargo.toml
│   │   ├── src/
│   │   │   ├── lib.rs
│   │   │   ├── pool.rs                 # Configuration connexion SQLx
│   │   │   ├── repositories/           # Un module par entité
│   │   │   │   ├── accounts.rs
│   │   │   │   ├── journal_entries.rs
│   │   │   │   ├── invoices.rs
│   │   │   │   ├── contacts.rs
│   │   │   │   ├── users.rs
│   │   │   │   ├── import_rules.rs
│   │   │   │   ├── bank_accounts.rs
│   │   │   │   └── companies.rs
│   │   │   └── errors.rs              # DbError
│   │   ├── migrations/                 # sqlx migrate
│   │   └── tests/                      # Tests d'intégration DB
│   │
│   ├── kesh-api/                   # Serveur Axum, routes, auth, ServeDir
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── main.rs                 # Point d'entrée, setup serveur
│   │       ├── config.rs               # Chargement .env via dotenvy
│   │       ├── routes/                 # Un module par domaine
│   │       │   ├── auth.rs             # Login, refresh, logout
│   │       │   ├── accounts.rs
│   │       │   ├── journal_entries.rs
│   │       │   ├── invoices.rs
│   │       │   ├── contacts.rs
│   │       │   ├── bank_imports.rs
│   │       │   ├── reconciliation.rs
│   │       │   ├── reports.rs
│   │       │   ├── users.rs
│   │       │   ├── companies.rs
│   │       │   ├── bank_accounts.rs
│   │       │   └── health.rs           # /health endpoint
│   │       ├── middleware/
│   │       │   ├── auth.rs             # JWT extraction, RBAC
│   │       │   └── rate_limit.rs       # Rate limiting login
│   │       ├── extractors.rs         # Axum extractors (CurrentUser, Role)
│   │       ├── errors.rs               # AppError → IntoResponse
│   │       └── static_files.rs         # ServeDir frontend SPA
│   │
│   ├── kesh-reconciliation/            # Réconciliation bancaire
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
```

```
│   │      ├── matching.rs          # Algorithme de matching transactions
│   │      ├── rules.rs              # Règles d'affectation automatique
│   │      ├── mutex.rs               # Mutex par compte bancaire
│   │      └── errors.rs
│   │
│   ├── kesh-i18n/                    # Internationalisation
│   │   ├── Cargo.toml
│   │   ├── src/
│   │   │   ├── lib.rs
│   │   │   ├── loader.rs             # Chargement fichiers .ftl
│   │   │   └── formatting.rs         # Montants suisses, dates
│   │   └── locales/
│   │       ├── fr-CH/
│   │       ├── de-CH/
│   │       ├── it-CH/
│   │       └── en-CH/
│   │
│   ├── kesh-report/                  # Rapports comptables → PDF/CSV
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── balance_sheet.rs      # Bilan
│   │       ├── income_statement.rs   # Compte de résultat
│   │       ├── trial_balance.rs      # Balance des comptes
│   │       ├── journal_report.rs     # Journaux
│   │       ├── pdf.rs                # Génération PDF tabulaire
│   │       └── csv.rs                # Export CSV
│   │
│   ├── kesh-seed/                    # Données de démo/test
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── demo_data.rs          # Données démo réalistes
│   │       └── main.rs               # CLI: cargo run -p kesh-seed
│   │
│   ├── kesh-import/                  # PUBLIABLE — Parseurs bancaires
│   │   ├── Cargo.toml
│   │   ├── src/
│   │   │   ├── lib.rs
│   │   │   ├── camt053/            # Parseur CAMT.053 (multi-version)
│   │   │   ├── csv/              # Parseur CSV générique (multi-encodage)
│   │   │   └── types.rs          # Types autonomes (pas de dépendance kesh)
│   │   └── tests/
│   │       ├── fixtures/             # Fichiers de test SIX officiels
│   │       └── camt053_tests.rs
│   │
│   ├── kesh-payment/                 # PUBLIABLE — Génération pain.001
│   │   ├── Cargo.toml
```

```
│   │   ├── src/
│   │   │   ├── lib.rs
│   │   │   ├── pain001/            # Générateur pain.001 (multi-version)
│   │   │   ├── validation.rs           # Validation XSD
│   │   │   └── types.rs               # Types autonomes
│   │   └── tests/
│   │       └── fixtures/               # Schémas XSD SIX
│   │
│   └── kesh-qrbill/                    # PUBLIABLE — Swiss QR Bill
│       ├── Cargo.toml
│       ├── src/
│       │   ├── lib.rs
│       │   ├── generator.rs          # Génération QR Code + mise en page
│       │   ├── pdf.rs                 # PDF pixel-perfect SIX (46×46mm)
│       │   ├── validation.rs           # Validation conformité SIX 2.2
│       │   └── types.rs               # Types autonomes
│       └── tests/
│           └── fixtures/
│
├── frontend/                           # SvelteKit SPA
│   ├── package.json
│   ├── svelte.config.js                # adapter-static, ssr=false
│   ├── vite.config.ts                  # Proxy /api/* → Axum en dev
│   ├── tsconfig.json
│   ├── tailwind.config.ts
│   ├── src/
│   │   ├── app.html
│   │   ├── app.css                     # Tailwind imports
│   │   ├── routes/
│   │   │   ├── +layout.svelte        # Layout principal (nav, auth check)
│   │   │   ├── +layout.ts             # ssr=false, prerender=false
│   │   │   ├── +page.svelte            # Page d'accueil post-login
│   │   │   ├── +error.svelte           # Page d'erreur (DB down, etc.)
│   │   │   ├── login/
│   │   │   ├── accounts/
│   │   │   ├── journal-entries/
│   │   │   ├── invoices/
│   │   │   ├── contacts/
│   │   │   ├── bank-import/
│   │   │   ├── reconciliation/
│   │   │   ├── reports/
│   │   │   ├── bank-accounts/
│   │   │   └── settings/               # Users, company, config
│   │   └── lib/
│   │       ├── features/
│   │       │   ├── journal-entries/
│   │       │   │   ├── JournalEntryForm.svelte
│   │       │   │   ├── JournalEntryList.svelte
```

```
│   │       │   │   ├── journal-entries.store.ts
│   │       │   │   ├── journal-entries.api.ts
│   │       │   │   ├── journal-entries.types.ts
│   │       │   │   └── JournalEntryForm.test.ts
│   │       │   ├── invoicing/
│   │       │   ├── bank-import/
│   │       │   ├── reconciliation/
│   │       │   ├── accounts/
│   │       │   ├── contacts/
│   │       │   ├── reports/
│   │       │   └── auth/
│   │       ├── shared/
│   │       │   ├── components/       # shadcn-svelte + composants partagés
│   │       │   ├── utils/
│   │       │   │   ├── api-client.ts  # Wrapper fetch (JWT, refresh, erreurs)
│   │       │   │   └── formatting.ts  # Intl.NumberFormat, Intl.DateTimeFormat
│   │       │   └── types/
│   │       │       ├── api.ts          # ApiError, PaginatedResponse
│   │       │       └── common.ts
│   │       └── app/
│   │           ├── stores/          # Stores globaux (currentUser, locale)
│   │           └── config.ts
│   ├── static/                        # Assets statiques (favicon, etc.)
│   └── tests/
│       └── e2e/                       # Playwright
│           ├── auth.spec.ts
│           ├── journal-entries.spec.ts
│           ├── invoicing.spec.ts
│           ├── bank-import.spec.ts
│           └── reconciliation.spec.ts
│
└── charts/                            # Plans comptables standards
    ├── pme.json
    ├── association.json
    └── independant.json
```

4.5.2 Mapping Exigences → Structure

Domaine FR	Backend	Frontend
FR1-FR8 Configuration/Onboarding	`kesh-api/routes/companies.rs`, `kesh-api/routes/health.rs`	`routes/settings/`, `features/auth/`

Domaine FR	Backend	Frontend
FR9-FR17 Utilisateurs/Sécurité	kesh-api/middleware/auth.rs, kesh-api/routes/auth.rs, kesh-api/routes/users.rs	features/auth/, shared/utils/api-client.ts
FR18-FR24 Plan comptable/Écritures	kesh-core/accounting/, kesh-core/chart_of_accounts/, kesh-db/repositories/	features/accounts/, features/journal-entries/
FR25-FR28 Contacts	kesh-db/repositories/contacts.rs	features/contacts/
FR29-FR30 Catalogue	kesh-db/repositories/ (ajout)	features/invoicing/ (catalogue intégré)
FR31-FR38 Facturation	kesh-qrbill/, kesh-api/routes/invoices.rs	features/invoicing/
FR39-FR41 Paiements (v0.2)	kesh-payment/	features/ (ajout v0.2)
FR42-FR53 Import/Réconciliation	kesh-import/, kesh-reconciliation/, kesh-api/routes/bank_imports.rs	features/bank-import/, features/reconciliation/
FR54-FR56 TVA (v0.2)	kesh-core/ (ajout v0.2)	features/ (ajout v0.2)
FR65-FR70 Rapports	kesh-report/	features/reports/
FR75-FR76 i18n	kesh-i18n/	shared/utils/formatting.ts
FR77-FR80 Déploiement	Dockerfile, docker-compose.yml, kesh-seed/	—

4.5.3 Frontières Architecturales

```
┌──────────────────────────────────────────────────────────┐
│                      Navigateur                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │  SvelteKit SPA (shadcn-svelte, stores)           │    │
│  │  └── api-client.ts (fetch + JWT)                 │    │
│  └──────────────────────────────────────────────────┘    │
└──────────────────────────┬───────────────────────────────┘
                           │
                           │ HTTP JSON /api/v1/*
┌──────────────────────────┴───────────────────────────────┐
│ kesh-api (Axum)                                          │
│  ├── middleware (JWT, RBAC, rate limit)                  │
│  ├── routes → appelle kesh-core + kesh-db                │
│  ├── ServeDir (SPA statique)                             │
│  └── AppError (toutes erreurs → JSON)                    │
```

```
│──────────────────────────────────────────────────────────│
│ kesh-core          │ kesh-reconciliation                 │
│  (logique pure)    │  (matching, rules, mutex)           │
│  (types, valid.)   │                                     │
│──────────────────────────────────────────────────────────│
│ kesh-db (repository pattern, SQLx)                       │
│  └── migrations/                                         │
│──────────────────────────────────────────────────────────│
│ MariaDB                                                  │
└──────────────────────────────────────────────────────────┘

Crates publiables (indépendants, zéro dépendance) :

┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│ kesh-import     │  │ kesh-payment    │  │ kesh-qrbill     │
│ CAMT.053/CSV    │  │ pain.001        │  │ QR Bill SIX     │
└─────────────────┘  └─────────────────┘  └─────────────────┘

      ↕ From/Into        ↕ From/Into      ↕ From/Into
      kesh-core/api      kesh-core/api    kesh-core/api
```

4.5.4 Flux de Données

```
Import bancaire :
    Fichier CAMT.053 → kesh-import (parse) → From/Into → kesh-
core (validation)
  → kesh-reconciliation (matching, mutex) → kesh-db (persistance)

Facturation :
  Frontend (saisie) → kesh-api (validation) → kesh-core (logique facture)
  → kesh-db (persistance) → kesh-qrbill (génération PDF) → Frontend (téléchargement)

Rapports :
  Frontend (demande) → kesh-api → kesh-db (requêtes agrégées)
  → kesh-report (mise en forme) → kesh-i18n (formatage) → PDF/CSV → Frontend
```

4.6 Résultats de Validation de l'Architecture

4.6.1 Validation de Cohérence [OK]

Compatibilité des décisions : - Axum 0.8.x + tower-http 0.5.x + SQLx 0.8.x + tokio : écosystème Tokio cohérent - SvelteKit 2.x + Svelte 5 + shadcn-svelte + Tailwind CSS v4 : compatibilité vérifiée - `rust_decimal` 1.39 + SQLx MariaDB : support natif du type Decimal - `fluent-bundle` 0.16 : stable, pas de dépendance conflictuelle - snake_case DB ↔ camelCase JSON ↔ snake_case Rust : conversions gérées par `serde(rename_all)`

Cohérence des patterns : - Repository pattern ↔ SQLx direct : cohérent - Feature-based frontend ↔ routes SvelteKit par domaine : aligné - Naming conventions cohérentes à travers toutes les couches

Alignement structure : - Chaque domaine FR a un répertoire backend ET frontend correspondant - Les frontières crate/module respectent la séparation des responsabilités - Les crates publiables sont effectivement indépendantes (zéro dépendance vérifiée)

4.6.2 Validation de Couverture des Exigences [OK]

Exigences fonctionnelles v0.1 : 100% couvertes **Exigences fonctionnelles v0.2 :** 3 gaps mineurs, non bloquants, extensibles : - FR57-FR59 Budgets : ajout `budgets.rs` dans `kesh-db` + logique dans `kesh-core` - FR63-FR64 Justificatifs : ajout `attachments.rs` dans `kesh-db` (filesystem, volume Docker dédié) - FR81 Modèles documents : personnalisation templates dans `kesh-report`

Exigences non-fonctionnelles : Toutes couvertes - Performance : SQLx async, parseurs Rust natifs - Sécurité : argon2id, JWT + refresh, RBAC, rate limiting, TLS = infra - Intégrité : `kesh-core` types forts, `rust_decimal`, validation balance - Accessibilité : shadcn-svelte (ARIA, navigation clavier) - Déploiement : Docker multi-stage, image < 100 Mo, logs stdout/stderr

4.6.3 Validation de Préparation à l'Implémentation [OK]

Complétude des décisions : 19 décisions architecturales + 14 décisions préliminaires, toutes documentées avec justifications et versions vérifiées.

Complétude de la structure : Arborescence complète (~80 fichiers/dossiers), mapping FR → répertoires explicite.

Complétude des patterns : Naming (DB, API, Rust, Svelte/TS), formats (API, JSON, HTTP), process (erreurs, loading, logging, verrouillage) — tous couverts. 7 règles obligatoires pour les agents AI.

4.6.4 Checklist de Complétude

[OK] Analyse des exigences - [x] Contexte projet analysé en profondeur - [x] Échelle et complexité évaluées - [x] Contraintes techniques identifiées - [x] Préoccupations transversales mappées

[OK] Décisions architecturales - [x] Décisions critiques documentées avec versions - [x] Stack technologique entièrement spécifiée - [x] Patterns d'intégration définis - [x] Considérations de performance adressées

[OK] Patterns d'implémentation - [x] Conventions de nommage établies - [x] Patterns de structure définis - [x] Patterns de communication spécifiés - [x] Patterns de processus documentés

[OK] Structure du projet - [x] Structure complète des répertoires définie - [x] Frontières des composants établies - [x] Points d'intégration mappés - [x] Mapping exigences → structure complet

4.6.5 Évaluation de Préparation

Statut global : PRÊT POUR L'IMPLÉMENTATION

Niveau de confiance : Élevé

Forces clés : - Séparation nette des responsabilités (10 crates, chacune un domaine) - Crates publiables indépendantes (potentiel communautaire) - Patterns explicites pour éviter les divergences entre agents AI - Architecture complète v0.2 prévue sans code mort en v0.1 - Décisions issues de revues collaboratives multi-perspectives (Party Mode)

Points à surveiller : - Performance sans cache (à mesurer en conditions réelles) - Complexité du workspace Cargo (10 crates = temps de compilation) - Réconciliation bancaire (algorithme de matching à affiner avec données réelles)

Gaps v0.2 identifiés (non bloquants) : - Budgets : repository + logique core à ajouter - Justificatifs : stockage filesystem (volume Docker dédié), métadonnées dans kesh-db - Modèles documents : personnalisation templates PDF

4.6.6 Consignes pour les Agents AI

— Suivre toutes les décisions architecturales exactement comme documentées
— Utiliser les patterns d'implémentation de manière cohérente
— Respecter la structure du projet et les frontières entre crates
— Se référer à ce document pour toute question architecturale
— Ne jamais utiliser de f64 pour les montants — `rust_decimal::Decimal` exclusivement

4.6.7 Modifications PRD à Reporter

1. Axum sert le frontend en production (plus nginx par défaut)
2. nginx optionnel (TLS/reverse proxy uniquement)
3. docker-compose simplifié à 2 containers (kesh + mariadb)
4. Le frontend s'affiche même si la DB est inaccessible

Chapitre 5

Partie 3 — Spécification UX

stepsCompleted : - 1 - 2 - 3 - 4 - 5 - 6 - 7 - 8-in-progress inputDocuments : - _bmad-output/planning-artifacts/prd.md - docs/change_request.md - docs/kesh-prd-v0.2.md documentCounts : prd : 2 brief : 0 other : 1 —

Chapitre 6

UX Design Specification Kesh

Author : Guy **Date :** 2026-04-02

6.1 Executive Summary

6.1.1 Vision Projet

Kesh est la première solution de comptabilité suisse open source, gratuite et auto-hébergée. Face aux solutions cloud qui retiennent les données en otage (exportation difficile, dépendance à l'abonnement), aux logiciels desktop vieillissants (Crésus, Banana) et aux outils open source non adaptés au contexte suisse (GnuCash), Kesh offre une alternative moderne, souveraine et accessible.

L'application web (SPA Svelte + API Axum + MariaDB) se déploie via docker-compose sur l'infrastructure de l'utilisateur — NAS, VPS ou serveur local. Les données comptables restent intégralement sous contrôle de l'utilisateur.

6.1.2 Utilisateurs Cibles

Persona	Profil	Compétence comptable	Besoin UX prioritaire
Marc (indépendant)	Graphiste, Lausanne, 5-10 factures/mois	Basique	Efficacité : facturer et réconcilier vite
Sophie (association)	Trésorière bénévole, aucune formation	Aucune	Guidage : comprendre sans se perdre
Thomas (admin)	Informaticien, gère 2 instances	Technique, pas comptable	Contrôle : déployer, configurer, maintenir

Persona	Profil	Compétence comptable	Besoin UX prioritaire
Lisa (fiduciaire)	Comptable pro, 8 clients	Experte	Puissance : raccourcis, batch, efficacité maximale

6.1.3 Principes UX Fondateurs

1. **Simple par défaut, puissant sur demande** — Progressive disclosure. Mode Guidé pour les débutants, mode Expert pour les professionnels. Préconfigurations par type d'organisation qui masquent 80% de la complexité pour 80% des utilisateurs, tout en restant entièrement personnalisables.
2. **L'UX commence avant l'application** — Le `.env` documenté, le guide d'installation PDF, le port configurable, l'URL affichée au démarrage font partie de l'expérience. L'installation est un touchpoint UX.
3. **Chaque erreur est un moment d'apprentissage** — Chaque message d'erreur dit ce qui s'est passé ET ce que l'utilisateur peut faire. Jamais de message cryptique. « Écriture déséquilibrée — le total des débits (500.00) ne correspond pas au total des crédits (0.00) » plutôt que « Erreur 422 ».
4. **Respect du temps et de l'autonomie** — 15 minutes pour être opérationnel. Données sous contrôle. Export facile. Onboarding interruptible et reprenant là où on en était. Pas d'otage, y compris dans l'application elle-même.

6.1.4 Défis UX Clés

1. **Deux profils extrêmes** — L'interface doit être immédiatement compréhensible pour un non-comptable (Sophie) sans ralentir un expert (Lisa). Mode Guidé / Expert comme réponse structurelle.
2. **Démystifier la comptabilité** — Aide contextuelle en langage naturel (« l'argent entre dans ce compte ») couplée à la terminologie comptable correcte. Sophie apprend, Lisa n'est pas ralentie.
3. **Flux multi-étapes complexes** — Import bancaire, réconciliation, facturation QR Bill. Mode guidé pas-à-pas pour les débutants, écran compact avec actions en lot pour les experts. Même fonctionnalité, deux présentations.
4. **Confiance dès le premier contact** — Transparence opérationnelle : logs clairs, `.env` documenté, URL affichée, backup responsabilisé, guide d'installation PDF. Le contraire du cloud opaque.

6.1.5 Opportunités UX

1. **Onboarding comme différenciateur** — Deux chemins : exploration avec données de démo ou production directe. 6 étapes atomiques, chacune persistée immédiatement, interruptible et repris via bannière contextuelle. Opérationnel en moins de 15 minutes.

2. **Mode Guidé / Expert** — Choix à l'onboarding (« Comment préférez-vous utiliser Kesh ? Guidé / Expert »), modifiable à tout moment dans le profil. Impact global sur la présentation des flux, les confirmations, les raccourcis et le niveau de détail.
3. **Séparation langue comptable / langue interface** — La langue comptable (plan comptable, terminologie) est fixée à l'onboarding au niveau de l'instance. La langue d'interface est modifiable par utilisateur. Lisa peut travailler sur un client zurichois (compta en DE) avec son interface en FR.
4. **Souveraineté comme valeur UX** — Export facile, données sur l'infrastructure de l'utilisateur, backup intégré aux outils NAS existants, aucune télémétrie, documentation honnête sur les responsabilités de sauvegarde.

6.1.6 Expérience d'Installation

Fichiers livrés : `docker-compose.yml`, `.env`, `guide-installation.pdf`

`.env` documenté : - Port configurable (`KESH_PORT`) — essentiel pour les NAS avec plusieurs containers - Mot de passe admin initial — changement forcé au premier login - Nom du projet (`COMPOSE_PROJECT_NAME`) — évite les collisions de noms - Variables purement techniques — la configuration métier se fait dans l'application

Guide d'installation PDF (4-5 pages) : - Prérequis, installation en 5 étapes, configuration du `.env` - Vérification (URL, healthcheck, premier login) - Problèmes fréquents (port occupé, container qui ne démarre pas) - Section NAS (Synology, QNAP) - Backup : responsabilisation claire, pas de fausse promesse

Feedback au démarrage : - Logs Docker : `[OK] Kesh est prêt → http://localhost:{KESH_PORT}` - Si services pas prêts : page d'attente élégante, pas d'erreur nginx

6.1.7 Flux d'Onboarding

Chemin A — Exploration : 1. Langue d'interface (FR / DE / IT / EN — noms dans leur langue, sans texte explicatif) 2. Mode d'utilisation (Guidé / Expert) 3. Charger les données de démo → exploration immédiate - Bannière permanente : [PARTIEL] « Instance de démonstration — données fictives » - Réinitialisation complète pour passer en production

Chemin B — Production : 1. Langue d'interface 2. Mode d'utilisation (Guidé / Expert) 3. Type d'organisation (Indépendant / Association / PME) 4. Langue comptable (FR / DE / IT / EN) 5. Coordonnées (nom/raison sociale, adresse, IDE optionnel) 6. Compte bancaire principal (banque, IBAN, QR-IBAN) — bouton « Configurer plus tard » disponible

Principes d'onboarding : - Chaque étape est atomique et persistée immédiatement en base - L'onboarding peut être interrompu et repris à tout moment - Bannière contextuelle : [INFO] « Configuration incomplète — Terminer la configuration » - Kesh fonctionne même avec un onboarding partiel — les fonctionnalités dépendantes des étapes manquantes affichent un message avec lien vers la configuration - L'onboarding est un raccourci vers les paramètres — tout est retrouvable et modifiable dans Administration

6.1.8 Correspondance Onboarding / Administration

Onboarding	Administration
Langue d'interface	Profil → Langue
Mode Guidé/Expert	Profil → Mode d'utilisation
Type d'organisation	Paramètres → Organisation
Langue comptable	Paramètres → Comptabilité
Coordonnées	Paramètres → Organisation
Compte bancaire	Paramètres → Comptes bancaires

6.1.9 Bannières Contextuelles

Bannière	Condition	Disparition
[INFO] Onboarding incomplet	Étapes non terminées	Toutes étapes complétées
[PARTIEL] Mode démo	Données fictives chargées	Après réinitialisation

6.1.10 États Vides

Chaque liste vide affiche un message d'appel à l'action contextuel : - En mode Guidé : explication + suggestion d'ordre + bouton - En mode Expert : bouton d'action uniquement

6.1.11 Navigation

Navigation par activité (vocabulaire utilisateur, pas technique) : - « Facturer » plutôt que « Module Facturation » - « Importer un relevé » plutôt que « Import bancaire CAMT.053 » - « Voir mes comptes » plutôt que « Balance des comptes »

6.1.12 Raccourcis Clavier (v0.1)

- `Ctrl+N` — nouvelle écriture
- `Ctrl+S` — sauvegarder
- `Tab` / `Shift+Tab` — navigation formulaires
- Navigation clavier complète dans les tableaux

6.1.13 Impacts PRD Identifiés

- **FR76** — La langue n'est plus configurée uniquement au niveau de l'instance. Langue comptable = instance (onboarding), langue interface = utilisateur (profil).
- **FR80** — Le script de seed devient aussi un outil d'onboarding (chemin « Exploration »).
- **FR82** — Page d'accueil « minimaliste » précisée : minimaliste ET contextuelle, avec checklist de démarrage pour les nouveaux utilisateurs.

6.2 Core User Experience

6.2.1 Expérience Définissante

L'expérience définissante de Kesh est le **parcours de première adoption** : installation → configuration → première écriture. C'est un entonnoir où chaque étape est un filtre. Si l'utilisateur bloque à l'une d'elles, il n'atteindra jamais l'usage quotidien (facturation, import, réconciliation).

Entonnoir d'adoption :

1. **Installation** (docker-compose) — `.env` documenté, port configurable, URL dans les logs, guide PDF
2. **Configuration du plan comptable** — Pré-chargé selon le type d'organisation. En mode Guidé : « Il est déjà configuré, vous n'avez rien à modifier pour commencer »
3. **Ouverture des comptes** — Choix explicite : « Je pars de zéro » ou « J'ai des soldes à reprendre ». Pas de formulaire imposé si les soldes sont à zéro
4. **Première écriture** — Mode Guidé : assistant pas-à-pas. Mode Expert : formulaire direct avec raccourcis clavier

Chaque étape doit être franchissable sans blocage. L'utilisateur qui arrive à la première écriture validée est converti.

6.2.2 Stratégie Plateforme

- **Web app uniquement** — SPA Svelte + API REST Axum
- **Desktop/laptop exclusivement** — largeur minimale 1280px, pas de responsive mobile
- **Interaction souris + clavier** — optimisée pour la saisie au clavier (Tab, autocomplétion, raccourcis), souris pour la navigation et les actions ponctuelles
- **Pas de mode offline** — connexion au serveur requise (auto-hébergé ou local)
- **Navigateurs** — Chrome, Firefox, Safari, Edge (2 dernières versions)

6.2.3 Interactions Sans Effort

Saisie d'écriture : - Tab entre les champs, flux naturel gauche → droite - Autocomplétion des comptes par numéro ou nom (« 1020 » ou « Banque ») - Validation instantanée de l'équilibre débit/crédit (feedback visuel en temps réel) - En mode Expert : saisie rapide sans lever les mains du clavier

Facturation QR Bill : - Sélection client → ajout de lignes (catalogue ou libre) → valider → PDF dans un nouvel onglet - Minimum de clics entre l'intention et le résultat - QR Code conforme SIX généré automatiquement

Import bancaire : - Glisser-déposer le fichier (ou sélection classique) - Prévisualisation avant import - Détection automatique du format et des doublons - Règles d'affectation appliquées automatiquement - Validation en lot des transactions pré-affectées

Navigation générale : - Pages < 300ms — aucun chargement perceptible - Retour arrière naturel (bouton navigateur fonctionnel) - Aucun cul-de-sac — toujours un chemin de retour ou une action suivante - Recherche globale accessible partout

6.2.4 Moments Critiques de Succès

Moment	Réaction attendue	Risque si échoué
`docker-compose up` → URL affichée	« Ça marche ! »	Abandon immédiat
Choix de langue + onboarding	« C'est dans ma langue, c'est pour moi »	Doute sur la qualité
Plan comptable pré-configuré	« Je n'ai rien à faire, c'est prêt »	Panique face à la complexité
Première écriture validée	« J'ai compris le principe »	« Trop compliqué pour moi »
Première facture QR Bill PDF	« Je peux envoyer ça à mon client »	Retour à l'ancien outil
Premier import réconcilié	« Ça me fait gagner du temps »	« Autant rester sur Excel »

6.2.5 Principes d'Expérience

1. **L'adoption se joue dans la première heure** — De l'installation à la première écriture, chaque étape doit être franchissable sans aide externe. Le chemin « Exploration » avec données de démo raccourcit ce parcours.
2. **Ne jamais bloquer, toujours guider** — Aucune étape obligatoire ne doit être un mur. Plan comptable pré-configuré, soldes à zéro par défaut, aide contextuelle à chaque champ. L'utilisateur avance toujours.
3. **Le clavier est roi pour la saisie** — La comptabilité est un métier de saisie. Tab, autocomplétion, raccourcis. La souris est un complément, pas l'outil principal.
4. **Le résultat visible crée la confiance** — Chaque action produit un feedback immédiat : écriture validée → bilan mis à jour, facture validée → PDF visible, import → transactions listées. Pas d'action silencieuse.

6.3 Desired Emotional Response

6.3.1 Objectifs Émotionnels Primaires

Persona	Émotion cible	Moment déclencheur
Marc	Efficacité sereine	Comptabilité mensuelle bouclée en 30 min
Sophie	Fierté et confiance	Bilan présenté à l'AG, correct et professionnel

Persona	Émotion cible	Moment déclencheur
Thomas	Contrôle et fiabilité	Mise à jour en 5 min, healthcheck au vert
Lisa	Fluidité et puissance	8 clôtures d'exercice sans friction

Émotion différenciante : La souveraineté — « Mes données sont chez moi, je ne dépends de personne, je peux partir quand je veux (mais je n'en ai pas envie). »

6.3.2 Cartographie du Parcours Émotionnel

Étape du parcours	Émotion visée	Design qui la soutient
Découverte (README, site)	Curiosité + crédibilité	« Open source, gratuit, suisse »
Installation	Soulagement → satisfaction	`docker-compose up` → URL en 2 min
Onboarding	Bienvenue + simplicité	« C'est dans ma langue, c'est pour moi »
Exploration démo	Compréhension + projection	« Je vois comment ça marche »
Première écriture	Accomplissement	« J'ai compris le principe débit/crédit »
Première facture QR Bill	Wow + utilité concrète	PDF pro dans un nouvel onglet
Premier import bancaire	Gain de temps	Réconciliation semi-automatique
Usage quotidien	Routine fluide	Raccourcis, autocomplétion, < 300ms
Erreur utilisateur	Apprentissage (pas honte)	Message clair + action suggérée
Clôture d'exercice	Fierté + accomplissement	Bilan/résultat en PDF, chiffres justes
Retour après absence	Retrouvailles faciles	Interface familière, données intactes

6.3.3 Micro-Émotions Critiques

À cultiver : - **Confiance** → Chaque calcul est juste (rust_decimal). Le bilan s'équilibre toujours. Les PDF sont conformes SIX. - **Compétence** → L'utilisateur apprend en utilisant. Les tooltips transforment l'ignorance en savoir, pas en honte. - **Autonomie** → Tout est modifiable, tout est exportable. L'onboarding est un raccourci, pas un passage obligé. - **Sérénité** → Pas de surprises. Pas de données qui disparaissent. Pas de mise à jour forcée. Pas d'abonnement qui expire.

À éliminer : - **Panique** → Plan comptable pré-configuré, mode Guidé, pas de terminologie sans explication - **Sentiment d'otage** → Export facile, données locales, open source, aucune dépendance externe - **Honte** → Messages d'erreur qui disent quoi faire, pas qui blâment. « Écriture déséquilibrée » pas « Erreur de saisie » - **Abandon** → Entonnoir d'adoption sans mur. Chaque étape skippable ou reportable. Données de démo pour explorer sans risque - **Méfiance** → Auto-hébergé, zéro télémétrie, documentation honnête sur les backups - **Frustration** → Performance < 300ms, navigation clavier, aucun cul-de-sac

6.3.4 Implications Design

Émotion visée	Choix UX
Confiance	Feedback visuel immédiat sur chaque action (balance en temps réel, toast de confirmation)
Compétence	Tooltips contextuels bilingues (langage naturel + terme comptable)
Autonomie	Tout paramètre modifiable après l'onboarding, export accessible partout
Sérénité	Confirmations avant actions destructives, bannières au lieu de modals quand possible
Fierté	PDF professionnels, rapports bien formatés, formats suisses corrects
Efficacité	Raccourcis clavier, autocomplétion, actions en lot, mode Expert

6.3.5 Principes de Design Émotionnel

1. **La confiance se construit par la précision** — Chaque montant affiché est exact. Chaque PDF est conforme. L'utilisateur n'a jamais besoin de vérifier si Kesh a bien calculé.
2. **L'apprentissage remplace la honte** — Quand l'utilisateur se trompe, le système explique et guide. Le vocabulaire des messages est neutre et orienté action, jamais accusateur.
3. **La souveraineté est un sentiment, pas juste une fonctionnalité** — Chaque interaction rappelle subtilement que l'utilisateur est maître : ses données, son serveur, ses exports, son rythme.
4. **Le calme est un luxe** — Pas de notifications intrusives, pas de clignotements, pas d'urgence artificielle. La comptabilité est stressante par nature — Kesh ne doit pas en rajouter.

6.4 UX Pattern Analysis & Inspiration

6.4.1 Analyse des Produits Inspirants

Bexio (comptabilité suisse cloud) : - Facturation et paiements fluides — le « core loop » quotidien fonctionne bien - Flux de paiement en deux temps : créer au fil de l'eau, envoyer en lot le jour de paiement — pattern naturel et efficace - Scan de factures fournisseurs envoyé à Bexio pour traitement — bon concept, mais données chez un tiers - Configuration dispersée dans de multiples menus — difficile de trouver où paramétrer - Messages d'erreur cryptiques (configuration comptes bancaires) - Saisie d'écritures plus complexe que la facturation - Export impossible en une fois — compte par compte, sans pièces jointes. Captivité délibérée des données. - Licence coûteuse pour un usage partiel

Odoo (ERP modulaire) : - Onboarding guidé efficace — bonne première impression - Complexité exponentielle avec les modules - Leçon : la simplicité doit tenir dans le temps, pas juste le premier jour

Todoist (gestion de tâches) : - Interface épurée, usage quotidien sans fatigue - Chaque action est rapide et prévisible - Stabilité de l'interface entre versions — confort de la familiarité - Modèle de « routine fluide » à transposer au contexte comptable

Paperless-ngx (gestion documentaire) : - Apprentissage local par classificateur bayésien — le système observe les classements manuels et apprend à suggérer - OCR via Tesseract, 100% local, aucune donnée externe - Plus on l'utilise, meilleur il devient — rétention par la valeur, pas la captivité - Modèle directement transposable aux règles d'affectation comptable

6.4.2 Patterns UX Transférables

Pattern unifié « Sélectionner → Vérifier → Valider → Résultat » :

Les trois flux principaux partagent la même structure :

Flux	Sélectionner	Vérifier	Valider	Résultat
Facturer	Client + lignes	Récapitulatif	Confirmer	PDF QR Bill
Payer	Factures en attente	Montants + IBAN	Confirmer	pain.001
Importer	Fichier bancaire	Prévisualisation	Confirmer	Écritures

L'utilisateur apprend ce pattern une fois et le retrouve partout.

Flux de paiement en deux temps (inspiré Bexio) :

- Temps 1 (au fil de l'eau) : facture reçue → créer un paiement → ajouté à la liste d'attente
- Temps 2 (jour de paiement) : sélectionner → vérifier → générer pain.001 → upload e-banking
- Sépare la décision individuelle de l'action groupée

Dossier inbox pour factures fournisseurs :

- Volume Docker monté (`./inbox/`) — Marc dépose ses scans/PDF
- Kesh détecte les nouveaux fichiers → liste « Factures à traiter »
- Split view : aperçu du scan à gauche, formulaire à droite
- Extraction automatique par priorité :
  1. QR Code détecté → extraction complète (v0.1)
  2. Tesseract OCR local → extraction texte (post-MVP)
  3. Classificateur bayésien → suggestion de compte (post-MVP)
  4. Aucune extraction → saisie manuelle
- Indicateur de source : [OUI] QR Code / [PARTIEL] IA locale / ○ Manuel

Apprentissage local à la Paperless-ngx :

- Le système observe les affectations manuelles de l'utilisateur
- Après N affectations similaires, il commence à suggérer
- Confiance croissante affichée à l'utilisateur
- 100% local — classificateur bayésien, pas de LLM, pas de cloud
- Le modèle appris fait partie des données utilisateur (exportable)

Hiérarchie de navigation par fréquence d'usage :

- Quotidien/hebdomadaire (accès direct) : Facturer, Paiements, Import relevé
- Mensuel (accès secondaire) : Réconciliation, Écritures manuelles, Rapports
- Rare (paramètres) : Plan comptable, Configuration, Export

Configuration centralisée (Anti-Bexio) :

- Un seul menu « Paramètres » avec sous-sections claires
- Pas de chasse au trésor dans N menus dispersés

Export global en un clic (Anti-Bexio) :

- Bouton visible dans le menu principal, pas caché dans les paramètres
- ZIP complet : CSV par table + pièces justificatives + métadonnées
- Message de souveraineté : la sortie est toujours accessible

6.4.3 Anti-Patterns à Éviter

Anti-pattern	Observé dans	Pourquoi l'éviter
Configuration dispersée	Bexio	L'utilisateur ne trouve pas où paramétrer
Messages d'erreur cryptiques	Bexio	L'utilisateur ne sait pas quoi corriger
Export verrouillé / par compte	Bexio	Captivité délibérée des données
Complexité exponentielle	Odoo	L'utilisateur perd le contrôle
Interface instable	Apps diverses	Perte de repères, méfiance

Anti-pattern	Observé dans	Pourquoi l'éviter
IA externe sans consentement	Apps diverses	Violation de la souveraineté

6.4.4 Stratégie d'Inspiration Design

À adopter : - Flux facturation/paiement Bexio — benchmark suisse connu - Flux en deux temps pour les paiements — naturel et efficace - Épure et stabilité Todoist — routine quotidienne sans fatigue - Apprentissage local Paperless-ngx — intelligence sans compromis

À adapter : - Configuration Bexio → centraliser au lieu de disperser - Onboarding Odoo → garder le guidage, contenir la complexité - Inbox Paperless-ngx → transposer au contexte factures fournisseurs

À éviter : - Export verrouillé (Bexio) → export global accessible - Messages cryptiques (Bexio) → messages humains avec action - Modules explosifs (Odoo) → produit unique, complexité stable - IA opaque → transparence sur la source des suggestions

6.4.5 Architecture Intelligence Locale

Couche	Technologie	Données	Phase
Extraction QR Code	Lib Rust locale	Aucune sortie	v0.1
OCR documents	Tesseract local	Aucune sortie	Post-MVP
Classification	Bayésien local	Aucune sortie	Post-MVP
Conseil comptable	LLM externe	Consentement requis	Post-MVP

6.5 Design System Foundation

6.5.1 Choix du Design System

shadcn-svelte + Tailwind CSS

Système de composants copiables (pas de dépendance externe) basé sur Bits UI (primitives accessibles pour Svelte) et stylé avec Tailwind CSS.

Stack technique : - shadcn-svelte (composants stylés) → Bits UI (composants accessibles) → Melt UI (primitives headless) - Tailwind CSS pour le styling utilitaire - Composants copiés dans le projet — aucune dépendance runtime

6.5.2 Justification du Choix

Critère	Évaluation
Svelte-natif	Composants conçus pour Svelte, pas un portage React
Contrôle total	Composants copiés dans le projet, modifiables librement
Accessibilité	Bits UI intègre WCAG AA (navigation clavier, ARIA, contraste)
Productivité	Tailwind CSS + Claude Code = itération rapide
Licence	MIT — compatible EUPL 1.2
Pas de lock-in	Aucune dépendance runtime, le code est dans le projet
Composants métier	Tables, formulaires, modals, toasts, dropdowns, date pickers
Pérennité	Même si le projet upstream ralentit, Kesh n'est pas impacté

6.5.3 Approche d'Implémentation

Phase 1 — Fondations (début du développement) : - Installation Tailwind CSS + configuration des design tokens - Import des composants shadcn-svelte de base : Button, Input, Select, Table, Dialog, Toast, Tooltip, DropdownMenu - Définition du thème Kesh

Phase 2 — Composants métier : - Composants spécifiques Kesh construits sur les primitives shadcn : formulaire de saisie d'écriture, split view factures, tableau de réconciliation - Composants de navigation : menu principal, breadcrumb, recherche globale

Phase 3 — Raffinement : - Variantes mode Guidé / Expert sur tous les composants - Thème finalisé, cohérence visuelle vérifiée sur tous les écrans

6.5.4 Stratégie de Personnalisation

Design tokens Kesh :

- **Couleurs — langage visuel fonctionnel :**
  - Vert = positif (revenu, actif, facture payée, import réussi)
  - Rouge = négatif (dépense, erreur, facture en retard)
  - Bleu = neutre/informatif (navigation, liens, sélection)
  - Jaune/Orange = attention (avertissement, facture en attente, démo)
  - Gris = désactivé/archivé
  - Ce code couleur correspond aux conventions comptables connues — Lisa le lit sans réfléchir
- **Typographie :**
  - Police système (Inter ou similaire), lisibilité maximale
  - `font-variant-numeric: tabular-nums` obligatoire pour tous les montants — chiffres à largeur fixe pour alignement parfait des colonnes comptables
  - Police tabulaire pour les montants (chiffres alignés au centime)

- **Espacements — adaptatifs selon le mode :**
  - Mode Guidé : espacements généreux (`gap-4`), plus d'air, moins d'information par écran, boutons plus grands avec labels explicites
  - Mode Expert : espacements compacts (`gap-2`), plus d'information par écran, boutons avec icônes et labels en tooltip
- **Tableaux :**
  - Composant central de Kesh. Lignes alternées, tri par colonne, pagination, sélection en lot
  - Montants alignés à droite avec `tabular-nums`
  - Optimisé pour la lisibilité des données comptables

Composants sur mesure à créer : - Formulaire de saisie d'écriture (débit/crédit avec autocomplétion) - Split view factures fournisseurs (aperçu PDF + formulaire) - Tableau de réconciliation bancaire - Bannières contextuelles (onboarding, démo, erreurs) - Indicateur d'équilibre débit/crédit en temps réel - Générateur de PDF QR Bill (prévisualisation)

6.5.5 Stratégie de Test du Design System

Tests unitaires des composants (Vitest + Testing Library) : - Chaque composant shadcn personnalisé a ses tests unitaires - Rendu correct avec différentes props et états - Comportement mode Guidé vs mode Expert - Gestion des cas limites (valeurs vides, texte long, montants négatifs, caractères spéciaux UTF-8) - Validation des formats suisses (montants avec apostrophe, dates dd.mm.yyyy)

Tests d'accessibilité : - Navigation clavier complète sur chaque composant interactif (Tab, Entrée, Échap, flèches) - Attributs ARIA corrects (labels, rôles, états) - Contraste WCAG AA vérifié sur tous les thèmes - Axe-core intégré dans la CI pour détecter les régressions

Tests de sécurité : - XSS : champs de saisie échappent le HTML (fournisseur nommé `<script>` ne s'exécute jamais) - CSRF : token valide sur chaque formulaire de modification - Entrées malveillantes testées sur tous les champs texte

Tests d'intégration : - Formulaire de saisie d'écriture : autocomplétion, validation débit/crédit, soumission - Split view factures : chargement PDF, pré-remplissage, validation - Tableau de réconciliation : sélection en lot, filtres, tri, pagination - Bannières contextuelles : affichage conditionnel, disparition - Verrouillage optimiste : deux sessions modifient la même écriture → modal de conflit

Tests E2E (Playwright) : - Structure par parcours utilisateur : - `marc-independent/` : onboarding, facturation, import, paiements - `sophie-association/` : onboarding, première écriture, réconciliation - `thomas-admin/` : déploiement, gestion utilisateurs - `lisa-fiduciary/` : opérations en lot, clôture d'exercice - Chaque flux testé dans les deux modes (Guidé / Expert) - Onboarding : chemin A (démo) et chemin B (production) - Onboarding interrompu et repris - États vides avec appels à l'action - Navigation clavier sur les flux critiques - Raccourcis clavier fonctionnels - Multilingue : 4 langues sur les écrans principaux - Import bancaire : fichiers SIX officiels, CSV multi-encodages, doublons, rejets partiels

Tests de performance : - Chaque page < 300ms (Lighthouse CI) - Tableaux avec 1000+ lignes : rendu fluide - Génération PDF < 3s

Tests visuels (post-MVP) : - Snapshots visuels des composants clés (régressions CSS) - Vérification du rendu PDF par comparaison visuelle

6.6 Expérience Utilisateur Définissante

6.6.1 L'Interaction Définissante

« Ça fonctionne et c'est gratuit. »

L'expérience définissante de Kesh n'est pas une interaction spécifique — c'est la découverte que tout fonctionne, dès le premier contact, sans contrepartie financière. L'utilisateur installe, explore, et constate : facturation QR Bill, import bancaire, écritures, bilan — tout est là, tout marche, et c'est gratuit.

La démonstration qui convainc : le chemin Exploration de l'onboarding. L'utilisateur charge les données de démo et voit un Kesh fonctionnel — factures, écritures, bilan, PDF. Il peut toucher, explorer, comprendre. La conversion se fait à ce moment : « Si la démo marche aussi bien, le vrai marchera aussi. »

6.6.2 Modèle Mental de l'Utilisateur

D'où il vient : - Bexio / logiciel cloud → « Je paie un abonnement pour 10% des fonctionnalités, et je ne peux pas récupérer mes données facilement » - Crésus / Banana → « Mon logiciel est vieillissant, l'interface date, les mises à jour sont payantes » - Excel → « Je bricole, c'est source d'erreurs, je n'ai pas de bilan propre » - GnuCash → « C'est gratuit mais pas adapté à la Suisse, pas de QR Bill, pas de CAMT.053 »

Ce qu'il attend : - Que ça fonctionne comme ce qu'il connaît (Bexio), en mieux - Que ce soit gratuit — pas de « gratuit mais limité » ou « gratuit pendant 30 jours » - Que ses données soient chez lui, pas chez un tiers - Qu'il n'ait pas besoin d'un diplôme comptable pour s'en servir

Ce qu'il ne cherche PAS : - De l'innovation technologique - Des fonctionnalités qu'il n'a jamais vues - Un ERP complet

6.6.3 Critères de Succès de l'Expérience Core

Critère	Indicateur	Mesure
« Ça marche »	Chemin Exploration → Kesh complet en 3 min	Temps entre docker-compose up et exploration de la démo
« C'est pour la Suisse »	QR Bill conforme, CAMT.053, plan comptable suisse	PDF QR Bill scannable par une banque

Critère	Indicateur	Mesure
« C'est dans ma langue »	Interface et plan comptable dans la langue choisie	4 langues fonctionnelles sans erreur
« Je m'y retrouve »	Navigation intuitive, vocabulaire compréhensible	Sophie peut créer une écriture sans aide externe
« C'est rapide »	Chaque action produit un résultat visible immédiatement	Pages < 300ms, PDF < 3s
« C'est gratuit, vraiment »	Aucune limitation, aucun paywall, aucune surprise	100% des fonctionnalités accessibles sans paiement

6.6.4 Patterns UX : Établis vs Novateurs

Patterns établis (90% de l'interface) : - Saisie comptable débit/crédit — pattern universel de la comptabilité - Flux facturation — identique à Bexio (benchmark suisse) - Tableaux avec tri/filtre/pagination — pattern standard des apps métier - Formulaires CRUD classiques — créer, lire, modifier, archiver - Navigation par menu latéral ou supérieur — pattern SPA standard - Notifications banner/modal — pattern UX standard

Patterns adaptés (innovation dans le familier) : - Flux de paiement en deux temps — inspiré Bexio, optimisé - Inbox factures fournisseurs avec split view — inspiré Paperless-ngx - Mode Guidé / Expert — pas nouveau conceptuellement, mais rare dans la comptabilité - Onboarding avec chemin démo — peu courant dans les logiciels comptables, différenciateur fort

Patterns novateurs (éducation nécessaire) : - Apprentissage local des affectations (post-MVP) — concept nouveau pour les utilisateurs comptables. Nécessite une explication claire : « Kesh apprend vos habitudes pour vous faire gagner du temps » - Extraction QR Code des factures reçues — le geste est nouveau (déposer un PDF, Kesh lit le QR), mais le résultat est familier (formulaire pré-rempli)

6.6.5 Mécanique de l'Expérience Définissante

1. Initiation — « J'installe et j'explore » - `docker-compose up` → URL dans les logs - Premier écran : choix de langue (4 drapeaux) - Chemin A (Exploration) : 3 clics → Kesh avec données de démo

2. Interaction — « Je touche et je comprends » - L'utilisateur explore : ouvre une facture, voit le PDF QR Bill - Regarde le plan comptable → « c'est déjà configuré » - Ouvre le bilan → « les chiffres sont là » - Essaie de créer une écriture → « ça marche comme je m'y attends »

3. Feedback — « Le système me répond » - Chaque action produit un résultat visible (toast, mise à jour, PDF dans un nouvel onglet) - Les erreurs expliquent et guident - La balance débit/crédit se met à jour en temps réel

4. Conversion — « J'adopte » - L'utilisateur réinitialise → chemin B (Production) - L'onboarding en 6 étapes → son Kesh, sa langue, son plan comptable - Première vraie écriture → « c'est parti » - Le message implicite : « Si c'était aussi facile, je reste »

6.7 Visual Design Foundation

6.7.1 Système de Couleurs

Palette — sobre, fonctionnelle, professionnelle :

Basée sur Tailwind Slate/Blue — éprouvée, accessible, cohérente avec le langage visuel fonctionnel défini dans le design system.

Rôle	Couleur	Hex	Usage
Primaire	Bleu ardoise	`#1e40af`	Navigation, boutons principaux, liens
Primaire clair	Bleu ciel	`#3b82f6`	Survol, sélection, focus
Succès	Vert	`#16a34a`	Facture payée, import réussi, balance OK
Erreur	Rouge	`#dc2626`	Écriture déséquilibrée, erreur, retard
Attention	Ambre	`#d97706`	Avertissement, démo, en attente
Info	Bleu clair	`#0ea5e9`	Tooltips, bannières info, onboarding
Fond principal	Blanc	`#ffffff`	Fond de page
Fond secondaire	Gris très clair	`#f8fafc`	Lignes alternées, sidebar
Texte principal	Gris foncé	`#1e293b`	Texte courant
Texte secondaire	Gris moyen	`#64748b`	Labels, descriptions, désactivé
Bordures	Gris clair	`#e2e8f0`	Séparateurs, contours de champs

Ratios de contraste WCAG AA : - Texte principal sur fond blanc : 13.5 :1 (AA+) - Texte secondaire sur fond blanc : 4.6 :1 (AA) - Bouton primaire (blanc sur bleu) : 7.2 :1 (AA+)

6.7.2 Système Typographique

Police unique : Inter

Police système, pas de chargement externe. Excellente lisibilité, support `tabular-nums` natif pour l'alignement des montants.

Élément	Taille	Poids	Particularité
Titres H1	24px	600 semi-bold	Pages principales
Titres H2	20px	600 semi-bold	Sections
Titres H3	16px	600 semi-bold	Sous-sections
Corps	14px	400 normal	Texte courant
Montants	14px	500 medium	`tabular-nums`, aligné à droite
Labels	14px	500 medium	Champs de formulaire
Petit texte	12px	400 normal	Tooltips, métadonnées

Hiérarchie typographique : - Une seule famille de police — cohérence maximale - Différenciation par taille et poids, pas par police - `font-variant-numeric: tabular-nums` sur toutes les colonnes de montants pour un alignement parfait au centime

6.7.3 Espacements & Layout

Système d'espacement adaptatif :

Élément	Mode Guidé	Mode Expert
Unité de base	4px	4px
Gap entre composants	16px (`gap-4`)	8px (`gap-2`)
Padding sections	24px (`p-6`)	16px (`p-4`)
Marge entre sections	32px (`my-8`)	16px (`my-4`)
Hauteur ligne tableau	48px	36px

Structure de page :

- **Header fixe** — logo Kesh, recherche globale, profil utilisateur (langue, mode), bannières contextuelles
- **Sidebar fixe gauche (200-240px)** — navigation par activité, organisée par fréquence d'usage :
  - Quotidien : Accueil, Facturer, Payer, Import
  - Mensuel : Écritures, Réconciliation, Rapports
  - Séparé : Paramètres
- **Zone de contenu fluide** — s'adapte à la largeur disponible
- **Footer discret** — version Kesh, disclaimer légal

Dimensions : - Largeur minimale : 1280px - Optimisé pour : 1440-1920px - Pas de responsive mobile — desktop uniquement

6.7.4 Considérations d'Accessibilité

- **Contraste** — tous les textes respectent WCAG AA (ratio minimum 4.5 :1 pour le texte normal, 3 :1 pour le grand texte)

- **Navigation clavier** — focus visible sur tous les éléments interactifs (outline bleu `#3b82f6`)
- **Zoom** — interface fonctionnelle à 200% de zoom navigateur
- **Couleur seule** — les états ne sont jamais communiqués uniquement par la couleur. Le vert « succès » est accompagné d'une icône ✓, le rouge « erreur » d'une icône ✗, etc.
- **Taille de cible** — zones cliquables minimum 44x44px (mode Guidé), 32x32px (mode Expert)

Chapitre 7

Partie 4 — Epics & Stories

stepsCompleted : - step-01-validate-prerequisites - step-02-design-epics - step-03-create-stories - step-04-final-validation inputDocuments : - _bmad-output/planning-artifacts/prd.md - _bmad-output/planning-artifacts/architecture.md - _bmad-output/planning-artifacts/ux-design-specification.md —

Chapitre 8

Kesh - Epic Breakdown

8.1 Vue d'ensemble

Ce document fournit le découpage complet en epics et stories pour Kesh, en décomposant les exigences du PRD, du UX Design et de l'Architecture en stories implémentables.

8.2 Inventaire des Exigences

8.2.1 Exigences Fonctionnelles

8.2.1.1 Configuration & Onboarding

— FR1 : L'administrateur peut installer Kesh via docker-compose en moins de 15 minutes
— FR2 : L'administrateur peut configurer l'application via des variables d'environnement
— FR3 : L'administrateur peut configurer le nom d'utilisateur et le mot de passe du compte admin initial via des variables d'environnement
— FR4 : L'assistant d'onboarding guide l'utilisateur à travers le type d'organisation, les coordonnées, et la configuration des comptes bancaires
— FR5 : Le système installe automatiquement le plan comptable et les journaux adaptés au type d'organisation choisi
— FR6 : L'administrateur peut configurer la politique de mot de passe
— FR7 : Le système affiche un disclaimer légal « ne remplace pas un fiduciaire »
— FR8 : Le système expose un endpoint de healthcheck (`/health`)

8.2.1.2 Gestion des Utilisateurs & Sécurité

— FR9 : L'administrateur peut créer, désactiver et gérer les comptes utilisateurs
— FR10 : L'administrateur peut attribuer un ou plusieurs rôles à un utilisateur (Admin, Comptable, Consultation)
— FR11 : Le système contrôle l'accès aux fonctionnalités en fonction des rôles (RBAC)

- FR12 : L'utilisateur peut s'authentifier avec un identifiant et un mot de passe
- FR13 : Le système renouvelle silencieusement la session et la termine après 15 minutes d'inactivité (configurable)
- FR14 : L'utilisateur peut changer son propre mot de passe
- FR15 : L'administrateur peut réinitialiser le mot de passe d'un utilisateur
- FR16 : Le système bloque les tentatives de connexion après 5 échecs en 15 minutes, avec un délai de déblocage de 30 minutes
- FR17 : Le système utilise le verrouillage optimiste pour gérer la concurrence entre utilisateurs

8.2.1.3 Plan Comptable & Écritures

- FR18 : L'utilisateur peut charger un plan comptable suisse standard (PME, association, indépendant)
- FR19 : L'utilisateur peut ajouter, modifier et archiver des comptes dans le plan comptable
- FR20 : L'utilisateur peut saisir des écritures en partie double (débit/crédit)
- FR21 : Le système refuse toute écriture déséquilibrée (débit ≠ crédit)
- FR22 : L'utilisateur peut saisir des écritures dans différents journaux (Achats, Ventes, Banque, Caisse, OD)
- FR23 : L'utilisateur peut supprimer des écritures tant que l'exercice est ouvert
- FR24 : Le système interdit la modification et la suppression des écritures d'un exercice clôturé

8.2.1.4 Carnet d'Adresses & Contacts

- FR25 : L'utilisateur peut gérer un carnet d'adresses unifié (personnes et entreprises)
- FR26 : L'utilisateur peut marquer un contact comme client, fournisseur, ou les deux
- FR27 : Le système valide le format et le checksum du numéro IDE (CHE) lorsqu'il est saisi (champ optionnel)
- FR28 : L'utilisateur peut associer des conditions de paiement par défaut à un contact

8.2.1.5 Catalogue Produits/Services

- FR29 : L'utilisateur peut gérer un catalogue de produits/services (nom, description, prix unitaire, taux TVA)
- FR30 : L'utilisateur peut sélectionner des articles du catalogue lors de la création d'une facture

8.2.1.6 Facturation

- FR31 : L'utilisateur peut créer une facture avec des lignes libres ou depuis le catalogue (quantité, prix, TVA)
- FR32 : L'utilisateur peut supprimer une facture en brouillon

— FR33 : L'utilisateur peut valider une facture, ce qui lui attribue un numéro séquentiel définitif
— FR34 : Le système génère un PDF QR Bill conforme aux spécifications SIX 2.2 pour chaque facture validée
— FR35 : L'utilisateur peut configurer le format de numérotation des factures
— FR36 : L'utilisateur peut annuler une facture validée uniquement par la création d'un avoir (v0.2)
— FR37 : Le système gère une séquence de numérotation séparée pour les avoirs (v0.2)
— FR38 : Le PDF s'ouvre dans un nouvel onglet du navigateur

8.2.1.7 Paiements (v0.2)

— FR39 : L'utilisateur peut sélectionner des factures fournisseurs ouvertes pour générer un lot de paiements
— FR40 : Le système génère un fichier pain.001.001.03 conforme au schéma XSD SIX
— FR41 : Le système valide le format IBAN avant la génération du fichier de paiement

8.2.1.8 Import Bancaire & Réconciliation

— FR42 : L'utilisateur peut importer des relevés bancaires aux formats CAMT.053 et CSV
— FR43 : Le système détecte les fichiers déjà importés et les transactions en doublon
— FR44 : Le système propose automatiquement des contreparties pour les transactions connues
— FR45 : L'utilisateur peut créer manuellement une contrepartie pour les transactions inconnues et réconcilier
— FR46 : Le système propose la création d'une règle d'affectation après une affectation manuelle
— FR47 : L'utilisateur peut gérer les règles d'affectation automatique
— FR48 : L'utilisateur peut éclater manuellement une transaction agrégée en sous-lignes
— FR49 : Le système extrait les sous-transactions (`TxDtls`) des fichiers CAMT quand elles sont présentes
— FR50 : L'utilisateur peut importer un relevé lié à un compte bancaire spécifique (multi-comptes)
— FR51 : Le système gère le rejet partiel d'un import avec un listing détaillé des erreurs
— FR52 : Le parseur CSV supporte UTF-8 et ISO-8859-1 avec détection automatique
— FR53 : L'utilisateur peut configurer des profils de format par banque pour l'import CSV

8.2.1.9 TVA (v0.2)

— FR54 : L'administrateur peut configurer les taux de TVA avec des dates de validité

— FR55 : Le système applique les arrondis TVA au centime par ligne (arrondi commercial)
— FR56 : L'utilisateur peut générer un rapport TVA par période (trimestriel/semestriel)

8.2.1.10 Budgets (v0.2)

— FR57 : L'utilisateur peut créer un budget annuel par compte individuel avec des montants mensuels
— FR58 : L'utilisateur peut gérer plusieurs versions d'un budget par exercice (initial, révisé)
— FR59 : L'utilisateur peut générer un rapport comparatif budget vs réalisé avec écarts

8.2.1.11 Clôture & Exercice (v0.2)

— FR60 : L'utilisateur peut clôturer un exercice comptable
— FR61 : Le système reporte automatiquement les soldes vers le nouvel exercice
— FR62 : L'utilisateur peut saisir un bilan d'ouverture (soldes de départ)

8.2.1.12 Pièces Justificatives (v0.2)

— FR63 : L'utilisateur peut attacher des fichiers (PDF, images) à une écriture
— FR64 : Le système stocke les justificatifs dans un volume de stockage dédié, séparé de la base de données

8.2.1.13 Rapports & Exports

— FR65 : L'utilisateur peut générer un bilan, un compte de résultat, une balance des comptes et des journaux
— FR66 : L'utilisateur peut exporter les rapports en PDF et CSV
— FR67 : Les PDF générés respectent les formats suisses (apostrophe séparateur de milliers, dates dd.mm.yyyy)
— FR68 : L'utilisateur peut exporter l'ensemble des données par table en CSV
— FR69 : L'utilisateur peut rechercher des écritures par montant, libellé, numéro de facture ou date
— FR70 : L'utilisateur peut trier, filtrer et paginer toutes les listes

8.2.1.14 Interface & Aide

— FR71 : Le système affiche des notifications banner pour les succès, avertissements et erreurs non-bloquantes
— FR72 : Le système affiche des modals pour les erreurs bloquantes (session expirée, conflit de version)
— FR73 : Le système fournit des tooltips contextuels sur les termes comptables et champs techniques
— FR74 : L'utilisateur peut accéder à 3 manuels PDF embarqués (démarrage, utilisateur, administrateur) (v0.2)

8.2.1.15 Internationalisation

- FR75 : Le système est disponible en français, allemand, italien et anglais
- FR76 : La langue de l'interface est configurée au niveau de l'instance. Elle est découplée de la langue des données comptables (libellés, noms de comptes). L'utilisateur peut changer la langue de l'interface sans impacter les données saisies.

8.2.1.16 Déploiement & Maintenance

- FR77 : Le système se déploie via une seule commande et un fichier de configuration (reverse proxy + application + base de données)
- FR78 : Le système détecte une nouvelle version au démarrage et avertit de faire un backup avant migration
- FR79 : Le système applique les migrations de schéma automatiquement
- FR80 : Le système fournit un script de seed rechargeable pour la démo et les tests

8.2.1.17 Modèles de Documents (v0.2)

- FR81 : L'utilisateur peut personnaliser les modèles de documents générés (factures, rapports) : logo, coordonnées, textes de pied de page

8.2.1.18 Page d'Accueil

- FR82 : Le système affiche une page d'accueil après connexion avec accès rapide aux fonctions principales (dernières écritures, factures ouvertes, soldes des comptes bancaires)

8.2.1.19 Comptes Bancaires

- FR83 : L'utilisateur peut configurer et gérer ses comptes bancaires (nom, IBAN, banque)
- FR84 : L'assistant d'onboarding propose la configuration des comptes bancaires avec validation QR-IBAN/IBAN

8.2.1.20 Lettrage (v0.2)

- FR85 : L'utilisateur peut lettrer des écritures entre elles pour marquer les correspondances (facture ↔ paiement)
- FR86 : L'utilisateur peut délettrer des écritures précédemment lettrées tant que l'exercice est ouvert

8.2.1.21 Versioning des parseurs (v0.2)

- FR87 : Le système identifie la version du format utilisé lors de l'import (CAMT.053, pain.001) et sélectionne le parseur/générateur correspondant

8.2.1.22 Traçabilité & Audit

— FR88 : Le système enregistre un journal d'audit des actions utilisateurs sur les données comptables (création, modification, suppression d'écritures, clôture d'exercice) avec l'identifiant utilisateur et l'horodatage

8.2.1.23 Résilience Frontend

— FR89 : Le frontend s'affiche même si la base de données est inaccessible, avec un message d'erreur explicite invitant à vérifier l'état du serveur

8.2.2 Exigences Non-Fonctionnelles

8.2.2.1 Performance

— NFR-PERF-1 : Les pages se chargent en moins de 300ms
— NFR-PERF-2 : L'import d'un relevé mensuel (200 transactions) s'exécute en moins de 2s
— NFR-PERF-3 : La génération d'un PDF (facture QR Bill, rapport) s'exécute en moins de 3s
— NFR-PERF-4 : Le système supporte 2-5 utilisateurs simultanés par instance sans dégradation

8.2.2.2 Sécurité

— NFR-SEC-1 : Tous les mots de passe sont hashés avec argon2 ou bcrypt, jamais stockés en clair
— NFR-SEC-2 : Aucune donnée n'est accessible sans authentification JWT valide
— NFR-SEC-3 : La communication est chiffrée via TLS en production (nginx reverse proxy)
— NFR-SEC-4 : Le rate limiting protège l'endpoint de connexion contre le brute-force
— NFR-SEC-5 : Les données comptables ne sont jamais transmises à un tiers (sauf IA externe avec consentement explicite, post-MVP)
— NFR-SEC-6 : Les credentials de base de données ne sont jamais exposés dans les logs ou l'API

8.2.2.3 Fiabilité & Intégrité

— NFR-REL-1 : La balance débit/crédit est garantie correcte à tout moment — aucun écart possible
— NFR-REL-2 : Aucune donnée comptable n'est perdue en cas de crash ou de redémarrage
— NFR-REL-3 : Les fichiers pain.001 générés passent la validation XSD officielle SIX
— NFR-REL-4 : Les QR Bill générés sont conformes aux spécifications SIX 2.2 (dimensions, position, contenu)
— NFR-REL-5 : Les migrations de schéma préservent l'intégrité des données des exercices passés

— NFR-REL-6 : L'arithmétique monétaire utilise exclusivement des types décimaux exacts (`rust_decimal`), jamais de flottants

8.2.2.4 Accessibilité

— NFR-ACC-1 : L'interface s'inspire de WCAG AA (contraste suffisant, navigation clavier, labels sur les formulaires) sans contrainte stricte de conformité
— NFR-ACC-2 : Le zoom navigateur reste fonctionnel à 200%

8.2.2.5 Internationalisation

— NFR-I18N-1 : Toutes les chaînes de l'interface sont externalisées (format Fluent `.ftl`)
— NFR-I18N-2 : Les formats régionaux suisses sont respectés : dates `dd.mm.yyyy`, montants avec apostrophe `1'234.56`
— NFR-I18N-3 : Les PDF générés respectent les mêmes formats régionaux

8.2.2.6 Maintenabilité

— NFR-MAINT-1 : Le code source est documenté selon les meilleures pratiques (doc comments Rust, JSDoc Svelte)
— NFR-MAINT-2 : Aucun code dupliqué (principe DRY)
— NFR-MAINT-3 : Tests unitaires sur toute la logique métier (moteur comptable, TVA, calculs financiers)
— NFR-MAINT-4 : Tests d'intégration sur les parseurs (CAMT.053, QR Bill, pain.001) avec les fichiers de test officiels SIX
— NFR-MAINT-5 : Tests E2E avec Playwright couvrant chaque parcours utilisateur
— NFR-MAINT-6 : L'API est versionnée (`/api/v1/`) dès le premier jour

8.2.2.7 Déploiement

— NFR-DEPLOY-1 : L'application se déploie via une seule commande `docker-compose up`
— NFR-DEPLOY-2 : L'image Docker résultante pèse moins de 100 Mo
— NFR-DEPLOY-3 : Les logs applicatifs sont émis sur stdout/stderr (standard Docker)
— NFR-DEPLOY-4 : Le healthcheck `/health` vérifie la connexion à la base de données

8.2.3 Exigences Additionnelles (Architecture)

8.2.3.1 Starter Template & Infrastructure

— ARCH-1 : Frontend initialisé via SvelteKit SPA (`adapter-static`, `ssr=false`) avec `npx sv create frontend`
— ARCH-2 : Backend organisé en Cargo workspace manuel avec 10 crates (`kesh-core`, `kesh-db`, `kesh-api`, `kesh-reconciliation`, `kesh-i18n`, `kesh-report`, `kesh-seed`, `kesh-import`, `kesh-payment`, `kesh-qrbill`)
— ARCH-3 : Axum sert le frontend en production via `tower-http::ServeDir` (plus nginx par défaut)

- ARCH-4 : TLS non géré par Kesh — HTTP pur, TLS = infrastructure (nginx/Traefik/Caddy)
- ARCH-5 : Docker-compose simplifié à 2 containers : kesh + mariadb (nginx optionnel pour TLS)
- ARCH-6 : Le frontend s'affiche même si la base de données est inaccessible (message d'erreur côté client)
- ARCH-7 : Workflow de développement : Vite dev server + proxy vers Axum pour hot reload frontend
- ARCH-8 : Outils recommandés : `cargo-nextest`, `cargo-deny` (audit licences/sécurité), `cargo-udeps` (dépendances inutilisées)

8.2.3.2 Séparation des Responsabilités

- ARCH-9 : `kesh-core` contient la logique métier pure, sans I/O — types forts, newtypes (Money, Iban, CheNumber), validation métier
- ARCH-10 : `kesh-db` implémente le repository pattern avec SQLx direct — contrôle total sur le SQL
- ARCH-11 : Crates publiables (`kesh-import`, `kesh-payment`, `kesh-qrbill`) avec zéro dépendance interne — types autonomes, conversion via `From/Into` côté kesh-core/api
- ARCH-12 : Migrations SQLx dans `crates/kesh-db/migrations/` — fichiers versionnés, zéro perte de données
- ARCH-13 : `kesh-reconciliation` dédié — matching, règles d'affectation, mutex par compte bancaire

8.2.3.3 Authentification & API

- ARCH-14 : JWT via `jsonwebtoken` crate — access token ~15 min + refresh token UUID opaque en base
- ARCH-15 : Hashing mots de passe avec Argon2id
- ARCH-16 : RBAC hiérarchique : Consultation < Comptable < Admin — chaque rôle hérite du précédent
- ARCH-17 : Rate limiting sur `/api/v1/auth/login` uniquement — compteur en mémoire par IP, middleware tower
- ARCH-18 : API REST préfixe `/api/v1/`, routes kebab-case, pagination offset/limit
- ARCH-19 : Format d'erreur structuré avec code métier + message + details
- ARCH-20 : Sérialisation `serde/serde_json`, champs JSON en camelCase (`#[serde(rename_all = "camelCase")]`)
- ARCH-21 : Montants transmis en string décimal (`"1234.56"`) — jamais de float dans le JSON

8.2.3.4 Frontend Architecture

- ARCH-22 : State management via Svelte stores natifs (writable, derived)
- ARCH-23 : Composants UI via shadcn-svelte (Svelte 5, Tailwind CSS v4) — code copié dans le projet, modifiable
- ARCH-24 : Communication API via `fetch` natif + wrapper léger (JWT, refresh, erreurs)

— ARCH-25 : Formatage montants/dates via `Intl.NumberFormat` / `Intl.DateTimeFormat` (locale `de-CH`)
— ARCH-26 : Organisation frontend par feature (`features/journal-entries/`, `features/invoicing/`, etc.)

8.2.3.5 Patterns d'Implémentation Obligatoires

— ARCH-27 : Jamais de f64 pour les montants — `rust_decimal::Decimal` exclusivement
— ARCH-28 : Toute écriture comptable doit être équilibrée — validation dans `kesh-core` avant persistance
— ARCH-29 : Tout code public documenté — `///` Rust, JSDoc Svelte
— ARCH-30 : Tests unitaires pour toute logique métier — pas de code métier sans test
— ARCH-31 : Verrouillage optimiste sur toute entité modifiable — champ `version` systématique
— ARCH-32 : Erreurs structurées avec code métier — jamais de string d'erreur en dur côté frontend

8.2.3.6 Conventions de Nommage

— ARCH-33 : Base de données — tables snake_case pluriel, colonnes snake_case, FK `{table_singulier}_id`, index `idx_{table}_{colonnes}`
— ARCH-34 : API REST — routes kebab-case pluriel, query params camelCase
— ARCH-35 : Code Rust — structs/enums PascalCase, fonctions/modules snake_case, newtypes PascalCase
— ARCH-36 : Code Svelte/TS — composants PascalCase, routes kebab-case, fonctions/variables camelCase, stores camelCase

8.2.3.7 Stratégie de Test

— ARCH-37 : Tests unitaires — `kesh-core` seul (logique pure, pas d'I/O), co-localisés (`#[cfg(test)] mod tests`)
— ARCH-38 : Tests d'intégration — `kesh-seed` + `kesh-db` (données réalistes en base), dans `crates/{crate}/tests/`
— ARCH-39 : Tests E2E Playwright — `kesh-api` complet avec base seedée, dans `frontend/tests/e2e/`
— ARCH-40 : Tests Svelte co-localisés (`Component.test.ts` à côté de `Component.svelte`)

8.2.3.8 Infrastructure CI/CD

— ARCH-41 : GitHub Actions pour CI/CD (build, tests, clippy, fmt, release Docker image)
— ARCH-42 : Logging via `tracing` crate (écosystème Tokio/Axum), stdout/stderr
— ARCH-43 : Configuration via variables d'environnement (`dotenvy` + fichier `.env`)

8.2.3.9 Séquence d'Implémentation Recommandée

— ARCH-44 : 1. Scaffold workspace Cargo + SvelteKit SPA → 2. `kesh-core` (types, newtypes, validation) → 3. `kesh-db` (repository, migrations, schéma) → 4. `kesh-api` (Axum, auth JWT, RBAC, ServeDir) → 5. Frontend (SvelteKit, shadcn-svelte, wrapper fetch, stores) → 6. Crates métier (import, qrbill, reconciliation, etc.)

8.2.4 Exigences UX Design

8.2.4.1 Design Tokens & Fondation Visuelle

— UX-DR1 : Configurer Tailwind CSS avec les design tokens Kesh : palette fonctionnelle (bleu ardoise `#1e40af` primaire, vert #16a34a succès, rouge #dc2626 erreur, ambre #d97706 attention, bleu clair `#0ea5e9` info), fonds (`#ffffff` principal, `#f8fafc` secondaire), textes (#1e293b principal, #64748b secondaire), bordures (`#e2e8f0`)
— UX-DR2 : Implémenter le système typographique unique basé sur Inter — H1 24px/600, H2 20px/600, H3 16px/600, corps 14px/400, montants 14px/500 avec `tabular-nums`, labels 14px/500, petit texte 12px/400
— UX-DR3 : Implémenter le système d'espacement adaptatif selon le mode — Mode Guidé : `gap-4`, `p-6`, `my-8`, hauteur ligne tableau 48px ; Mode Expert : `gap-2`, `p-4`, `my-4`, hauteur ligne tableau 36px
— UX-DR4 : Définir la structure de page fixe — header fixe (logo, recherche globale, profil), sidebar fixe gauche 200-240px (navigation par activité organisée par fréquence), zone de contenu fluide, footer discret (version, disclaimer)
— UX-DR5 : Appliquer `font-variant-numeric: tabular-nums` sur toutes les colonnes de montants pour alignement parfait au centime

8.2.4.2 Composants Sur Mesure à Créer

— UX-DR6 : Créer le composant formulaire de saisie d'écriture — débit/crédit avec autocomplétion des comptes par numéro ou nom, validation instantanée de l'équilibre, flux Tab gauche→droite, indicateur d'équilibre en temps réel
— UX-DR7 : Créer le composant split view factures fournisseurs — aperçu PDF à gauche, formulaire de saisie à droite, pré-remplissage depuis extraction QR Code
— UX-DR8 : Créer le composant tableau de réconciliation bancaire — matching automatique affiché, sélection en lot, création de contrepartie en 2 clics, filtres et tri, pagination
— UX-DR9 : Créer les bannières contextuelles — onboarding incomplet (bleu), mode démo (jaune), erreurs persistantes ; conditions d'affichage et de disparition définies
— UX-DR10 : Créer l'indicateur d'équilibre débit/crédit en temps réel — feedback visuel immédiat pendant la saisie, vert si équilibré, rouge si déséquilibré
— UX-DR11 : Créer le composant de prévisualisation PDF QR Bill — rendu dans l'interface avant ouverture dans un nouvel onglet
— UX-DR12 : Créer le composant d'import par glisser-déposer — zone de drop pour fichiers bancaires (CAMT.053, CSV), prévisualisation avant import, détection automatique du format

8.2.4.3 Accessibilité

- UX-DR13 : Garantir les ratios de contraste WCAG AA — minimum 4.5 :1 pour texte normal, 3 :1 pour grand texte ; vérifier texte principal sur blanc (13.5 :1), texte secondaire sur blanc (4.6 :1), bouton primaire blanc sur bleu (7.2 :1)
- UX-DR14 : Implémenter la navigation clavier complète — focus visible sur tous les éléments interactifs (outline bleu `#3b82f6`), Tab/Shift+Tab dans les formulaires, navigation clavier dans les tableaux
- UX-DR15 : Garantir que les états ne sont jamais communiqués uniquement par la couleur — vert succès accompagné de icone check, rouge erreur accompagné de icone croix, etc.
- UX-DR16 : Respecter les tailles de cible cliquable — minimum 44x44px en mode Guidé, minimum 32x32px en mode Expert
- UX-DR17 : Interface fonctionnelle à 200% de zoom navigateur
- UX-DR18 : Intégrer axe-core dans la CI pour détecter les régressions d'accessibilité

8.2.4.4 Interaction Mode Guidé / Expert

- UX-DR19 : Implémenter le choix Mode Guidé / Expert à l'onboarding (« Comment préférez-vous utiliser Kesh ? ») avec possibilité de changer à tout moment dans le profil utilisateur
- UX-DR20 : En mode Guidé — espacements généreux, boutons plus grands avec labels explicites, confirmations avant actions, aide contextuelle visible, états vides avec explication + suggestion d'ordre + bouton
- UX-DR21 : En mode Expert — espacements compacts, boutons avec icones et labels en tooltip, raccourcis clavier actifs (`Ctrl+N` nouvelle écriture, `Ctrl+S` sauvegarder), actions en lot, états vides avec bouton d'action uniquement
- UX-DR22 : Appliquer les variantes Mode Guidé / Expert sur tous les composants métier — formulaire d'écriture (assistant pas-à-pas vs formulaire direct), import bancaire (étapes guidées vs écran compact), réconciliation (une par une vs validation en lot)

8.2.4.5 Flux d'Onboarding

- UX-DR23 : Implémenter le Chemin A (Exploration) — 1. Langue d'interface (4 langues, noms dans leur langue), 2. Mode Guidé/Expert, 3. Charger données de démo ; bannière permanente jaune « Instance de démonstration — données fictives » ; réinitialisation complète pour passer en production
- UX-DR24 : Implémenter le Chemin B (Production) — 1. Langue d'interface, 2. Mode Guidé/Expert, 3. Type d'organisation (Indépendant/Association/PME), 4. Langue comptable (FR/DE/IT/EN), 5. Coordonnées (nom/raison sociale, adresse, IDE optionnel), 6. Compte bancaire principal (banque, IBAN, QR-IBAN) avec « Configurer plus tard »
- UX-DR25 : Chaque étape d'onboarding est atomique et persistée immédiatement en base — l'onboarding peut être interrompu et repris via bannière contextuelle bleue « Configuration incomplète — Terminer la configuration »

— UX-DR26 : Kesh fonctionne même avec un onboarding partiel — les fonctionnalités dépendantes des étapes manquantes affichent un message avec lien vers la configuration
— UX-DR27 : L'onboarding est un raccourci vers les paramètres — correspondance explicite : Langue interface → Profil, Mode → Profil, Type orga → Paramètres > Organisation, Langue comptable → Paramètres > Comptabilité, Coordonnées → Paramètres > Organisation, Compte bancaire → Paramètres > Comptes bancaires

8.2.4.6 Navigation & Structure

— UX-DR28 : Navigation par activité avec vocabulaire utilisateur — « Facturer » pas « Module Facturation », « Importer un relevé » pas « Import bancaire CAMT.053 », « Voir mes comptes » pas « Balance des comptes »
— UX-DR29 : Hiérarchie de navigation par fréquence d'usage — Quotidien (accès direct) : Facturer, Paiements, Import relevé ; Mensuel (accès secondaire) : Réconciliation, Écritures manuelles, Rapports ; Rare (paramètres) : Plan comptable, Configuration, Export
— UX-DR30 : Configuration centralisée — un seul menu « Paramètres » avec sous-sections claires, pas de dispersion (anti-pattern Bexio)
— UX-DR31 : Export global accessible — bouton visible dans le menu principal, ZIP complet (CSV par table + pièces justificatives + métadonnées)
— UX-DR32 : Aucun cul-de-sac dans la navigation — toujours un chemin de retour ou une action suivante, bouton navigateur (retour arrière) fonctionnel
— UX-DR33 : Recherche globale accessible partout depuis le header

8.2.4.7 Patterns d'Interaction

— UX-DR34 : Implémenter le pattern unifié « Sélectionner → Vérifier → Valider → Résultat » pour les trois flux principaux — Facturer (Client + lignes → Récapitulatif → Confirmer → PDF QR Bill), Payer (Factures en attente → Montants + IBAN → Confirmer → pain.001), Importer (Fichier bancaire → Prévisualisation → Confirmer → Écritures)
— UX-DR35 : Implémenter le flux de paiement en deux temps — Temps 1 (au fil de l'eau) : facture reçue → créer un paiement → ajouté à la liste d'attente ; Temps 2 (jour de paiement) : sélectionner → vérifier → générer pain.001
— UX-DR36 : Implémenter l'autocomplétion des comptes — par numéro (« 1020 ») ou par nom (« Banque »), dans tous les champs de saisie de compte
— UX-DR37 : Raccourcis clavier v0.1 — `Ctrl`+`N` nouvelle écriture, `Ctrl`+`S` sauvegarder, Tab/`Shift`+Tab navigation formulaires, navigation clavier complète dans les tableaux

8.2.4.8 Messages & Feedback

— UX-DR38 : Chaque message d'erreur dit ce qui s'est passé ET ce que l'utilisateur peut faire — « Écriture déséquilibrée — le total des débits (500.00) ne correspond pas au total des crédits (0.00) » plutôt que « Erreur 422 »

— UX-DR39 : Les tooltips comptables utilisent un langage bilingue — terme en langage naturel (« l'argent entre dans ce compte ») couplé à la terminologie comptable correcte (« Débit »)
— UX-DR40 : Feedback visuel immédiat sur chaque action — écriture validée → bilan mis à jour, facture validée → PDF visible, import → transactions listées ; pas d'action silencieuse
— UX-DR41 : Confirmations avant actions destructives (suppression, clôture) — bannières au lieu de modals quand possible, pour préserver le calme de l'interface

8.2.4.9 Expérience d'Installation

— UX-DR42 : Logs Docker affichent l'URL de l'application au démarrage — « Kesh est pret → http ://localhost :{KESH_PORT} »
— UX-DR43 : Si services pas prêts au premier accès — page d'attente élégante, pas d'erreur nginx brute
— UX-DR44 : Fichiers livrés : `docker-compose.yml`, `.env` documenté (port configurable, mot de passe admin initial, nom du projet), `guide-installation.pdf` (4-5 pages : prérequis, installation 5 étapes, vérification, problèmes fréquents, section NAS, backup)

8.2.4.10 Stratégie de Test UX

— UX-DR45 : Tests unitaires des composants shadcn personnalisés — rendu avec différentes props/états, comportement Mode Guidé vs Expert, cas limites (valeurs vides, texte long, montants négatifs, caractères UTF-8), formats suisses (montants apostrophe, dates dd.mm.yyyy)
— UX-DR46 : Tests E2E Playwright structurés par parcours utilisateur — `marc-independent/` (onboarding, facturation, import, paiements), `sophie-association/` (onboarding, première écriture, réconciliation), `thomas-admin/` (déploiement, gestion utilisateurs), `lisa-fiduciary/` (opérations en lot, clôture)
— UX-DR47 : Chaque flux E2E testé dans les deux modes (Guidé / Expert) + chemin A (démo) et chemin B (production) pour l'onboarding
— UX-DR48 : Tests de performance — chaque page < 300ms (Lighthouse CI), tableaux 1000+ lignes rendu fluide, génération PDF < 3s
— UX-DR49 : Tests de sécurité frontend — XSS (champs échappent le HTML), CSRF (token valide sur formulaires de modification), entrées malveillantes sur tous les champs texte

8.2.5 Carte de Couverture des Exigences

FR	Epic	Description
FR1-FR3	Epic 1	Installation, configuration env, compte admin
FR6	Epic 1	Politique de mot de passe
FR8	Epic 1	Healthcheck

FR	Epic	Description
FR9-FR17	Epic 1	Gestion utilisateurs, auth, RBAC, JWT, rate limiting, verrouillage optimiste
FR4-FR5	Epic 2	Onboarding, plan comptable auto
FR7	Epic 2	Disclaimer légal
FR75-FR76	Epic 2	Internationalisation, découplage langue interface/données
FR80	Epic 2	Script de seed
FR82	Epic 2	Page d’accueil
FR83-FR84	Epic 2	Comptes bancaires, onboarding banque
FR18-FR24	Epic 3	Plan comptable, écritures, journaux, immutabilité
FR69-FR70	Epic 3	Recherche, tri, filtre, pagination
FR71-FR73	Epic 3	Notifications banner/modal, tooltips
FR88	Epic 3	Journal d’audit
FR25-FR28	Epic 4	Carnet d’adresses, contacts, IDE
FR29-FR30	Epic 4	Catalogue produits/services
FR31-FR35	Epic 5	Facturation, QR Bill PDF, numérotation
FR38	Epic 5	PDF nouvel onglet
FR42-FR53	Epic 6	Import CAMT.053/CSV, doublons, réconciliation, règles, profils banque
FR65-FR68	Epic 7	Bilan, résultat, balance, journaux, export PDF/CSV
FR77-FR79	Epic 8	Docker-compose (2 containers), migrations, détection version
FR89	Epic 8	Frontend accessible même si DB inaccessible
FR54-FR56	Epic 9	TVA taux configurables, arrondis, rapport
FR36-FR37	Epic 10	Avoirs, numérotation séparée
FR39-FR41	Epic 10	Paiements pain.001, validation IBAN
FR57-FR59	Epic 11	Budgets annuels, versions, rapport comparatif

FR	Epic	Description
FR60-FR62	Epic 12	Clôture exercice, report soldes, bilan ouverture
FR63-FR64	Epic 13	Justificatifs, stockage filesystem
FR74	Epic 13	Manuels PDF embarqués
FR81	Epic 13	Modèles documents personnalisables
FR85-FR86	Epic 13	Lettrage et délettrage
FR87	Epic 13	Versioning parseurs SIX

Couverture : 89/89 FRs mappées (100%)

8.3 Liste des Epics

8.3.1 v0.1 — Cœur comptable + facturation

8.3.2 Epic 1 : Fondations & Authentification

L'administrateur peut installer Kesh, se connecter et gérer les utilisateurs. Le système d'authentification JWT avec RBAC est opérationnel. L'infrastructure technique (workspace Cargo, SvelteKit SPA, MariaDB, design tokens) est en place. **FRs couvertes :** FR1-FR3, FR6, FR8-FR17 **ARCH :** Scaffold workspace Cargo + SvelteKit, kesh-core (types, newtypes, validation), kesh-db (schema, migrations, repositories), kesh-api (Axum, auth JWT, RBAC, ServeDir), frontend (shadcn-svelte, wrapper fetch, stores) **UX-DR :** UX-DR1-DR5 (design tokens, typographie, espacements, layout, tabular-nums), UX-DR13-DR18 (accessibilité fondation), UX-DR37 (raccourcis clavier base)

8.3.3 Epic 2 : Onboarding & Configuration

L'utilisateur peut configurer son organisation, choisir sa langue, explorer la démo et accéder à une page d'accueil contextuelle. L'internationalisation 4 langues est fonctionnelle. **FRs couvertes :** FR4-FR5, FR7, FR75-FR76, FR80, FR82-FR84 **UX-DR :** UX-DR19-DR27 (mode Guidé/Expert, onboarding chemins A/B, atomicité, bannières), UX-DR28-DR33 (navigation, configuration centralisée, recherche globale), UX-DR42-DR44 (expérience installation)

8.3.4 Epic 3 : Plan Comptable & Écritures

L'utilisateur peut gérer le plan comptable, saisir des écritures en partie double avec validation d'équilibre en temps réel, rechercher et filtrer les écritures. Le journal d'audit trace toutes les actions. **FRs couvertes :** FR18-FR24, FR69-FR70, FR71-FR73, FR88 **UX-DR :** UX-DR6 (formulaire écriture), UX-DR10 (indicateur équilibre), UX-DR36 (autocomplétion comptes), UX-DR38-DR41 (messages, tooltips, feedback)

8.3.5 Epic 4 : Carnet d'Adresses & Catalogue

L'utilisateur peut gérer ses clients et fournisseurs avec validation IDE, et maintenir un catalogue de produits/services pour la facturation. **FRs couvertes :** FR25-FR30

8.3.6 Epic 5 : Facturation QR Bill

L'utilisateur peut créer des factures avec lignes libres ou depuis le catalogue, valider et générer des PDF QR Bill conformes SIX 2.2. **FRs couvertes :** FR31-FR35, FR38 **ARCH :** kesh-qrbill (crate publiable indépendante) **UX-DR :** UX-DR11 (prévisualisation PDF), UX-DR34 (pattern Sélectionner→Vérifier→Valider→Résultat)

8.3.7 Epic 6 : Import Bancaire & Réconciliation

L'utilisateur peut importer des relevés CAMT.053 et CSV, détecter les doublons, réconcilier les transactions avec matching automatique et règles d'affectation évolutives. **FRs couvertes :** FR42-FR53 **ARCH :** kesh-import (crate publiable), kesh-reconciliation (matching, rules, mutex) **UX-DR :** UX-DR7 (split view factures), UX-DR8 (tableau réconciliation), UX-DR12 (glisser-déposer)

8.3.8 Epic 7 : Rapports & Exports

L'utilisateur peut générer bilan, compte de résultat, balance des comptes et journaux en PDF et CSV avec formats suisses. **FRs couvertes :** FR65-FR68 **ARCH :** kesh-report (PDF tabulaire + CSV), kesh-i18n (formatage suisse) **UX-DR :** UX-DR31 (export global)

8.3.9 Epic 8 : Déploiement & Opérations

L'administrateur peut déployer Kesh via docker-compose, les migrations s'appliquent automatiquement et la détection de version avertit avant migration. **FRs couvertes :** FR77-FR79, FR89 **UX-DR :** UX-DR42-DR44 (logs, page d'attente, fichiers livrés)

8.3.10 v0.2 — Complétude comptable

8.3.11 Epic 9 : TVA Suisse

L'utilisateur peut configurer les taux TVA avec historique, appliquer les arrondis commerciaux et générer des rapports TVA par période. **FRs couvertes :** FR54-FR56

8.3.12 Epic 10 : Avoirs & Paiements

L'utilisateur peut créer des avoirs liés aux factures et générer des fichiers de paiement pain.001 conformes SIX. **FRs couvertes :** FR36-FR37, FR39-FR41 **ARCH :** kesh-payment (crate publiable) **UX-DR :** UX-DR35 (flux paiement en deux temps)

8.3.13 Epic 11 : Budgets

L'utilisateur peut créer des budgets annuels par compte, gérer des versions et suivre les écarts budget vs réalisé. **FRs couvertes :** FR57-FR59

8.3.14 Epic 12 : Clôture d'Exercice

L'utilisateur peut clôturer un exercice comptable, reporter les soldes et saisir un bilan d'ouverture. **FRs couvertes :** FR60-FR62

8.3.15 Epic 13 : Justificatifs, Lettrage & Compléments v0.2

L'utilisateur peut attacher des justificatifs aux écritures, lettrer des écritures entre elles, personnaliser les modèles de documents et accéder aux manuels embarqués. **FRs couvertes :** FR63-FR64, FR74, FR81, FR85-FR87

8.4 Epic 1 : Fondations & Authentification

Objectif : L'administrateur peut installer Kesh, se connecter et gérer les utilisateurs. Le système d'authentification JWT avec RBAC est opérationnel. L'infrastructure technique est en place.

8.4.1 Story 1.1 : Scaffold Cargo workspace & SvelteKit

As a développeur **I want** un workspace Cargo structuré et un projet SvelteKit initialisé **So that** je puisse commencer à développer sur des fondations propres

Acceptance Criteria :

- **Given** workspace vide, **When** cargo init pour chaque crate (kesh-core, kesh-db, kesh-api, kesh-reconciliation, kesh-i18n, kesh-report, kesh-seed, kesh-import, kesh-payment, kesh-qrbill), **Then** Cargo.toml racine avec [workspace] members compile sans erreur
- **Given** pas de frontend, **When** npx sv create frontend (Svelte 5, TypeScript, Playwright, adapter-static), **Then** SvelteKit démarre en mode dev avec ssr=false
- **Given** workspace complet, **When** cargo build –workspace, **Then** tous les crates compilent sans erreur
- **And** .gitignore, README.md, et structure de répertoires conforme à l'architecture
- **And** docker-compose.dev.yml avec MariaDB locale pour le développement (kesh-db testable dès cette story)

8.4.2 Story 1.2 : Docker & configuration

As a administrateur **I want** installer Kesh via docker-compose en une commande **So that** l'application soit opérationnelle en moins de 15 minutes

Acceptance Criteria :

- **Given** docker-compose.dev.yml avec 2 containers (kesh + mariadb), **When** docker-compose up, **Then** les 2 containers démarrent et le healthcheck passe au vert
- **Given** .env.example documenté, **When** l'admin copie .env.example vers .env et configure KESH_ADMIN_PASSWORD et KESH_PORT, **Then** l'application démarre avec ces valeurs
- **Given** Dockerfile multi-stage (build Rust + build Svelte → image finale), **When** docker build, **Then** l'image résultante pèse moins de 100 Mo
- **Given** application démarrée, **When** GET /health, **Then** réponse 200 avec statut de connexion DB
- **And** les logs Docker affichent l'URL de l'application au démarrage

8.4.3 Story 1.3 : Types core (newtypes & validation)

As a développeur **I want** des types métier forts avec validation intégrée **So that** l'intégrité des données soit garantie dès la couche logique

Acceptance Criteria :

- **Given** un montant, **When** création de Money(Decimal), **Then** le type utilise rust_decimal exclusivement, jamais de f64
- **Given** un IBAN, **When** création de Iban(String), **Then** le format et le checksum sont validés (retour d'erreur si invalide)
- **Given** un numéro IDE, **When** création de CheNumber(String), **Then** le format CHE-xxx.xxx.xxx et le checksum sont validés
- **Given** un QR-IBAN, **When** création de QrIban(String), **Then** le format QR-IBAN est validé (plage QR-IID 30000-31999)
- **And** chaque type a des tests unitaires couvrant les cas valides et invalides
- **And** documentation /// sur chaque type public

8.4.4 Story 1.4 : Schéma de base & repository pattern

As a développeur **I want** un schéma de base MariaDB et un pattern d'accès aux données **So that** les stories suivantes puissent persister des données

Acceptance Criteria :

- **Given** kesh-db configuré, **When** sqlx migrate run, **Then** les tables users et companies sont créées
- **Given** table users, **Then** colonnes : id, username, password_hash, role, active, version, created_at, updated_at
- **Given** table companies, **Then** colonnes : id, name, address, ide_number, org_type, accounting_language, instance_language, version, created_at, updated_at
- **Given** pool MariaDB configuré, **When** connexion, **Then** le pool SQLx se connecte via la variable DATABASE_URL
- **And** repository pattern implémenté pour users et companies (create, find_by_id, update, list)
- **And** tests d'intégration avec base de données de test

— **And** schéma : table fiscal_years (id, company_id, name, start_date, end_date, status (open/closed), created_at, updated_at) — nécessaire dès les écritures pour le contrôle d'immutabilité post-clôture

8.4.5 Story 1.5 : Authentification (login/logout/JWT)

As a utilisateur **I want** me connecter avec un identifiant et un mot de passe **So that** je puisse accéder à l'application de manière sécurisée

Acceptance Criteria :

— **Given** un utilisateur valide, **When** POST /api/v1/auth/login avec username et password, **Then** réponse 200 avec access_token JWT et refresh_token
— **Given** un access_token valide, **When** requête avec header Authorization : Bearer {token}, **Then** la requête est autorisée
— **Given** un access_token expiré, **When** requête API, **Then** réponse 401
— **Given** un utilisateur connecté, **When** POST /api/v1/auth/logout, **Then** le refresh_token est invalidé en base
— **And** les mots de passe sont hashés avec Argon2id, jamais stockés en clair
— **And** le JWT contient : user_id, role, exp (15 min)

8.4.6 Story 1.6 : Refresh token & gestion de session

As a utilisateur **I want** que ma session se renouvelle silencieusement **So that** je ne sois pas déconnecté pendant que je travaille

Acceptance Criteria :

— **Given** un refresh_token valide, **When** POST /api/v1/auth/refresh, **Then** nouveau access_token + nouveau refresh_token
— **Given** inactivité de 15 minutes (configurable via env), **When** tentative de refresh, **Then** refresh_token expiré, réponse 401
— **Given** 5 tentatives de login échouées en 15 minutes pour une IP, **When** 6ème tentative, **Then** réponse 429 Too Many Requests
— **Given** IP bloquée, **When** attente de 30 minutes, **Then** les tentatives de login sont à nouveau autorisées
— **And** le refresh_token est un UUID opaque stocké en base (table refresh_tokens : id, user_id, token, expires_at)
— **And** un changement de mot de passe ou une désactivation de compte invalide tous les refresh_tokens de l'utilisateur

8.4.7 Story 1.7 : Gestion des utilisateurs (CRUD)

As a administrateur **I want** créer et gérer les comptes utilisateurs **So that** chaque personne ait son propre accès avec le bon niveau de droits

Acceptance Criteria :

— **Given** rôle Admin, **When** POST /api/v1/users avec username, password, role, **Then** l'utilisateur est créé avec le rôle spécifié

- **Given** rôle Admin, **When** PUT /api/v1/users/ :id/disable, **Then** le compte est désactivé et ses refresh_tokens invalidés
- **Given** un utilisateur, **When** PUT /api/v1/auth/password avec ancien et nouveau mot de passe, **Then** le mot de passe est mis à jour et les autres sessions invalidées
- **Given** rôle Admin, **When** PUT /api/v1/users/ :id/reset-password, **Then** un nouveau mot de passe temporaire est défini
- **Given** politique de mot de passe configurée (longueur minimale via env), **When** création ou changement de mot de passe, **Then** la politique est appliquée
- **And** les rôles possibles sont : Admin, Comptable, Consultation
- **And** un utilisateur désactivé ne peut plus se connecter mais son historique d'actions reste intact

8.4.8 Story 1.8 : RBAC & verrouillage optimiste

As a administrateur **I want** que les accès soient contrôlés par rôle et que la concurrence soit gérée **So that** les données soient protégées contre les accès non autorisés et les conflits

Acceptance Criteria :

- **Given** rôle Consultation, **When** tentative de POST/PUT/DELETE sur une ressource protégée, **Then** réponse 403 Forbidden
- **Given** rôle Comptable, **When** opérations CRUD sur les données comptables, **Then** autorisé
- **Given** rôle Comptable, **When** tentative de gestion des utilisateurs, **Then** réponse 403
- **Given** rôle Admin, **When** toute opération, **Then** autorisé (hérite Comptable + gestion users)
- **Given** une entité avec version=3, **When** PUT avec version=3, **Then** mise à jour réussie, version passe à 4
- **Given** une entité avec version=4, **When** PUT avec version=3 (stale), **Then** réponse 409 Conflict avec message explicite
- **And** le middleware RBAC est appliqué sur toutes les routes /api/v1/* (sauf auth)

8.4.9 Story 1.9 : Design system & tokens

As a développeur frontend **I want** un design system Kesh configuré avec les tokens visuels **So that** tous les composants partagent une identité visuelle cohérente

Critères d'acceptation :

- **Given** Tailwind CSS + shadcn-svelte installés, **When** inspection des composants, **Then** les design tokens Kesh sont appliqués (palette : bleu ardoise #1e40af primaire, vert #16a34a succès, rouge #dc2626 erreur, ambre #d97706 attention)
- **Given** typographie, **When** inspection, **Then** Inter est la police unique, avec `font-variant-numeric: tabular-nums` sur les montants
- **Given** mode Guidé, **When** espacements, **Then** gap-4, p-6, my-8, hauteur ligne tableau 48px

— **Given** mode Expert, **When** espacements, **Then** gap-2, p-4, my-4, hauteur ligne tableau 36px
— **And** composants shadcn-svelte de base importés : Button, Input, Select, Table, Dialog, Toast, Tooltip, DropdownMenu

8.4.10 Story 1.10 : Layout & page de login

As a utilisateur **I want** une interface structurée avec une page de connexion **So that** je puisse me connecter et naviguer dans l'application

Critères d'acceptation :

— **Given** le layout, **When** affichage, **Then** header fixe (logo, zone recherche, profil), sidebar fixe gauche (200-240px), zone contenu fluide, footer discret
— **Given** la page de login, **When** saisie username/password et soumission, **Then** appel API login, stockage JWT, redirection vers accueil
— **Given** largeur navigateur, **When** inférieure à 1280px, **Then** l'interface reste fonctionnelle (pas de responsive mobile, mais pas de cassure)
— **And** navigation clavier complète (focus visible outline bleu #3b82f6)
— **And** contraste WCAG AA sur tous les textes (minimum 4.5 :1)

8.4.11 Story 1.11 : Wrapper fetch & accessibilité

As a développeur frontend **I want** un client API robuste et une base d'accessibilité **So that** toutes les communications avec l'API soient fiables et l'interface accessible

Critères d'acceptation :

— **Given** un access_token expiré, **When** le wrapper fetch détecte 401, **Then** refresh automatique du token, si échec → redirection login
— **Given** une erreur API, **When** réponse 4xx/5xx, **Then** le wrapper parse l'erreur structurée et la rend disponible au composant
— **Given** une requête en cours, **When** loading, **Then** variable loading booléenne disponible pour afficher spinner/skeleton
— **And** interface fonctionnelle à 200% de zoom navigateur
— **And** raccourcis clavier : Ctrl+S sauvegarder, Tab/Shift+Tab navigation formulaires
— **And** axe-core configuré pour les tests d'accessibilité

8.5 Epic 2 : Onboarding & Configuration

Objectif : L'utilisateur peut configurer son organisation, choisir sa langue, explorer la démo et accéder à une page d'accueil contextuelle.

8.5.1 Story 2.1 : Internationalisation (i18n) backend

As a utilisateur **I want** utiliser Kesh dans ma langue **So that** l'interface soit dans la langue que je comprends

Critères d'acceptation :

- **Given** kesh-i18n configuré, **When** chargement des fichiers Fluent .ftl pour FR/DE/IT/EN, **Then** toutes les chaînes UI sont disponibles dans les 4 langues
- **Given** langue instance configurée via env (KESH_LANG), **When** démarrage, **Then** l'API expose les traductions dans la langue configurée
- **Given** un montant 1234.56, **When** formatage suisse, **Then** affichage "1'234.56" (apostrophe séparateur de milliers)
- **Given** une date 2026-04-03, **When** formatage suisse, **Then** affichage "03.04.2026" (dd.mm.yyyy)
- **And** les fichiers .ftl sont organisés par locale : fr-CH/, de-CH/, it-CH/, en-CH/
- **And** tests unitaires couvrant le formatage montants et dates pour les 4 langues

8.5.2 Story 2.2 : Flux d'onboarding — Chemin A (Exploration)

As a utilisateur curieux **I want** explorer Kesh avec des données de démo **So that** je puisse comprendre l'application avant de l'utiliser en production

Critères d'acceptation :

- **Given** premier accès à Kesh, **When** affichage, **Then** écran de choix de langue (4 langues, noms dans leur langue, sans texte explicatif)
- **Given** langue choisie, **When** étape suivante, **Then** choix du mode d'utilisation (Guidé / Expert)
- **Given** mode choisi, **When** sélection "Explorer avec des données de démo", **Then** le script de seed charge des données réalistes et l'utilisateur accède à un Kesh fonctionnel
- **Given** mode démo actif, **When** navigation, **Then** bannière permanente jaune "Instance de démonstration — données fictives"
- **Given** mode démo, **When** clic "Réinitialiser pour la production", **Then** toutes les données de démo sont supprimées et l'onboarding Chemin B démarre
- **And** le script de seed (kesh-seed) passe par kesh-core pour respecter les validations métier
- **And** chaque étape est atomique et persistée immédiatement en base

8.5.3 Story 2.3 : Flux d'onboarding — Chemin B (Production)

As a utilisateur **I want** configurer mon organisation pour commencer à travailler **So that** Kesh soit opérationnel avec mes données

Critères d'acceptation :

- **Given** onboarding Chemin B, **When** étape "Type d'organisation", **Then** choix entre Indépendant, Association, PME
- **Given** type choisi, **When** validation, **Then** le plan comptable et les journaux adaptés sont installés automatiquement (FR4, FR5)
- **Given** étape "Langue comptable", **When** choix FR/DE/IT/EN, **Then** la langue comptable est fixée au niveau instance (découplée de la langue interface)
- **Given** étape "Coordonnées", **When** saisie nom/raison sociale, adresse, IDE optionnel, **Then** données persistées, IDE validé si saisi (format CHE + checksum)

— **Given** étape “Compte bancaire”, **When** saisie banque, IBAN, QR-IBAN, **Then** validation IBAN/QR-IBAN, données persistées ; bouton “Configurer plus tard” disponible
— **Given** onboarding interrompu, **When** retour dans l’application, **Then** bannière bleue “Configuration incomplète — Terminer la configuration”
— **And** Kesh fonctionne même avec onboarding partiel — fonctionnalités dépendantes affichent un message avec lien vers la configuration
— **And** chaque étape correspond à une section dans Administration (Profil, Paramètres > Organisation, etc.)
— **And** schéma : table bank_accounts (id, company_id, bank_name, iban, qr_iban, is_primary, version, created_at, updated_at)

8.5.4 Story 2.4 : Page d’accueil & navigation

As a utilisateur **I want** accéder rapidement aux fonctions principales depuis la page d’accueil **So that** je puisse travailler efficacement

Critères d’acceptation :

— **Given** utilisateur connecté, **When** affichage page d’accueil, **Then** affichage des dernières écritures, factures ouvertes, soldes des comptes bancaires (FR82)
— **Given** sidebar, **When** affichage, **Then** navigation par activité organisée par fréquence — Quotidien : Facturer, Paiements, Import ; Mensuel : Écritures, Réconciliation, Rapports ; Rare : Paramètres
— **Given** vocabulaire de navigation, **When** labels, **Then** langage utilisateur (“Facturer” pas “Module Facturation”, “Importer un relevé” pas “Import CAMT.053”)
— **Given** menu Paramètres, **When** affichage, **Then** configuration centralisée avec sous-sections claires (un seul endroit, anti-pattern Bexio)
— **Given** recherche globale, **When** saisie dans le header, **Then** recherche accessible partout
— **And** disclaimer légal “ne remplace pas un fiduciaire” visible (FR7)
— **And** aucun cul-de-sac — toujours un chemin de retour, bouton navigateur fonctionnel

8.5.5 Story 2.5 : Mode Guidé / Expert

As a utilisateur **I want** choisir entre un mode guidé et un mode expert **So that** l’interface s’adapte à mon niveau de compétence

Critères d’acceptation :

— **Given** profil utilisateur, **When** changement du mode (Guidé ↔ Expert), **Then** l’interface s’adapte immédiatement sans rechargement
— **Given** mode Guidé, **When** affichage, **Then** espacements généreux (gap-4, p-6), boutons plus grands avec labels explicites, confirmations avant actions, aide contextuelle visible
— **Given** mode Expert, **When** affichage, **Then** espacements compacts (gap-2, p-4), boutons avec icônes et labels en tooltip, raccourcis clavier actifs (Ctrl+N nouvelle écriture)

— **Given** liste vide en mode Guidé, **When** affichage, **Then** explication + suggestion d'ordre + bouton d'action
— **Given** liste vide en mode Expert, **When** affichage, **Then** bouton d'action uniquement
— **And** le mode est choisi à l'onboarding et modifiable à tout moment dans le profil

8.6 Epic 3 : Plan Comptable & Écritures

Objectif : L'utilisateur peut gérer le plan comptable, saisir des écritures en partie double avec validation d'équilibre en temps réel, et rechercher les écritures.

8.6.1 Story 3.1 : Plan comptable (chargement & gestion)

As a utilisateur **I want** disposer d'un plan comptable suisse et le personnaliser **So that** ma comptabilité soit structurée correctement

Critères d'acceptation :

— **Given** type d'organisation choisi à l'onboarding, **When** chargement plan comptable, **Then** le plan standard correspondant (PME, association, indépendant) est chargé depuis les fichiers JSON dans charts/
— **Given** plan comptable chargé, **When** affichage, **Then** arborescence des comptes avec numéro, nom, type (actif/passif/charge/produit)
— **Given** un compte, **When** ajout d'un nouveau compte, **Then** le compte est créé avec numéro, nom, type, et rattachement au plan
— **Given** un compte existant, **When** modification, **Then** le nom et les attributs modifiables sont mis à jour (verrouillage optimiste)
— **Given** un compte, **When** archivage, **Then** le compte n'apparaît plus dans les sélections mais reste visible dans les écritures existantes
— **And** les comptes sont stockés dans kesh-db avec repository pattern
— **And** schéma : table accounts (id, company_id, number, name, type, parent_id, active, version, created_at, updated_at)
— **And** les fichiers JSON des plans comptables suisses (charts/pme.json, charts/association.json, charts/independant.json) sont créés dans cette story, basés sur les plans comptables standards suisses

8.6.2 Story 3.2 : Saisie d'écritures en partie double

As a utilisateur **I want** saisir des écritures comptables avec validation instantanée **So that** ma comptabilité soit toujours équilibrée

Critères d'acceptation :

— **Given** formulaire d'écriture, **When** saisie, **Then** champs : date, journal (Achats/Ventes/Banque/Caisse/OD), libellé, lignes débit/crédit avec compte et montant
— **Given** champ compte, **When** saisie "1020" ou "Banque", **Then** autocomplétion par numéro ou nom du compte (UX-DR36)

— **Given** lignes saisies, **When** calcul en temps réel, **Then** indicateur d'équilibre affiché — vert si débit=crédit, rouge si déséquilibré (UX-DR10)
— **Given** écriture déséquilibrée (débit ≠ crédit), **When** tentative de validation, **Then** refus avec message explicite : "Écriture déséquilibrée — le total des débits (X) ne correspond pas au total des crédits (Y)" (FR21)
— **Given** écriture équilibrée, **When** validation, **Then** écriture persistée avec numéro séquentiel, toutes les lignes stockées
— **And** navigation Tab gauche→droite entre les champs
— **And** montants en rust_decimal, jamais de f64
— **And** schéma : tables journal_entries (id, company_id, entry_number, date, journal, description, version, created_at, updated_at) et journal_entry_lines (id, entry_id, account_id, debit, credit)

8.6.3 Story 3.3 : Modification & suppression d'écritures

As a utilisateur **I want** modifier ou supprimer mes écritures tant que l'exercice est ouvert **So that** je puisse corriger mes erreurs

Critères d'acceptation :

— **Given** une écriture dans un exercice ouvert, **When** modification, **Then** les champs sont éditables, le verrouillage optimiste s'applique (version)
— **Given** une écriture modifiée, **When** validation, **Then** la balance débit/crédit est re-vérifiée, l'écriture est mise à jour
— **Given** une écriture dans un exercice ouvert, **When** suppression, **Then** l'écriture est supprimée après confirmation
— **Given** une écriture dans un exercice clôturé, **When** tentative de modification ou suppression, **Then** refus avec message : "Impossible de modifier une écriture d'un exercice clôturé" (FR24)
— **And** le journal d'audit enregistre chaque modification et suppression (FR88)

8.6.4 Story 3.4 : Recherche, pagination & tri

As a utilisateur **I want** retrouver facilement mes écritures **So that** je puisse naviguer efficacement dans ma comptabilité

Critères d'acceptation :

— **Given** liste d'écritures, **When** recherche par montant, **Then** les écritures correspondantes sont affichées
— **Given** liste d'écritures, **When** recherche par libellé, **Then** correspondance partielle (LIKE)
— **Given** liste d'écritures, **When** recherche par numéro de facture, **Then** correspondance exacte
— **Given** liste d'écritures, **When** recherche par plage de dates, **Then** filtrage par date début/fin
— **Given** une liste, **When** clic sur un en-tête de colonne, **Then** tri ascendant/descendant
— **Given** une liste longue, **When** pagination, **Then** affichage par pages avec offset/limit, total affiché

— **And** le pattern de recherche/tri/pagination est réutilisable sur toutes les listes (contacts, factures, etc.)
— **And** réponse API format : { “items” : [...], “total” : N, “offset” : 0, “limit” : 50 }

8.6.5 Story 3.5 : Notifications, aide contextuelle & audit

As a utilisateur **I want** être informé clairement de chaque action et comprendre les termes comptables **So that** je puisse utiliser l’application en confiance

Critères d’acceptation :

— **Given** action réussie (saisie, sauvegarde), **When** feedback, **Then** notification banner verte avec message de confirmation (FR71)
— **Given** avertissement (import partiel, doublon détecté), **When** feedback, **Then** notification banner orange avec détails
— **Given** erreur non-bloquante, **When** feedback, **Then** notification banner rouge avec message explicite et action suggérée
— **Given** erreur bloquante (session expirée, conflit de version), **When** feedback, **Then** modal avec explication et actions possibles (FR72)
— **Given** un terme comptable (débit, crédit, bilan), **When** survol ou focus, **Then** tooltip bilingue : terme en langage naturel + terminologie comptable (FR73, UX-DR39)
— **Given** une action sur une donnée comptable, **When** création/modification/suppression, **Then** le journal d’audit enregistre : user_id, action, entité, horodatage (FR88)
— **And** schéma : table audit_log (id, user_id, action, entity_type, entity_id, details_json, created_at)

8.7 Epic 4 : Carnet d’Adresses & Catalogue

Objectif : L’utilisateur peut gérer ses clients et fournisseurs avec validation IDE, et maintenir un catalogue de produits/services pour la facturation.

8.7.1 Story 4.1 : Carnet d’adresses (CRUD contacts)

As a utilisateur **I want** gérer mes clients et fournisseurs dans un carnet d’adresses unifié **So that** je puisse les utiliser pour la facturation et les paiements

Critères d’acceptation :

— **Given** carnet d’adresses, **When** création d’un contact, **Then** saisie : nom/raison sociale, adresse, email, téléphone, type (personne/entreprise)
— **Given** un contact, **When** marquage client/fournisseur, **Then** le contact peut être flagué client, fournisseur, ou les deux (FR26)
— **Given** champ IDE, **When** saisie d’un numéro IDE, **Then** validation format CHE-xxx.xxx.xxx et checksum (champ optionnel) (FR27)
— **Given** un contact, **When** modification, **Then** verrouillage optimiste appliqué (version)
— **Given** liste de contacts, **When** affichage, **Then** recherche, tri, pagination

— **And** schéma : table contacts (id, company_id, name, contact_type, is_client, is_supplier, address, email, phone, ide_number, default_payment_terms, version, created_at, updated_at)
— **And** tests unitaires sur la validation IDE (CHE checksum)

8.7.2 Story 4.2 : Conditions de paiement & catalogue produits/services

As a utilisateur **I want** associer des conditions de paiement à mes contacts et gérer un catalogue **So that** la création de factures soit rapide et cohérente

Critères d'acceptation :

— **Given** un contact, **When** association de conditions de paiement par défaut, **Then** les conditions sont utilisées automatiquement lors de la facturation (FR28)
— **Given** catalogue, **When** création d'un produit/service, **Then** saisie : nom, description, prix unitaire, taux TVA applicable (FR29)
— **Given** catalogue, **When** modification/archivage d'un article, **Then** verrouillage optimiste appliqué
— **Given** création de facture, **When** sélection d'articles du catalogue, **Then** les lignes sont pré-remplies avec nom, prix, TVA (FR30)
— **And** schéma : table products (id, company_id, name, description, unit_price, vat_rate, active, version, created_at, updated_at)
— **And** les conditions de paiement sont un champ texte libre sur le contact (ex : "30 jours net", "10 jours 2% escompte")

8.8 Epic 5 : Facturation QR Bill

Objectif : L'utilisateur peut créer des factures avec lignes libres ou depuis le catalogue, valider et générer des PDF QR Bill conformes SIX 2.2.

8.8.1 Story 5.1 : Création de factures (brouillon)

As a utilisateur **I want** créer une facture avec des lignes de détail **So that** je puisse facturer mes clients

Critères d'acceptation :

— **Given** formulaire facture, **When** création, **Then** sélection du client (depuis carnet d'adresses), date, conditions de paiement (pré-remplies depuis le contact)
— **Given** facture, **When** ajout de ligne libre, **Then** saisie : description, quantité, prix unitaire, taux TVA ; montant calculé automatiquement (FR31)
— **Given** facture, **When** ajout depuis catalogue, **Then** sélection d'un article → ligne pré-remplie, quantité modifiable (FR31)
— **Given** facture en brouillon, **When** suppression, **Then** la facture est supprimée définitivement (FR32)
— **Given** facture en brouillon, **When** modification, **Then** toutes les lignes sont éditables
— **And** schéma : tables invoices (id, company_id, contact_id, invoice_number, status (draft/validated/cancelled), date, due_date, payment_terms, total_amount,

version, created_at, updated_at) et invoice_lines (id, invoice_id, description, quantity, unit_price, vat_rate, line_total)
— **And** le total facture est calculé comme la somme des lignes

8.8.2 Story 5.2 : Validation & numérotation des factures

As a utilisateur **I want** valider une facture pour qu'elle reçoive un numéro définitif **So that** la facture soit officielle et comptabilisée

Critères d'acceptation :

— **Given** facture en brouillon, **When** validation, **Then** un numéro séquentiel définitif est attribué (FR33)
— **Given** facture validée, **When** tentative de modification, **Then** refus — une facture validée est immuable (seul un avoir peut l'annuler)
— **Given** facture validée, **When** vérification, **Then** l'écriture comptable correspondante est générée automatiquement (débit client, crédit produit/service)
— **Given** configuration, **When** paramétrage du format de numérotation, **Then** format configurable (ex : "F-2026-0001", "FACT-001") (FR35)
— **And** le compteur de numérotation est par exercice, sans trou dans la séquence

8.8.3 Story 5.3 : Génération PDF QR Bill

As a utilisateur **I want** générer un PDF QR Bill conforme SIX pour chaque facture validée **So that** je puisse l'envoyer à mon client

Critères d'acceptation :

— **Given** facture validée, **When** génération PDF, **Then** le PDF contient la facture et la partie paiement QR Bill conforme SIX 2.2 (FR34)
— **Given** QR Bill, **When** validation conformité, **Then** dimensions 46×46mm, position A4, police conforme, croix suisse, QR Code scannable
— **Given** QR Bill, **When** contenu QR Code, **Then** les données de paiement sont conformes (IBAN/QR-IBAN, montant, référence, débiteur)
— **Given** PDF généré, **When** affichage, **Then** le PDF s'ouvre dans un nouvel onglet du navigateur (FR38)
— **Given** PDF, **When** génération, **Then** temps < 3 secondes
— **And** kesh-qrbill est une crate publiable indépendante (zéro dépendance sur kesh-core)
— **And** les montants dans le PDF utilisent le format suisse (apostrophe séparateur : 1'234.56)
— **And** tests unitaires avec les fichiers de test SIX officiels (docs/six-references/)

8.9 Epic 6 : Import Bancaire & Réconciliation

Objectif : L'utilisateur peut importer ses relevés bancaires et réconcilier les transactions avec matching automatique et règles d'affectation.

8.9.1 Story 6.1 : Import CAMT.053

As a utilisateur **I want** importer mes relevés bancaires au format CAMT.053 **So that** les transactions apparaissent dans Kesh

Critères d'acceptation :

- **Given** fichier CAMT.053 valide, **When** import, **Then** toutes les transactions sont extraites avec date, montant, référence, détails
- **Given** fichier CAMT.053 avec sous-transactions (TxDtls), **When** import, **Then** les sous-transactions sont extraites individuellement (FR49)
- **Given** import, **When** lié à un compte bancaire, **Then** les transactions sont associées au bon compte (FR50)
- **Given** fichier, **When** glisser-déposer ou sélection, **Then** prévisualisation des transactions avant confirmation d'import
- **Given** parseur, **When** version du format détectée, **Then** le parseur sélectionne la version correspondante (multi-version)
- **And** kesh-import est une crate publiable indépendante (zéro dépendance sur kesh-core)
- **And** types autonomes dans kesh-import, conversion via From/Into vers kesh-core
- **And** tests d'intégration avec les fichiers de test SIX officiels (docs/six-references/)
- **And** schéma : tables bank_imports (id, company_id, bank_account_id, file_hash, filename, imported_at) et bank_transactions (id, import_id, bank_account_id, date, amount, reference, details, status (pending/reconciled), matched_entry_id)

8.9.2 Story 6.2 : Import CSV (multi-encodage & profils banque)

As a utilisateur **I want** importer des relevés CSV de différentes banques **So that** je puisse gérer toutes mes banques dans Kesh

Critères d'acceptation :

- **Given** fichier CSV UTF-8, **When** import, **Then** les transactions sont correctement parsées (FR42)
- **Given** fichier CSV ISO-8859-1, **When** import, **Then** détection automatique de l'encodage et parsage correct (FR52)
- **Given** format CSV inconnu, **When** import, **Then** l'utilisateur peut configurer un profil de format par banque (mapping colonnes) (FR53)
- **Given** profil banque configuré, **When** import suivant de la même banque, **Then** le profil est appliqué automatiquement
- **Given** rejet de lignes, **When** format de date non reconnu, **Then** listing détaillé des erreurs avec numéros de ligne (FR51)
- **And** schéma : table bank_profiles (id, company_id, bank_name, column_mapping_json, date_format, encoding, created_at)

8.9.3 Story 6.3 : Détection de doublons & rejet partiel

As a utilisateur **I want** que Kesh détecte les doublons et gère les imports partiels **So that** aucune transaction ne soit comptée deux fois

Critères d'acceptation :

- **Given** fichier déjà importé (même hash), **When** tentative de réimport, **Then** avertissement "fichier déjà importé" avec option de forcer (FR43)
- **Given** transactions qui chevauchent un import précédent, **When** import, **Then** les doublons sont détectés et signalés — aucune transaction en double
- **Given** fichier avec erreurs partielles, **When** import, **Then** les transactions valides sont importées, les invalides sont rejetées avec listing détaillé (FR51)
- **And** détection de doublons par combinaison : date + montant + référence + compte bancaire

8.9.4 Story 6.4 : Réconciliation & matching automatique

As a utilisateur **I want** que les transactions connues soient automatiquement proposées pour réconciliation **So that** le travail de réconciliation soit minimal

Critères d'acceptation :

- **Given** transactions importées, **When** réconciliation, **Then** le système propose automatiquement des contreparties pour les transactions connues (factures en attente de paiement) (FR44)
- **Given** proposition de matching, **When** affichage, **Then** tableau avec transaction bancaire ↔ écriture comptable proposée, score de confiance
- **Given** propositions, **When** validation en lot, **Then** toutes les propositions acceptées sont réconciliées en une action
- **Given** transaction réconciliée, **When** vérification, **Then** l'écriture comptable correspondante est créée ou liée
- **And** le matching considère : montant exact, référence facture, nom client/fournisseur
- **And** mutex par compte bancaire pour éviter les imports concurrents (kesh-reconciliation)

8.9.5 Story 6.5 : Réconciliation manuelle & règles d'affectation

As a utilisateur **I want** réconcilier manuellement les transactions et créer des règles pour l'avenir **So that** les prochains imports soient plus automatisés

Critères d'acceptation :

- **Given** transaction sans proposition, **When** création manuelle de contrepartie, **Then** l'utilisateur sélectionne le compte comptable et crée l'écriture (FR45)
- **Given** affectation manuelle effectuée, **When** feedback, **Then** le système propose de créer une règle d'affectation automatique (FR46)
- **Given** règles d'affectation, **When** gestion, **Then** CRUD des règles (description contient "X" → compte Y) (FR47)
- **Given** transaction agrégée (plusieurs sous-montants), **When** éclatement, **Then** l'utilisateur peut diviser en sous-lignes avec comptes différents (FR48)
- **And** les règles sont appliquées par priorité lors des imports suivants
- **And** schéma : table import_rules (id, company_id, match_pattern, account_id, priority, active, created_at)

8.10 Epic 7 : Rapports & Exports

Objectif : L'utilisateur peut générer bilan, compte de résultat, balance des comptes et journaux en PDF et CSV avec formats suisses.

8.10.1 Story 7.1 : Rapports comptables (bilan, résultat, balance, journaux)

As a utilisateur **I want** générer mes rapports comptables **So that** je puisse vérifier ma situation financière et préparer ma clôture

Critères d'acceptation :

- **Given** données comptables, **When** génération du bilan, **Then** actifs et passifs affichés par classe de compte, totaux calculés, balance vérifiée (FR65)
- **Given** données comptables, **When** génération du compte de résultat, **Then** charges et produits affichés, résultat net calculé
- **Given** données comptables, **When** génération de la balance des comptes, **Then** tous les comptes avec soldes débit/crédit affichés
- **Given** données comptables, **When** génération des journaux, **Then** écritures listées par journal (Achats, Ventes, Banque, Caisse, OD) avec totaux
- **Given** un rapport, **When** filtrage par période, **Then** seules les écritures de la période sont incluses
- **And** kesh-report génère les données, kesh-i18n formate les montants et dates

8.10.2 Story 7.2 : Export PDF & CSV

As a utilisateur **I want** exporter mes rapports en PDF et CSV **So that** je puisse les partager ou les archiver

Critères d'acceptation :

- **Given** un rapport généré, **When** export PDF, **Then** le PDF respecte les formats suisses : apostrophe séparateur de milliers (1'234.56), dates dd.mm.yyyy (FR67)
- **Given** un rapport, **When** export CSV, **Then** fichier CSV avec séparateur point-virgule, encodage UTF-8
- **Given** menu export, **When** export global par table, **Then** l'utilisateur peut exporter l'ensemble des données (comptes, écritures, contacts, factures) en CSV (FR68)
- **Given** export global, **When** génération, **Then** ZIP contenant un CSV par table + métadonnées
- **Given** export, **When** bouton dans le menu principal, **Then** accessible directement (pas caché dans les paramètres) — souveraineté des données
- **And** génération PDF < 3 secondes
- **And** les messages d'erreur disent ce qui s'est passé ET ce que l'utilisateur peut faire (UX-DR38)

8.11 Epic 8 : Déploiement & Opérations

Objectif : L'administrateur peut déployer Kesh via docker-compose avec migrations automatiques et le frontend reste accessible même si la DB est indisponible.

8.11.1 Story 8.1 : Docker-compose & Dockerfile production

As a administrateur **I want** déployer Kesh en production avec une seule commande **So that** l'application soit opérationnelle rapidement

Critères d'acceptation :

— **Given** docker-compose.yml avec 2 containers (kesh + mariadb), **When** docker-compose up -d, **Then** les deux containers démarrent et communiquent
— **Given** Dockerfile multi-stage (build Rust + build Svelte → image finale Alpine), **When** docker build, **Then** l'image pèse moins de 100 Mo
— **Given** application en production, **When** Axum sert le frontend via tower-http ::ServeDir + API /api/v1/*, **Then** tout est servi par un seul point d'entrée HTTP
— **Given** nginx/Traefik/Caddy en amont (optionnel), **When** reverse proxy TLS configuré, **Then** Kesh fonctionne en HTTP pur derrière le proxy
— **And** les logs sont émis sur stdout/stderr (standard Docker)
— **And** l'image Docker expose le port configurable via KESH_PORT
— **And** cette story concerne le déploiement production — distinct du docker-compose.dev.yml créé en Story 1.2 pour le développement

8.11.2 Story 8.2 : Migrations automatiques & détection de version

As a administrateur **I want** que les migrations s'appliquent automatiquement au démarrage **So that** les mises à jour soient simples et sûres

Critères d'acceptation :

— **Given** nouvelle version de Kesh, **When** démarrage du container, **Then** le système détecte la nouvelle version et affiche "Backup recommandé avant de continuer" (FR78)
— **Given** migrations pendantes, **When** démarrage, **Then** sqlx migrate run s'exécute automatiquement (FR79)
— **Given** migration échouée, **When** erreur, **Then** le système log l'erreur clairement et ne démarre pas (pas de données corrompues)
— **Given** migrations réussies, **When** vérification, **Then** les données des exercices passés restent intactes et lisibles
— **And** les fichiers de migration sont versionnés dans crates/kesh-db/migrations/

8.11.3 Story 8.3 : Résilience frontend (DB inaccessible)

As a utilisateur **I want** voir une page d'erreur claire si la base de données est indisponible **So that** je comprenne le problème sans paniquer

Critères d'acceptation :

— **Given** Axum démarré mais MariaDB indisponible, **When** accès à l'application, **Then** le frontend SPA est servi normalement (fichiers statiques) (FR89)
— **Given** frontend chargé sans DB, **When** appel API, **Then** message d'erreur explicite invitant à vérifier l'état du serveur
— **Given** page d'erreur frontend, **When** affichage, **Then** page d'attente élégante avec message clair, pas d'erreur technique brute (UX-DR43)
— **Given** DB redevenue accessible, **When** refresh, **Then** l'application fonctionne normalement sans redémarrage
— **And** le healthcheck /health retourne 503 quand la DB est inaccessible, 200 quand tout est OK

8.11.4 Story 8.4 : CI/CD GitHub Actions

As a développeur **I want** une pipeline CI/CD automatisée **So that** chaque changement soit validé automatiquement avant merge

Critères d'acceptation :

— **Given** push ou pull request sur le repo, **When** GitHub Actions se déclenche, **Then** le workflow CI exécute : cargo build, cargo test, cargo clippy, cargo fmt –check
— **Given** workflow CI, **When** tests frontend, **Then** npm run check, npm run test (Vitest), npm run build exécutés
— **Given** merge sur main, **When** workflow release, **Then** l'image Docker est buildée et publiée
— **Given** échec de CI, **When** notification, **Then** le développeur voit clairement quel check a échoué
— **And** fichiers dans .github/workflows/ : ci.yml (build + tests) et release.yml (Docker image)
— **And** axe-core intégré dans la CI pour détecter les régressions d'accessibilité (UX-DR18)

8.11.5 Story 8.5 : Guide d'installation PDF

As a administrateur **I want** un guide d'installation clair et concis **So that** je puisse installer Kesh sans aide externe

Critères d'acceptation :

— **Given** guide-installation.pdf, **When** lecture, **Then** contenu de 4-5 pages couvrant : prérequis (Docker, docker-compose), installation en 5 étapes, configuration du .env, vérification (URL, healthcheck, premier login)
— **Given** section problèmes fréquents, **When** consultation, **Then** les cas courants sont documentés : port occupé, container qui ne démarre pas, DB inaccessible
— **Given** section NAS, **When** consultation, **Then** instructions spécifiques pour Synology et QNAP
— **Given** section backup, **When** consultation, **Then** responsabilisation claire : dump MariaDB + volume fichiers, pas de fausse promesse
— **And** le guide est livré avec docker-compose.yml et .env.example (UX-DR44)
— **And** le guide est dans la langue de l'instance

8.12 Epic 9 : TVA Suisse

Objectif : L'utilisateur peut configurer les taux TVA avec historique, appliquer les arrondis commerciaux et générer des rapports TVA par période.

8.12.1 Story 9.1 : Configuration des taux TVA

As a administrateur **I want** configurer les taux de TVA avec des dates de validité **So that** les changements de taux soient gérés correctement dans le temps

Critères d'acceptation :

- **Given** configuration TVA, **When** création d'un taux, **Then** saisie : libellé, pourcentage, date début validité, date fin validité (optionnel) (FR54)
- **Given** taux TVA avec historique, **When** saisie d'une écriture ou facture, **Then** le taux applicable à la date de l'opération est automatiquement sélectionné
- **Given** changement de taux (ex : 7.7% → 8.1%), **When** nouveau taux configuré, **Then** l'ancien taux reste applicable pour les opérations antérieures à la date de bascule
- **Given** taux TVA, **When** affichage, **Then** liste complète avec historique des taux (actifs et expirés)
- **And** schéma : table vat_rates (id, company_id, label, rate, valid_from, valid_to, active, created_at)
- **And** les taux standard suisses sont pré-configurés à l'onboarding (normal, réduit, spécial, exonéré)

8.12.2 Story 9.2 : Calcul TVA & rapport par période

As a utilisateur **I want** que la TVA soit calculée correctement et pouvoir générer un rapport **So that** je puisse remplir mon décompte AFC

Critères d'acceptation :

- **Given** une ligne de facture ou écriture avec TVA, **When** calcul, **Then** arrondi commercial au centime par ligne (FR55)
- **Given** arrondi, **When** montant 123.455, **Then** résultat 123.46 (arrondi commercial : .5 → vers le haut)
- **Given** données comptables, **When** génération rapport TVA trimestriel ou semestriel, **Then** détail par taux : chiffre d'affaires, TVA due, TVA récupérable, solde (FR56)
- **Given** rapport TVA, **When** vérification, **Then** les montants correspondent aux écritures comptables (vérifiable manuellement)
- **And** tous les calculs TVA utilisent rust_decimal (jamais de f64)
- **And** export du rapport en PDF et CSV

8.13 Epic 10 : Avoirs & Paiements

Objectif : L'utilisateur peut créer des avoirs liés aux factures et générer des fichiers de paiement pain.001 conformes SIX.

8.13.1 Story 10.1 : Avoirs (notes de crédit)

As a utilisateur **I want** annuler une facture validée par la création d'un avoir **So that** la comptabilité reste intègre sans supprimer de documents

Critères d'acceptation :

- **Given** facture validée, **When** création d'un avoir, **Then** l'avoir est lié à la facture d'origine avec référence (FR36)
- **Given** avoir créé, **When** validation, **Then** numéro séquentiel attribué depuis une séquence de numérotation séparée (ex : AV-2026-0001) (FR37)
- **Given** avoir validé, **When** comptabilisation, **Then** l'écriture de contre-passation est générée automatiquement (débit produit, crédit client)
- **Given** avoir validé, **When** vérification du solde client, **Then** le solde revient à zéro (facture + avoir = 0)
- **And** un avoir génère un PDF similaire à la facture, avec mention "Avoir" et référence à la facture d'origine

8.13.2 Story 10.2 : Génération de fichiers de paiement pain.001

As a utilisateur **I want** générer un fichier de paiement pour mes factures fournisseurs **So that** je puisse l'uploader dans mon e-banking

Critères d'acceptation :

- **Given** factures fournisseurs ouvertes, **When** sélection pour paiement, **Then** l'utilisateur peut sélectionner une ou plusieurs factures à payer (FR39)
- **Given** sélection effectuée, **When** vérification, **Then** affichage récapitulatif : fournisseur, montant, IBAN pour chaque paiement
- **Given** IBAN invalide pour un fournisseur, **When** validation, **Then** le système signale l'erreur et exclut ce paiement du lot (FR41)
- **Given** validation OK, **When** génération, **Then** fichier pain.001.001.09.ch.03 conforme au schéma XSD SIX (FR40)
- **And** kesh-payment est une crate publiable indépendante (zéro dépendance sur kesh-core)
- **And** tests de validation contre le schéma XSD officiel SIX (docs/six-references/)

8.13.3 Story 10.3 : Flux de paiement en deux temps

As a utilisateur **I want** préparer mes paiements au fil de l'eau et les envoyer en lot **So that** je puisse séparer la décision individuelle de l'action groupée

Critères d'acceptation :

- **Given** facture fournisseur reçue, **When** création d'un paiement, **Then** le paiement est ajouté à la liste d'attente (Temps 1)
- **Given** jour de paiement, **When** sélection des paiements en attente, **Then** l'utilisateur sélectionne, vérifie et génère le fichier pain.001 (Temps 2)
- **Given** le flux, **When** exécution, **Then** le pattern "Sélectionner → Vérifier → Valider → Résultat" est respecté (UX-DR34)

— **And** le fichier généré est disponible pour téléchargement et upload dans l'e-banking (UX-DR35)

8.14 Epic 11 : Budgets

Objectif : L'utilisateur peut créer des budgets annuels par compte, gérer des versions et suivre les écarts budget vs réalisé.

8.14.1 Story 11.1 : Budgets annuels & suivi

As a utilisateur **I want** créer un budget et suivre les écarts avec le réalisé **So that** je puisse piloter mes finances

Critères d'acceptation :

— **Given** exercice comptable, **When** création de budget, **Then** saisie de montants mensuels par compte individuel (FR57)
— **Given** budget existant, **When** création d'une nouvelle version, **Then** le budget peut avoir plusieurs versions par exercice (initial, révisé) (FR58)
— **Given** budget et données comptables, **When** génération rapport comparatif, **Then** affichage par compte : budget mensuel, réalisé mensuel, écart en montant et pourcentage (FR59)
— **Given** rapport budget vs réalisé, **When** filtrage, **Then** par période (mois, trimestre, année) et par compte
— **And** schéma : tables budgets (id, company_id, fiscal_year, version_name, created_at) et budget_lines (id, budget_id, account_id, month, amount)
— **And** export du rapport en PDF et CSV

8.15 Epic 12 : Clôture d'Exercice

Objectif : L'utilisateur peut clôturer un exercice comptable, reporter les soldes et saisir un bilan d'ouverture.

8.15.1 Story 12.1 : Clôture & report des soldes

As a utilisateur **I want** clôturer mon exercice et démarrer le suivant **So that** ma comptabilité soit propre d'année en année

Critères d'acceptation :

— **Given** exercice ouvert, **When** clôture, **Then** confirmation requise (action irréversible), toutes les écritures de l'exercice deviennent immuables (FR60)
— **Given** exercice clôturé, **When** report des soldes, **Then** les soldes des comptes de bilan sont automatiquement reportés dans le nouvel exercice (FR61)
— **Given** exercice clôturé, **When** tentative de modification d'écriture, **Then** refus avec message explicite (lié à FR24)
— **Given** nouvel exercice, **When** saisie du bilan d'ouverture, **Then** l'utilisateur peut saisir ou ajuster les soldes de départ (FR62)

— **Given** clôture, **When** journal d'audit, **Then** l'action est enregistrée avec utilisateur et horodatage
— **And** la clôture est une opération privilégiée (rôle Admin ou Comptable)
— **And** un exercice clôturé ne peut pas être ré-ouvert — seules des contre-passations dans le nouvel exercice sont possibles

8.16 Epic 13 : Justificatifs, Lettrage & Compléments v0.2

Objectif : L'utilisateur peut attacher des justificatifs aux écritures, lettrer des écritures entre elles, personnaliser les modèles de documents et accéder aux manuels embarqués.

8.16.1 Story 13.1 : Pièces justificatives

As a utilisateur **I want** attacher des fichiers à mes écritures **So that** je puisse conserver les justificatifs avec les données comptables

Critères d'acceptation :

— **Given** une écriture, **When** ajout d'un fichier (PDF, image JPG/PNG), **Then** le fichier est uploadé et lié à l'écriture (FR63)
— **Given** fichier uploadé, **When** stockage, **Then** le fichier est stocké dans un volume Docker dédié, séparé de la base de données (FR64)
— **Given** écriture avec justificatif, **When** consultation, **Then** le fichier est téléchargeable/visualisable depuis la vue de l'écriture
— **Given** fichier volumineux, **When** upload, **Then** taille maximale configurée (ex : 10 Mo par fichier)
— **And** les justificatifs sont inclus dans l'export global (ZIP)
— **And** la stratégie de backup doit couvrir le volume de fichiers en plus du dump MariaDB

8.16.2 Story 13.2 : Lettrage

As a utilisateur **I want** lettrer des écritures entre elles **So that** je puisse marquer les correspondances facture ↔ paiement

Critères d'acceptation :

— **Given** deux ou plusieurs écritures, **When** lettrage, **Then** les écritures sont marquées avec un code de lettrage commun (FR85)
— **Given** écritures lettrées, **When** affichage, **Then** le code de lettrage est visible et les écritures liées sont identifiables
— **Given** écritures lettrées dans un exercice ouvert, **When** délettrage, **Then** le lien de lettrage est supprimé (FR86)
— **Given** exercice clôturé, **When** tentative de délettrage, **Then** refus (cohérent avec l'immutabilité post-clôture)
— **And** le lettrage aide à identifier les factures non payées et les paiements non affectés

— **And** schéma : colonne lettering_code sur journal_entry_lines (nullable, même code = même lettrage)

8.16.3 Story 13.3 : Versioning parseurs, modèles & manuels

As a utilisateur **I want** que Kesh gère les versions des formats SIX et me fournisse de la documentation **So that** l'application reste compatible et je puisse me former

Critères d'acceptation :

— **Given** import CAMT.053 ou génération pain.001, **When** détection du format, **Then** le système identifie la version et sélectionne le parseur/générateur correspondant (FR87)
— **Given** modèles de documents (factures, rapports), **When** personnalisation, **Then** l'utilisateur peut modifier : logo, coordonnées, textes de pied de page (FR81)
— **Given** application, **When** accès à l'aide, **Then** 3 manuels PDF embarqués accessibles : guide de démarrage, manuel utilisateur, manuel administrateur (FR74)
— **And** les manuels sont dans la langue de l'instance
— **And** les modèles personnalisés sont stockés en base et appliqués à tous les PDF générés